alcifra

User Manual

A sovereign workspace
— the web workbench and the desktop client

generated 2026-11-11
© 2026
Hebel Consulting GmbH
Schweighofplatz 7
6010 Kriens (LU)
Switzerland
support@alcifra.com

Contents

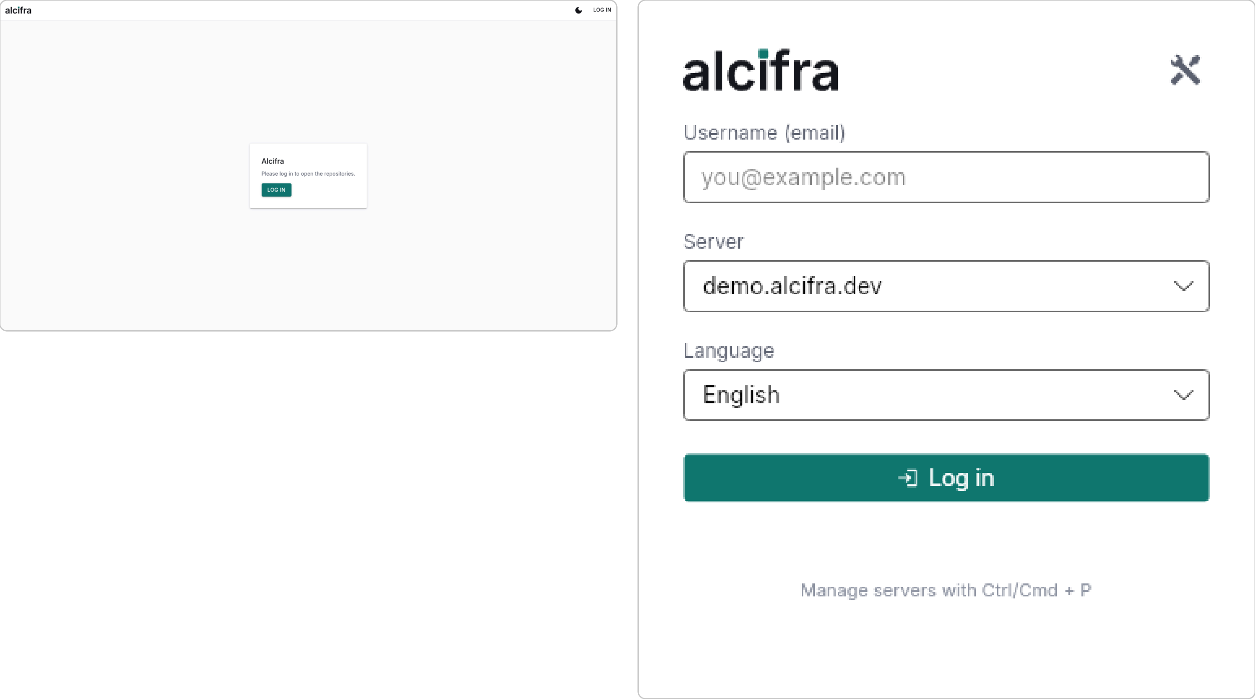
1 Introduction 1
2 Getting started 3
3 Browsing & previewing documents 4
4 Adding & versioning documents 6
5 Organizing 12
6 Working from your file manager (WebDAV) 13
7 Contacts, calendars & your tasks 17
8 Your own device 36
9 Metadata & classification 39
10 Search 40
11 Collaboration 41
12 Sharing outside Alcifra 43
13 Workflow & records 48
14 Administration & account 49
15 Appendix — preparing a smartcard or token 56
16 Appendix — glossary & links 69
17 Appendix: the desktop client’s log 70
18 Appendix — who does what 71
19 Appendix — how Alcifra is put together 74
20 Appendix — security posture 77
21 Index 81

1 Introduction

Alcifra is a new kind of workspace, **sovereign by design**: it unites **document management**, encryption to each person's own key and open protocols, so an organisation keeps its documents, its keys and its choice of devices. It is a showcase of how a senior, AI-driven software developer can produce a complex, enterprise-grade system in a relatively short period — every feature in this manual is really implemented and really tested.

You reach the same archive through **two clients**, which mirror each other feature for feature:

- the **web workbench** — a browser application, nothing to install;
- the **desktop client** — a native Windows / macOS / Linux application that can additionally open a document in its real desktop program and drag documents in and out of the operating-system file manager.



Please log in to open the repositories.

Username (email): you@example.com

Server: demo.alcifra.dev

Language: English

Manage servers with Ctrl/Cmd + P

Figure 1: The two clients: the web sign-in (left) and the desktop logon window (right).

1.1 Core concepts

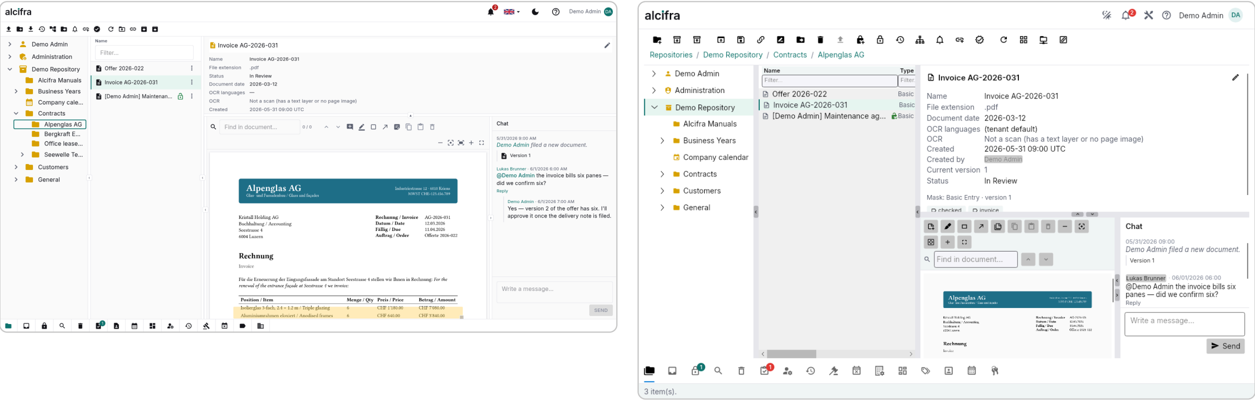
> **Repository** — a top-level archive; it is simply a folder with no parent. **Folder** / **document** — the tree inside a repository. **Version** — every change to a document's file is kept as a new version; nothing is overwritten. **Mask (document type)** — a template of **index fields** (metadata) attached to a document. **ACL** — a per-document access-control list of **rights** (see, read, edit, ...) granted to users and groups.

2 Getting started

Signing in. Open the web client and choose **Log in**, or start the desktop client and use its logon window; enter your e-mail and password. Your organisation may additionally require a second factor (a one-time code or a passkey). You can pick the interface **language** (English, German, Italian, Spanish) and switch between a **light** and **dark** appearance at any time.

If sign-in suddenly refuses everything, you have most likely mistyped several times in a row. Repeated failed attempts are turned away for a short while — about a minute at first, longer if they keep coming — and the wait clears on its own, so there is nobody to ring and nothing to reset. A passkey keeps working throughout, which is the quickest way back in if you have one.

The workbench. After signing in you land on the **Repositories** workbench, laid out as: the **tree** of repositories and folders, the **contents** list of the selected folder, the **detail** pane (index data) over the **preview**, and — along the bottom — the **tab bar** that switches between Repositories, Intray, Search, Tasks and the rest.


alcifra
Demo Admin
Administration
Demo Repository
Alcifra Manuals
Business Years
Company calendar
Contracts
Alpenglas AG
Customers
General
Offer 2026-022
Invoice AG-2026-031
[Demo Admin] Maintenance ag…
Find in document…
Chat
Write a message…
SEND
Repositories / Demo Repository / Contracts / Alpenglas AG
Name
Type
Filter…
Basic
Send
3 item(s).

Invoice AG-2026-031
Name: Invoice AG-2026-031
File extension: .pdf
Status: In Review
Document date: 2026-03-12
OCR languages: —
OCR: Not a scan (has a text layer or no page image)
Created: 2026-05-31 09:00 UTC

5/31/2026 9:00 AM
Demo Admin filed a new document.
Version 1

Lukas Brunner · 6/1/2026 6:00 AM
@Demo Admin the invoice bills six panes — did we confirm six?
Reply

Demo Admin · 6/1/2026 7:00 AM
Yes — version 2 of the offer has six. I'll approve it once the delivery note is filed.

Invoice AG-2026-031
Name: Invoice AG-2026-031
File extension: .pdf
Document date: 2026-03-12
OCR languages: (tenant default)
OCR: Not a scan (has a text layer or no page image)
Created: 2026-05-31 09:00 UTC
Created by: Demo Admin
Current version: 1
Status: In Review
Mask: Basic Entry · version 1
checked · invoice

05/31/2026 09:00
Demo Admin filed a new document.
Version 1

Lukas Brunner · 06/01/2026 06:00
@Demo Admin the invoice bills six panes — did we confirm six?
Reply

Figure 2: The workbench in the web client (left) and the desktop client (right): tree · contents · detail · preview, with the bottom tab bar.

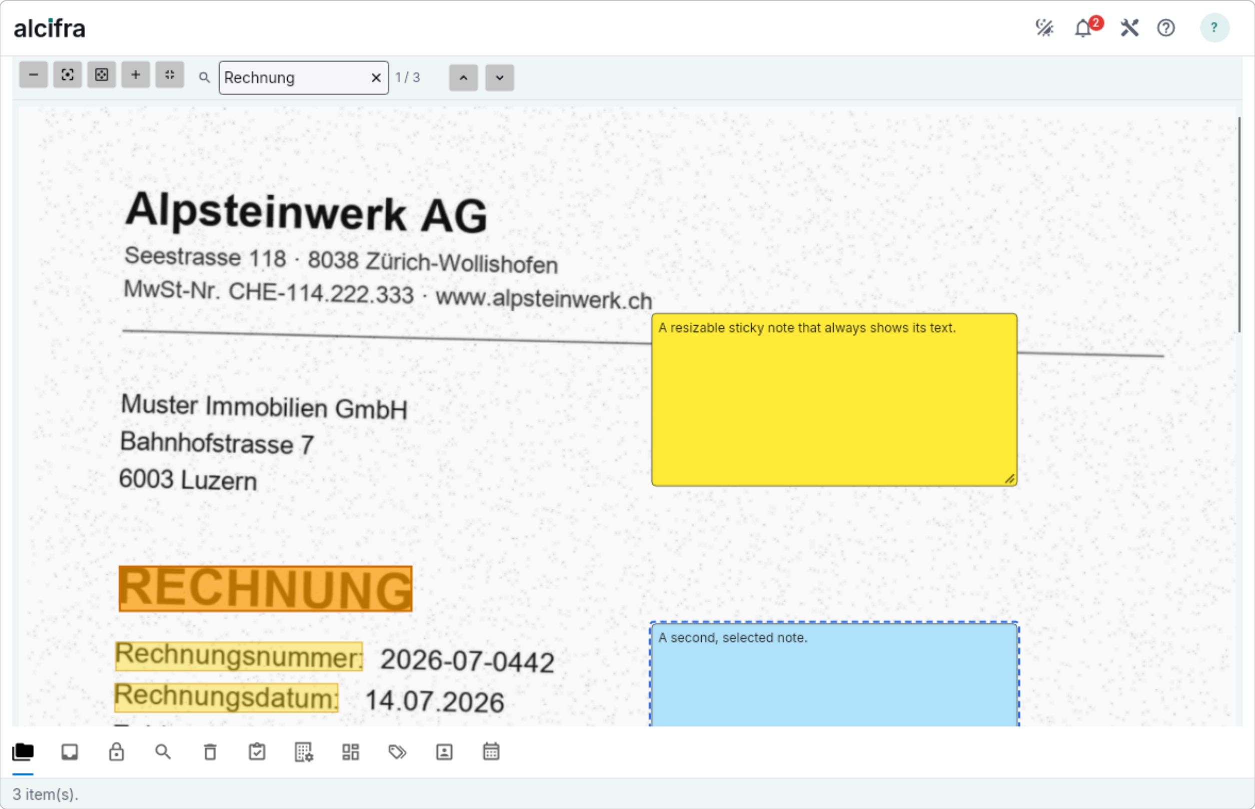
3 Browsing & previewing documents

Expand the **tree** to a folder and its documents appear in the **contents** list, which you can sort by any column. Selecting a document shows its **index data** in the detail pane and renders a **preview** below — PDFs, images, and converted Office/e-mail/Markdown documents alike. Full-text search hits are highlighted directly on the preview, and you can click any word to copy it — **shift-click** appends instead of replacing, so a few clicks collect a phrase (an invoice number, then its date) without touching the keyboard. That is the fast way to fill index fields on the **Intray** tab: click the values in the scan's preview, paste them into the field.

A **contact** or an **appointment** shows as its own card instead: the same form the Contacts and Calendar tabs edit it with, read-only. The pencil above the card opens that form to change it. Only an item the archive filed as a contact or an appointment gets a card; any other .vcf or .ics file keeps its ordinary preview. The card fills the preview. At its bottom, a collapsed **vCard source** or **iCalendar source** shows the item exactly as stored, the same text as the editor's **Advanced: the stored item**, with a long photo shortened; **Copy** copies the whole of it.

A **mail** shows the same way: who sent it, to whom and when, its attachments with **Open** and **Save**, and its text. The web client shows the mail as it was formatted, without anything that could run or call home; the desktop client shows its text. In the desktop client you can also **open** the document in its native application — from the row's context menu, the ribbon, a double-click, or the keyboard shortcut **⌘O** (**Ctrl+O** on Windows and Linux). The same shortcut opens the selected item on the **Intray** tab.

The preview's toolbar carries the **zoom** controls. A document opens fitted to the width of the pane; *fit page* shrinks it until the whole page is visible at once — useful when the pane is wider than it is tall, where fitting the width pushes the bottom of the page out of sight. Zooming out after that walks back down to the whole page and stops there. Ctrl+scroll (⌘+scroll on a Mac) zooms as well.



Alpsteinwerk AG
Seestrasse 118 · 8038 Zürich-Wollishofen
MwSt-Nr. CHE-114.222.333 · www.alpsteinwerk.ch

Muster Immobilien GmbH
Bahnhofstrasse 7
6003 Luzern

RECHNUNG

Rechnungsnummer: 2026-07-0442
Rechnungsdatum: 14.07.2026

A resizable sticky note that always shows its text.

A second, selected note.

3 item(s).

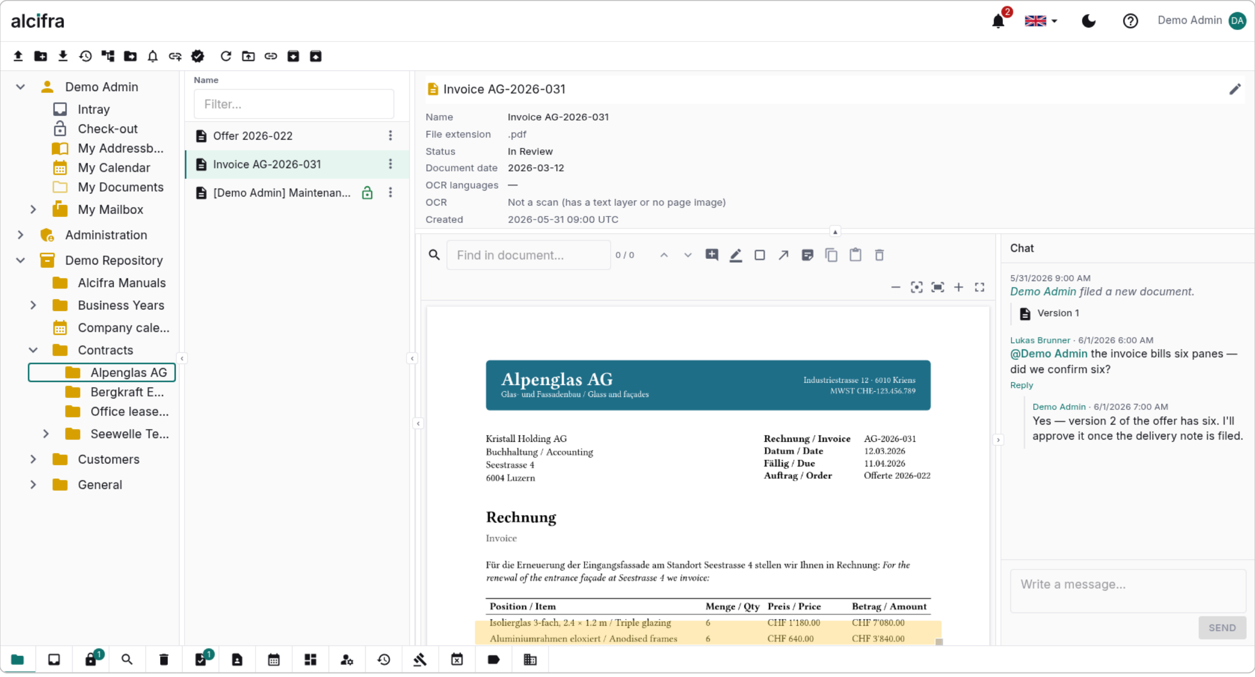
Figure 3: The preview with search hits highlighted on the page — click a word to copy it (shift-click to append), or step through the matches.

4 Adding & versioning documents

4.1 How documents get in

There is more than one way in, and which you want depends on whether the document is finished, whether it belongs to something already filed, and how many you have. They all end in the same place.

Route	Use it when
Upload on the ribbon	You are already looking at the folder it belongs in. Picks files and files them straight into the open folder.
Drag onto a folder — a row in the list, or a node in the tree	You can see the destination. The document is filed there; the view follows to that folder so you can see it arrive.
Drag onto empty space in the contents list	Same as above for the folder you are already in.
Drag onto a document	The file is a newer copy of that document — it is added as a **version**, not as a second document, and the history is kept.
Drag onto your own space ▸ Intray	It is not ready to file, or you do not yet know where it belongs. It waits in the Intray until you classify and file it.
Drag a document onto your own space ▸ Intray	You want to start from an existing document as a **template**. A copy lands in your Intray carrying that document's document type and index data, so you edit what differs. Nothing is created in the archive until you file it.
Drag onto your own space ▸ Check-out	You checked a document out, edited it on your computer, and are bringing it back. The file must still carry the document's name — that is what says which document it belongs to.
WebDAV	You would rather work in Finder, Explorer or Files. Mount the archive as a drive and copy documents in like any other folder.
Import	You are bringing in a whole folder tree at once, exported from Alcifra or elsewhere.
Email attachment	You filed an email and want one of its attachments as a document of its own.


alcifra
2
Demo Admin
DA
Demo Admin
Intray
Check-out
My Addressb...
My Calendar
My Documents
My Mailbox
Administration
Demo Repository
Alcifra Manuals
Business Years
Company cale...
Contracts
Alpenglas AG
Bergkraft E...
Office lease...
Seewelle Te...
Customers
General
Name
Filter...
Offer 2026-022
Invoice AG-2026-031
[Demo Admin] Maintenan...
Find in document...
0 / 0
Chat
Write a message...
SEND

Invoice AG-2026-031

Name Invoice AG-2026-031
File extension .pdf
Status In Review
Document date 2026-03-12
OCR languages —
OCR Not a scan (has a text layer or no page image)
Created 2026-05-31 09:00 UTC

Alpenglas AG
Glas- und Fassadenbau / Glass and façades
Industriestrasse 12 · 6010 Kriens
MWST CHE-123.456.789

Kristall Holding AG
Buchhaltung / Accounting
Seestrasse 4
6004 Luzern

Rechnung / Invoice AG-2026-031
Datum / Date 12.03.2026
Fällig / Due 11.04.2026
Auftrag / Order Offerte 2026-022

Rechnung

Invoice

Für die Erneuerung der Eingangsfassade am Standort Seestrasse 4 stellen wir Ihnen in Rechnung: *For the renewal of the entrance façade at Seestrasse 4 we invoice:*

Position / Item	Menge / Qty	Preis / Price	Betrag / Amount
Isolierglas 3-fach, 2.4 × 1.2 m / Triple glazing	6	CHF 1'180.00	CHF 7'080.00
Aluminiumrahmen eloxiert / Anodised frames	6	CHF 640.00	CHF 3'840.00

5/31/2026 9:00 AM
Demo Admin filed a new document.
Version 1

Lukas Brunner · 6/1/2026 6:00 AM
@Demo Admin the invoice bills six panes — did we confirm six?
Reply

Demo Admin · 6/1/2026 7:00 AM
Yes — version 2 of the offer has six. I'll approve it once the delivery note is filed.

Figure 4: Your own space expanded — it carries **your name** at the top of the tree: **Intray** and **Check-out** sit above your own folders. Drop files on **Intray** to stage them, or drop an edited working copy on **Check-out** to bring it back — and drag a document onto **Intray** to start new work from it as a template.

> **Two of these do not create a document.** A drop onto the **Intray** stages an item — it becomes a document only when you file it. A drop onto **Check-out** replaces your working copy — the document gets a new version only when you check it in. Everything else files immediately.

Uploading. Drag a file straight onto a folder (the bytes go directly to object storage — the server never proxies them). **The Intray** is a staging area: drop scans or files there, then classify each one (name, document type, index data) and **file** it into the archive.

What the archive will not keep. Programs and scripts are refused — an `.exe`, a `.bat`, a shell script, and anything that turns out to be a program whatever it has been named. Everything else is kept, including the formats a document system is often too narrow for: drawings, disk images, archives, video. When an **email's** attachment is refused the message itself is still archived, and a line in its chat thread names the attachment that was left out, so nothing goes missing quietly.

When the name is already taken. A folder cannot hold two things of the same name, so filing `Invoice.pdf` where an `Invoice` already sits asks you what you meant rather than refusing:

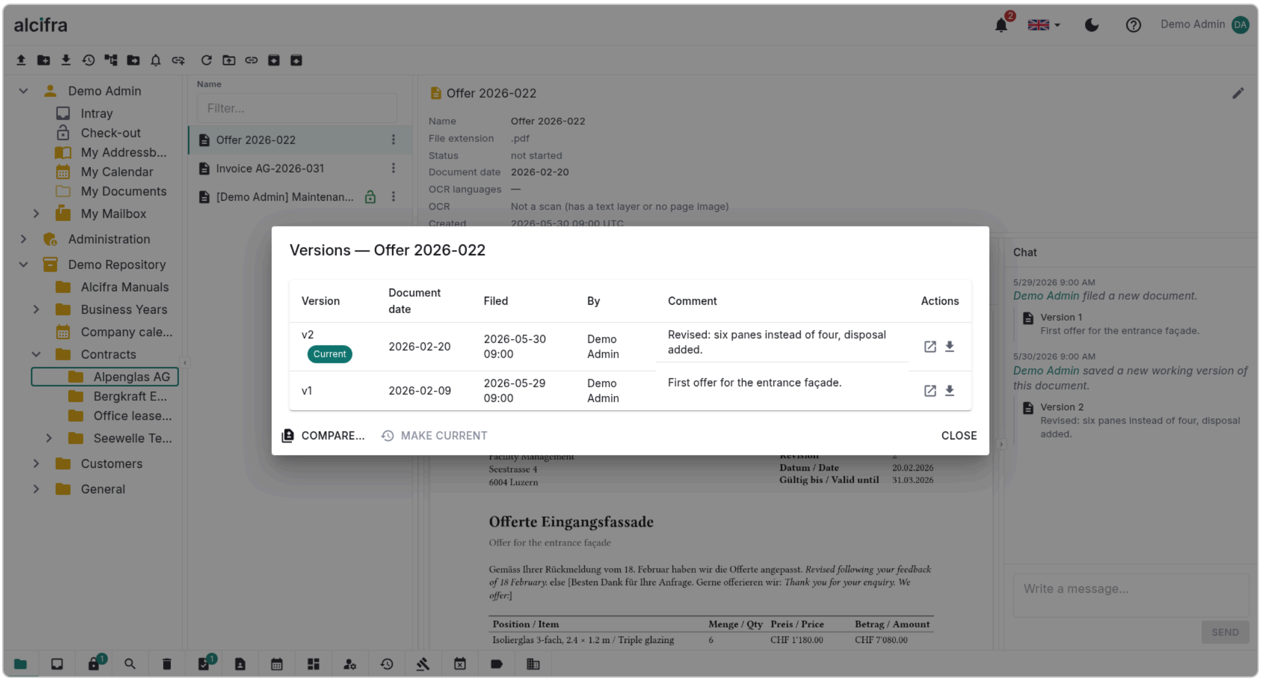
Choice	What happens
A new version of that document	The usual answer: the file is a newer revision of what is already there. The document keeps its index data and its history, and the previous version stays available.
A new document with a different name	The two are genuinely different documents that happen to share a file name. A free name is offered — `Invoice (2)` — and you can change it.

Either way you can add a **comment** saying what this file is, which becomes the version's comment. There is no "overwrite": nothing in Alcifra replaces content in place, and the honest equivalent is a new version.

> If the name is held by a **folder** rather than a document, only the second choice is possible — a folder has no versions to add one to.

Versions. Uploading a new file to an existing document adds a **version** — the history is preserved. The **Versions** dialog lists every version; you can compare two of them or **make current** an older one. When you upload a file that already exists, Alcifra warns you of the **duplicate**.

Give each version a **comment** saying what changed. It is the difference between a history someone can read and a list of timestamps: "Price corrected after the framework-agreement review" answers the question a reader of the list actually has, and no date can.


Name: Offer 2026-022
File extension: .pdf
Status: not started
Document date: 2026-02-20
OCR languages: —
OCR: Not a scan (has a text layer or no page image)
Created: 2026-05-30 09:00 UTC

Versions — Offer 2026-022

Version	Document date	Filed	By	Comment	Actions
v2 Current	2026-02-20	2026-05-30 09:00	Demo Admin	Revised: six panes instead of four, disposal added.	
v1	2026-02-09	2026-05-29 09:00	Demo Admin	First offer for the entrance façade.	

Chat

5/29/2026 9:00 AM
Demo Admin filed a new document.
Version 1
First offer for the entrance façade.

5/30/2026 9:00 AM
Demo Admin saved a new working version of this document.
Version 2
Revised: six panes instead of four, disposal added.

Seestrasse 4
6004 Luzern

Datum / Date: 20.02.2026
Gültig bis / Valid until: 31.03.2026

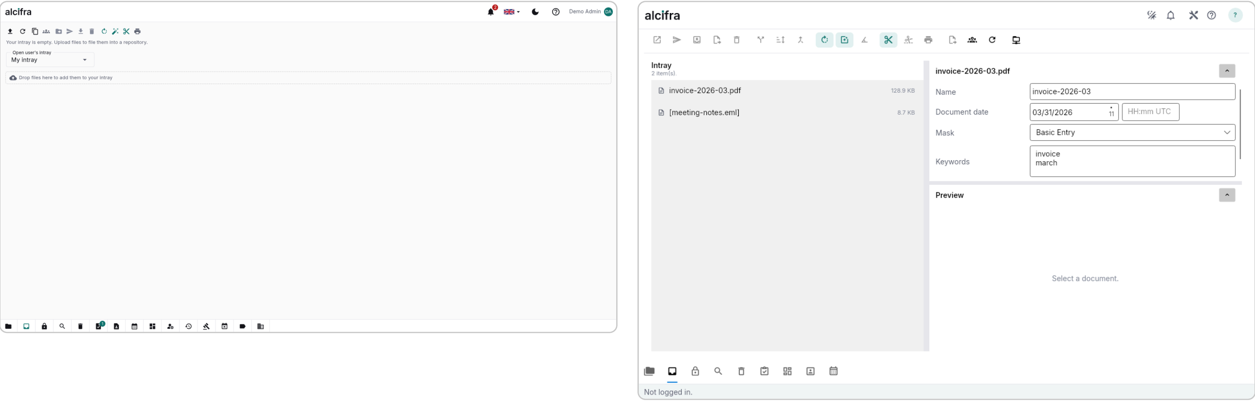
Offerte Eingangsfassade

Offer for the entrance façade

Gemäss Ihrer Rückmeldung vom 18. Februar haben wir die Offerte angepasst. *Revised following your feedback of 18 February.* else [Besten Dank für Ihre Anfrage. Gerne offerieren wir: *Thank you for your enquiry. We offer:*]

Position / Item	Menge / Qty	Preis / Price	Betrag / Amount
Isolierglas 3-fach, 2.4 × 1.2 m / Triple glazing	6	CHF 1'180.00	CHF 7'080.00

Figure 5: The **Versions** dialog: every version of the document with who saved it, when, and — the part that makes the list worth reading — the comment describing what changed.


Your intray is empty. Upload files to file them into a repository.

Open user's intray: My intray

Intray
2 item(s).

invoice-2026-03.pdf	128.9 KB
[meeting-notes.eml]	8.7 KB

invoice-2026-03.pdf
Name: invoice-2026-03
Document date: 03/31/2026
Mask: Basic Entry
Keywords: invoice
march

Figure 6: The Intray: staged items waiting to be classified and filed, in the web (left) and desktop (right) clients.

4.2 Tidying a scan before filing

A scan rarely arrives filing-ready: a batch holds several documents, a page went through the feeder sideways, a blank back rode along. The Intray is where all of that is put right — **before** filing, because a staged item can be reshaped freely, while a filed document's pages are part of the record. Every operation here works on `.pdf` and multi-page `.tif` files; anything else simply does not offer them.

Operation	What it does
Split into pages	One new item per page. The original is kept, so a split that turns out wrong is undone by deleting its output — a scan can be the only copy of a piece of paper.
Rotate/Sort	Opens the page dialog below: put the pages in order, delete a page with the bin on its tile, and turn a page with the ⟲ ⟳ buttons under it. Nothing is written until **Apply order** — one save for the whole arrangement, and **Cancel** discards everything. Offered from a single page up, because rotating a one-page scan that went in upside-down is exactly what it is for.
Join items	Several staged items become one, in the order you chose them. The sources are kept.
Cut at separator sheets	Cuts a batch into one item per document at the printed separator sheets between them (see below).

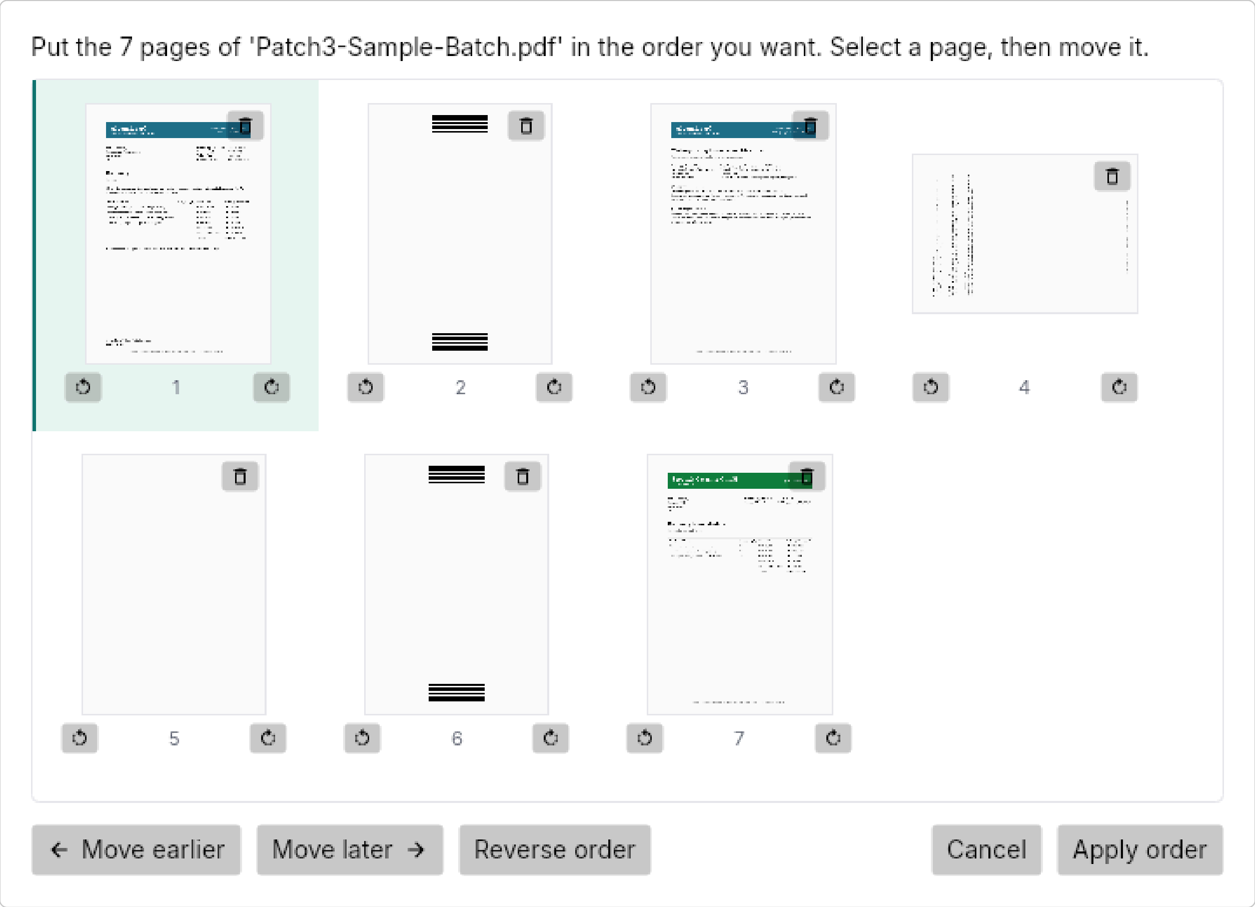

Put the 7 pages of 'Patch3-Sample-Batch.pdf' in the order you want. Select a page, then move it.

Figure 7: The **Rotate/Sort** dialog on the sample batch: page 4 went through the scanner upside-down and has been turned a quarter so far — one more press of its ⟳ button puts it upright. The bin on each tile deletes that page; nothing is saved until **Apply order**.

Rotation does not damage a PDF. Turning a PDF page only records the new orientation — the page's real, searchable text is untouched. A `.tif` page has no such notion and is re-encoded when turned, the same trade its straightening makes.

Separator sheets. For a stack of paper documents, print the **separator sheet** (the printer button in the Intray toolbar), lay one between each document, and feed the whole pile through the scanner in one go. **Cut at separator sheets** then produces one item per document and discards the sheets themselves. Sample files to try this with — a batch as the scanner would produce it, and the sheet — are served by your own installation under `/download/samples/`.

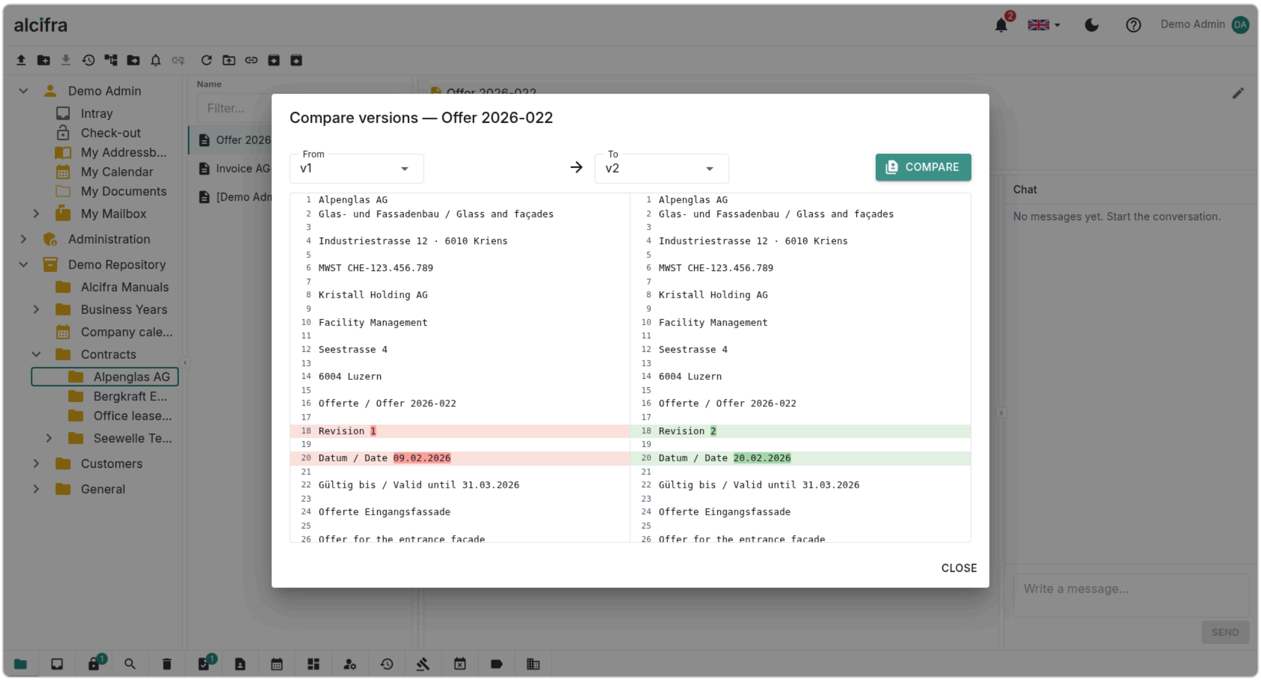
The automatic toggles. Three sticky switches in the Intray toolbar act on every **arriving** upload — including files saved into the Intray folder of the mounted network drive:

Toggle	**What it does to an arriving scan**
Auto-rotate	Turns pages that arrived upside-down or sideways the right way round. Works for `.pdf` and `.tif`.
Auto-straighten	Corrects the slight skew of a crooked scan — for `.tif`, and for a PDF that is itself a pure scan (image pages, no real text). A born-digital PDF is left alone: straightening re-renders the pages, and doing that would replace its real text with a recognised approximation.

Cut at separator sheets	Applies the separator cut to every arriving batch, so a scanner that feeds straight into the Intray needs no manual step at all.

> **Digitally signed documents are never touched.** Every page operation would break the signature, so on a signed item none is offered and the automatic steps skip it — the refusal is the feature.

Already filed? Rotate/Sort also exists on the **Check-out** tab. It edits the **working copy** of a document you have checked out — the archived version stays exactly as it was, and your rearrangement becomes the new version only when you press **Check in** (or is thrown away by **Discard**, like any other working-copy edit). Split and join stay in the Intray: they turn one item into several documents, which is filing work.


alcifra
Demo Admin
DA
Demo Admin
Intray
Check-out
My Addressb...
My Calendar
My Documents
My Mailbox
Administration
Demo Repository
Alcifra Manuals
Business Years
Company cale...
Contracts
Alpenglas AG
Bergkraft E...
Office lease...
Seewelle Te...
Customers
General
Name
Filter...
Offer 2026
Invoice AG
[Demo Adm
Offer 2026-022
Chat
Write a message...
SEND

Compare versions — Offer 2026-022

From: v1 → To: v2

COMPARE

```
Alpenglas AG
Glas- und Fassadenbau / Glass and façades

Industriestrasse 12 · 6010 Kriens

MWST CHE-123.456.789

Kristall Holding AG

Facility Management

Seestrasse 4

6004 Luzern

Offerte / Offer 2026-022

Revision 1

Datum / Date 09.02.2026

Gültig bis / Valid until 31.03.2026

Offerte Eingangsfassade

Offer for the entrance facade
```

```
Alpenglas AG
Glas- und Fassadenbau / Glass and façades

Industriestrasse 12 · 6010 Kriens

MWST CHE-123.456.789

Kristall Holding AG

Facility Management

Seestrasse 4

6004 Luzern

Offerte / Offer 2026-022

Revision 2

Datum / Date 20.02.2026

Gültig bis / Valid until 31.03.2026

Offerte Eingangsfassade

Offer for the entrance facade
```

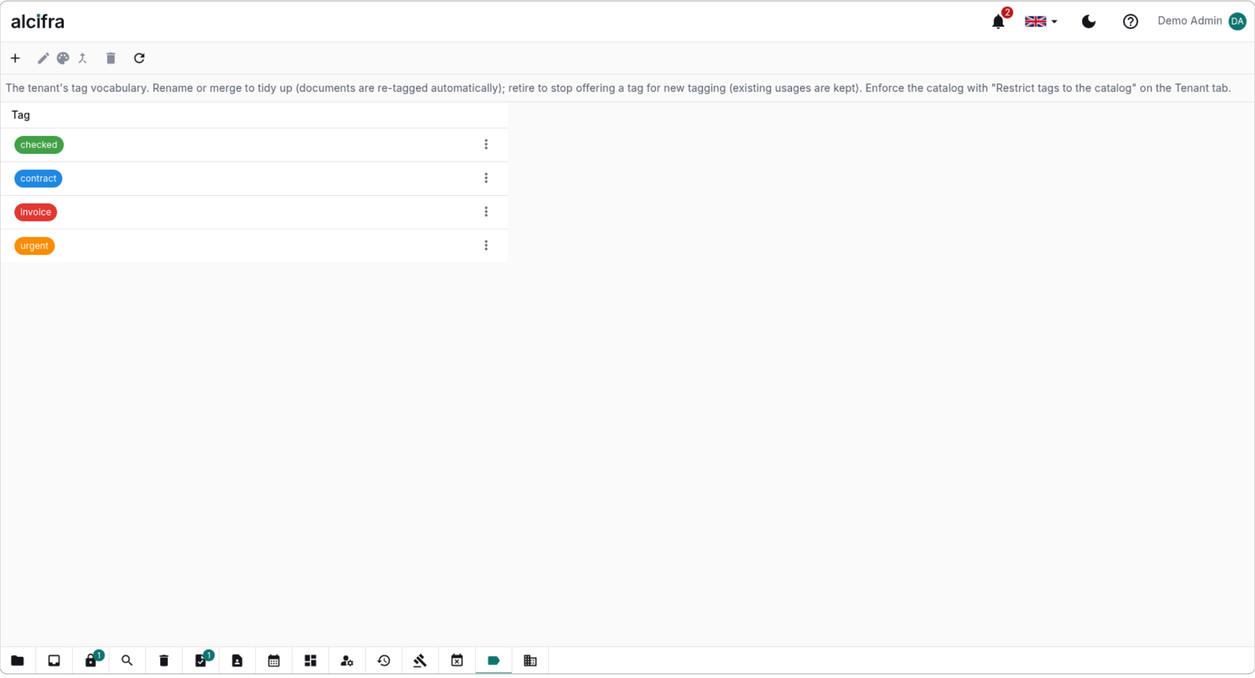
No messages yet. Start the conversation.

CLOSE

Figure 8: **Compare versions**: two revisions of a document side by side — old on the left, new on the right, changed lines aligned and the changed words highlighted within them. Works for any pair whose content yields text, notes and emails included.

5 Organizing

Create **folders**, and **move** documents between them by drag-and-drop. A **reference** (shortcut) lets one document appear in several places without copying it. **Tags** label documents for quick grouping — your tenant can maintain a curated tag catalogue with colours. **Sensitivity labels** mark how confidential a document is. Every user also has a **personal repository** for private documents.



The tenant's tag vocabulary. Rename or merge to tidy up (documents are re-tagged automatically); retire to stop offering a tag for new tagging (existing usages are kept). Enforce the catalog with "Restrict tags to the catalog" on the Tenant tab.

Tag
checked
contract
invoice
urgent

Figure 9: The tag catalogue: curated, colour-coded tags an administrator maintains for the whole tenant.

6 Working from your file manager (WebDAV)

The whole archive is reachable over **WebDAV** as a network drive — not just your personal space but the shared repositories you have permission to access, with your rights enforced on every operation. The mounted drive is called **webdav** and mirrors the Repositories tree exactly: your Personal space, then the repositories you can see, and nothing else.

The **WebDAV** button does the next useful thing rather than always the same thing, and its tooltip says which. In the **desktop client**: if none of your devices reaches files yet, it enrols **This computer** (see Section 8.2), shows its password once and then mounts; otherwise it mounts the drive and opens it (on Windows as a persistent drive letter, S: or the next one free); and if the drive is already mounted it goes straight there. So “show me my documents in Finder” is one click once you are set up.

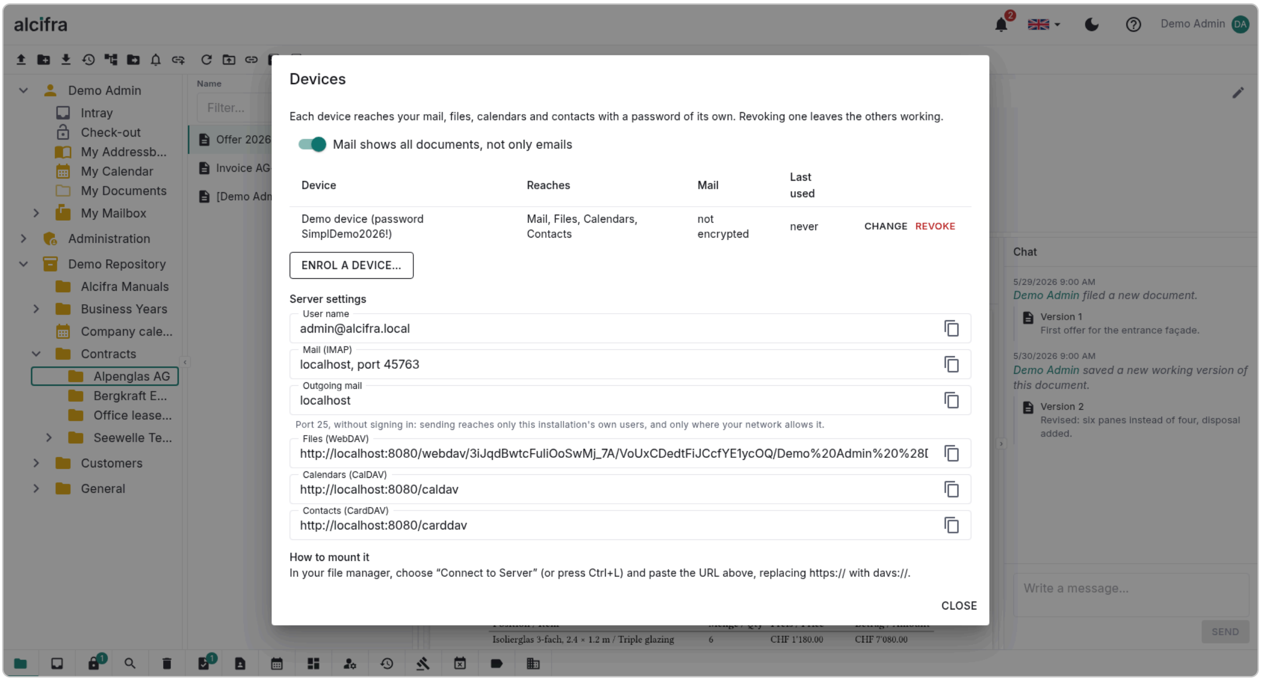
It opens where you already are. There is one button per tab, and each opens **its own** folder inside the drive rather than the top of the archive:

Where you press it	What opens
Repositories ribbon	The folder selected in the tree — the drive mirrors the tree exactly, so “where I am” and “which folder on the drive” are the same place. With nothing selected, the whole archive opens.
Intray tab, lower left	Your **Intray** folder.
Check-out tab, lower left	Your **Check-out** folder, where working copies live.

> **If you connect to more than one Alcifra** — a local one and a hosted one, say — your computer cannot give both drives the same name, so the second is called something like **Alcifra–1**. The button always opens the drive belonging to the server you are signed in to, whichever name it ended up with.

The drive signs in with your e-mail and a **device password** from **Devices…** (see Section 8.2), never your login password. The mount URL ends in `/webdav`, so your file manager names the drive “webdav”; on Windows you can rename it like any network drive.

The **web client** cannot mount a drive: a browser is not allowed to. Instead of leaving you with a URL and no idea what to do with it, its **Devices** dialog shows the mount steps for the operating system you are on — Finder’s *Go* ▸ *Connect to Server* on macOS, **Map network drive** on Windows, a `davs://` address on Linux — next to the values to paste into them.



Devices

Each device reaches your mail, files, calendars and contacts with a password of its own. Revoking one leaves the others working.

Mail shows all documents, not only emails

Device	Reaches	Mail	Last used	
Demo device (password SimplDemo2026!)	Mail, Files, Calendars, Contacts	not encrypted	never	CHANGE REVOKE

ENROL A DEVICE...

Server settings

User name: admin@alcifra.local
Mail (IMAP): localhost, port 45763
Outgoing mail: localhost
Port 25, without signing in: sending reaches only this installation's own users, and only where your network allows it.
Files (WebDAV): http://localhost:8080/webdav/3iJqdBwtcFuliOoSwMj_7A/VoUxCDedtFiJCcfYE1ycOQ/Demo%20Admin%20%28
Calendars (CalDAV): http://localhost:8080/caldav
Contacts (CardDAV): http://localhost:8080/carddav

How to mount it

In your file manager, choose “Connect to Server” (or press Ctrl+L) and paste the URL above, replacing https:// with davs://.

CLOSE

5/29/2026 9:00 AM
Demo Admin filed a new document.
Version 1
First offer for the entrance façade.

5/30/2026 9:00 AM
Demo Admin saved a new working version of this document.
Version 2
Revised: six panes instead of four, disposal added.

Figure 10: The web client’s Devices dialog: your user name and every server address, each with a copy button, and the mount steps for the operating system you are using.

6.1 Editing a document in place

Open a document from the mounted drive, change it, save it — that is the whole gesture, and it works the way you would expect from any network drive.

What happens underneath is worth knowing, because it changes what other people see. Simple editors write the file straight back, and the change becomes a new version immediately. An **office suite**, however, never overwrites a file in place: it writes a hidden companion file, then a temporary copy of the new content, and finally renames that copy over the original. Alcifra recognises that sequence and turns it into a **check-out** — the same one you get from pressing **Check out** in the app.

So after saving from your file manager:

- the document is **checked out to you**, and appears on your **Check-out** tab marked **automatic**;
- your edit is waiting there as your working copy — it is **not yet a new version**, and other people still see the previous one;
- nobody else can change the document until you are finished;
- you finish by pressing **Check in**, from either client or by saving into the mount’s **Check-out** folder.

> **Saving is not the same as publishing.** The system cannot tell when you have finished editing — only you can, and **Check in** is how you say so. Until then your work is kept safely as your working copy. A check-out left untouched for a long time is released automatically by your organisation’s idle rule, and you are warned before that happens.

The **automatic** marker exists so this never looks like a fault: a document you never explicitly checked out has become yours, and the tab says why, naming the program that did it.

If someone else already has the document checked out, your editor is told the file is locked and opens it read-only, rather than letting you edit for ten minutes and failing at the save. Documents you are not allowed to edit, and documents frozen by a legal hold, behave in the mount exactly as they do in the app: they refuse.

6.1.1 When something in the Intray is overwritten

The **Intray** holds bare files rather than documents: no versions, no history — which is what makes it quick to drop things into, and what makes an overwrite final in a way it never is for a filed document. That matters because a program can overwrite one without meaning to. One office suite saved correctly into an Intray item and then, six seconds later, rolled its own save back by moving an **empty** backup slot over the file.

Nothing is lost when that happens. Every write that replaces an Intray item first **sets the outgoing bytes aside**, and an administrator can list what was set aside for you and restore any of it. So if an Intray item has turned empty or wrong after a save, say so rather than starting again — the previous bytes are almost certainly still there. Filed documents need none of this: they keep every version by design, which is the argument for filing anything you would mind losing.

> **A warning from your own software is not a fault in ours.** Some commercial vendors treat a WebDAV connection to a site that is not on their own allow-list as suspicious, or refuse it outright. That is a policy decision in the respective vendors' software, not a limitation of Alcifra — the same mount that one program declines will typically work in another on the same machine.

6.2 Reading the archive in your mail program (IMAP)

Where WebDAV turns the archive into a network drive, **IMAP** lets a mail program browse it: point any IMAP-speaking mail client at your Alcifra server and browse the same ACL-filtered tree as mailboxes. **INBOX** is your personal folder; the shared repositories you can see appear beside it. Archived e-mails read natively; every **other** document can appear too — as a message carrying the file as an attachment — if you switch that on.

> **This is your archive wearing an IMAP face — it is not a mail service.** Alcifra does not collect your mail from your provider, does not send mail, and is not a replacement for your e-mail account. Nothing here changes where your mail lives. What it gives you is the second account in your mail program: keep your real one alongside it, and **file** mail across from one to the other, which is what the drag described below is for.

Shortcuts show up as mailboxes too. A folder you have filed a reference to appears in the list wherever the reference sits, with everything beneath it — so the destinations you file into most can be gathered in your personal folder and reached from the mail program without hunting through the whole tree.

Your mail program signs in with your e-mail and a **device password** from **Devices…** in the account menu (see Section 8.2), whose dialog also shows the **server** and its ports. The switch **Mail shows all documents** is yours, for every device: off (the default) lists only e-mails; on lists everything you may see, with the file attached to open straight from the mail client.

> **The mail client cannot rearrange the folders.** Folders are created, renamed and deleted in Alcifra — an IMAP client trying the same is politely refused. Messages, though, are yours to work with: drag an e-mail into a folder to **file it into the archive** (it becomes a proper e-mail document, named by its subject), move a message to re-file it, copy one to leave a **reference**,

> and delete + expunge to send it to the **recycle bin** — never a hard delete. Read/unread marks are remembered per person.

Encrypting what your mail program receives. If you want the messages a mail client fetches to be readable **only with your own key**, switch on **Encrypt mail content** for that device under **Devices…** (see Section 8.2): from its next connection on, every message arrives sealed to your enrolled reader certificates, and the mail program decrypts it once it can use the key. Switching it on needs an enrolled, **attested hardware key** — a smartcard or a security key — which you enrol with **Enrol a certificate…** in the account menu (Section 15.8); in the desktop client, **Add a card or token…** reads the certificate and its attestation off the token in one step. Before choosing a card, read Section 15 on what it covers: it is the strongest option, and some mail programs cannot use a card at all. Where your organisation seals all mail, the switch is on for every device and cannot be turned off.

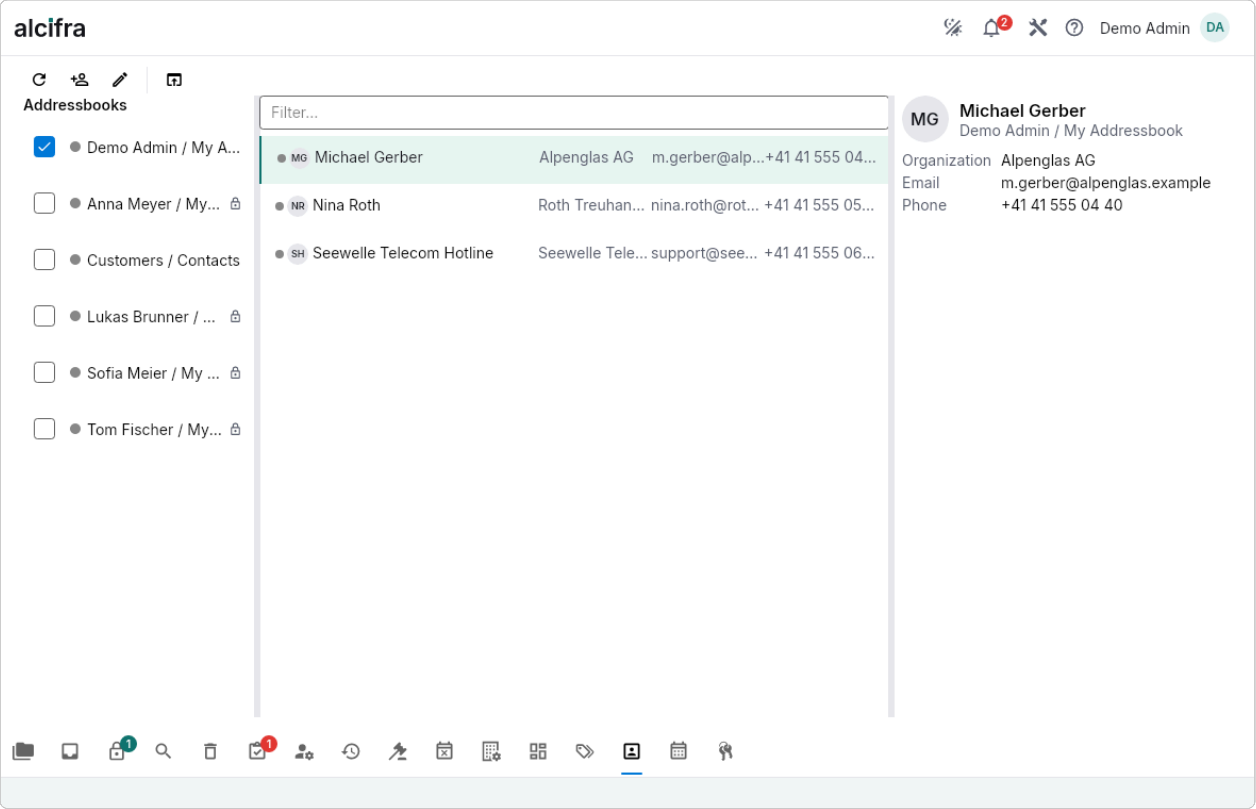
> **Earlier versions let you generate or upload an encrypted-mail certificate in the browser.** That self-service certificate is retired: a key that exists as a file is exactly what a hardware key avoids. If you had one, the archive carried it over where your organisation's rules allow, and told you once, by notification, if mail is no longer sealed to it.

The setup profile a phone installs from a device's hand-over also carries the **archive's own mail-in certificate**: composing an encrypted message **to** the archive's ingest address (`archive@` your installation's mail domain) then works out of the box — the archive decrypts it on arrival and files a normal, searchable document.

Notes, too. Point a notes app that syncs over IMAP at the same account and it finds the **Notes** mailbox — your personal **Notes** folder in disguise. Every note becomes a proper archive document with the **Note** document type, and editing a note on your phone adds a **new version** in the archive rather than a new copy — the full history stays browsable in the workbench. The Notes folder is typed: it accepts only notes, and notes live only there (references to them may go anywhere).

7 Contacts, calendars & your tasks

Two of the workbench tabs are not about documents at all — or rather, they are about documents that happen to be a **contact card** or a **calendar entry**. **Contacts** lists the people in your addressbooks; **Calendar** lists your appointments. Both read the same archive, with the same permissions, and both can be **subscribed to from your phone**.



MG Michael Gerber	Alpenglas AG	m.gerber@alp...	+41 41 555 04...
NR Nina Roth	Roth Treuhan...	nina.roth@rot...	+41 41 555 05...
SH Seewelle Telecom Hotline	Seewelle Tele...	support@see...	+41 41 555 06...

MG **Michael Gerber**
Demo Admin / My Addressbook

Organization	Alpenglas AG
Email	m.gerber@alpenglas.example
Phone	+41 41 555 04 40

Figure 11: The Contacts tab: addressbooks on the left, the people in the ticked ones merged into one list.

Collections overlay rather than replace. The left-hand list is a set of tick-boxes, not a single choice — tick two addressbooks and you see both sets of people in one list, each row carrying a small colour swatch saying which collection it came from. Your own **My Addressbook** and **My Calendar** start ticked, so the tab opens with something in it. The colour is yours to set and yours alone: changing it does not change what anybody else sees.

Every user's personal space contains **My Addressbook** and **My Calendar** from the start. They cannot be deleted or moved — they are the fixed points your phone subscribes to, and provisioning them would be pointless if the next click could remove them. Addressbooks and calendars can also live **anywhere else** in the archive: a shared repository can hold a team addressbook, and whoever has permission sees it in the same list.

7.1 Adding and editing a contact or an appointment

New contact and **New appointment** open a form. Fill it in and save, and the item is created — nothing exists until you save, so closing the form leaves nothing behind. If more than one collection is ticked, the form asks which one it should go into; with only one candidate it does not ask, and the confirmation names where it went.

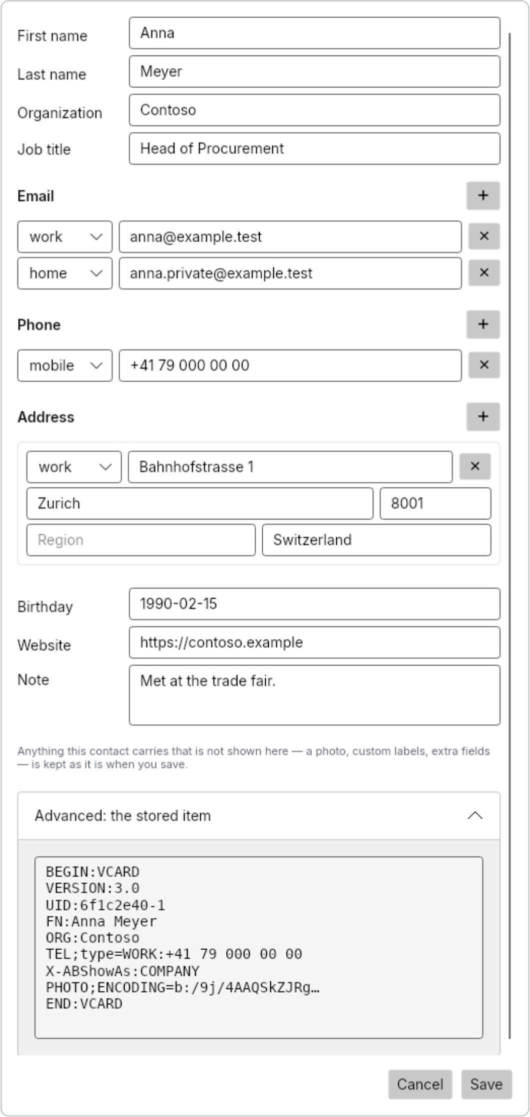
Edit opens the **same** form on an existing item. That is deliberate rather than incidental: a create form with fewer fields than the editor would quietly discard whatever you typed into the ones it lacked.

A contact holds rather more than a name — several **e-mail addresses** and **phone numbers**, each with its own type, several **postal addresses**, an organisation, a job title, a birthday, a website and a note. Add and remove rows as you need them.

> **Everything the form does not show is kept.** A contact created on a phone carries things this form does not model — a photo, custom labels, fields belonging to some other program. Saving here **preserves** all of it rather than dropping what it does not understand, which is what makes it safe to edit in the archive a card that lives on your phone.

7.2 Advanced: the stored item

Under every contact and appointment form there is a collapsed **Advanced: the stored item** section. Open it and you see the item exactly as stored — a **vCard** for a contact, an **iCalendar** entry for an appointment — including the properties the form above does not show.



First name	Anna
Last name	Meyer
Organization	Contoso
Job title	Head of Procurement

Email

work	anna@example.test
home	anna.private@example.test

Phone

mobile	+41 79 000 00 00

Address

work	Bahnhofstrasse 1
Zurich	8001
Region	Switzerland

Birthday	1990-02-15
Website	https://contoso.example
Note	Met at the trade fair.

Anything this contact carries that is not shown here — a photo, custom labels, extra fields — is kept as it is when you save.

Advanced: the stored item

```
BEGIN:VCARD
VERSION:3.0
UID:6f1c2e40-1
FN:Anna Meyer
ORG:Contoso
TEL;type=WORK:+41 79 000 00 00
X-ABShowAs:COMPANY
PHOTO;ENCODING=b:/9j/4AAQSkZJRg…
END:VCARD
```

Figure 12: The contact editor with **Advanced: the stored item** open — the stored vCard, including the properties no form field shows.

It is editable, and this is the one place in the product where saving **replaces** rather than merges. Delete a line here and the property is gone; that is what “raw” has to mean, and a merge would put it back while telling you the save had succeeded. Two things are refused rather than accepted: text that is not a valid card or entry, and a change to the **UID** — the identifier every phone and mail program uses to recognise this item. Changing the UID would not rename the item, it would make it a **different** one, so your phone would keep the old copy and add the new. In both cases nothing is written and the stored item is untouched. Removing the UID line is not a change; the stored one is kept.

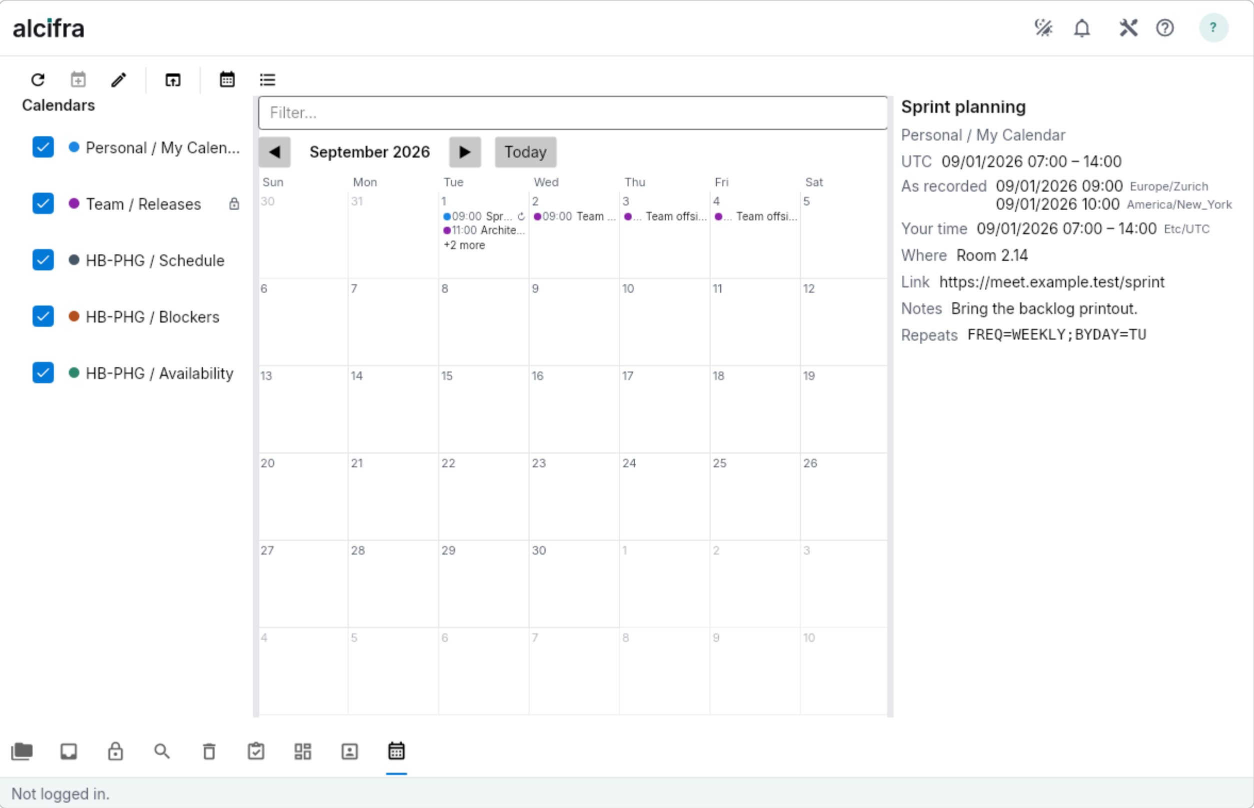
> **While you are editing the raw text, the fields above go read-only.** Both describe the same item and only one of them can be saved. A line under the box says which way the save will go.

Every save — from the form or from the raw box — writes a **new version**, exactly as any other change to a document does. A raw edit you regret is recoverable from the version history like anything else.

7.3 Subscribing from your phone (CalDAV & CardDAV)

Addressbooks and calendars speak **CardDAV** and **CalDAV** — the standards phones, tablets and mail programs already use. Point a device at your Alcifra server and it finds every addressbook and calendar you are allowed to see, wherever it sits in the archive; you then pick which to sync. Edits travel both ways: a contact changed on your phone becomes a new version in the archive, and one changed in the workbench appears on the phone.

A device signs in with your e-mail and its own **device password** from **Devices…** in the account menu (see Section 8.2), the same one its file manager and mail program use.


alcifra
?
Calendars
☑ Personal / My Calen...
☑ Team / Releases
☑ HB-PHG / Schedule
☑ HB-PHG / Blockers
☑ HB-PHG / Availability
Filter...
September 2026
Today
Sun
Mon
Tue
Wed
Thu
Fri
Sat
09:00 Spr...
11:00 Archite...
+2 more
09:00 Team ...
... Team offsi...
... Team offsi...
Not logged in.

Sprint planning

Personal / My Calendar

UTC 09/01/2026 07:00 – 14:00

As recorded 09/01/2026 09:00 Europe/Zurich
09/01/2026 10:00 America/New_York

Your time 09/01/2026 07:00 – 14:00 Etc/UTC

Where Room 2.14

Link https://meet.example.test/sprint

Notes Bring the backlog printout.

Repeats `FREQ=WEEKLY;BYDAY=TU`

Figure 13: The Calendar tab, ordered by time — undated entries sort last rather than into the top rows.

7.4 Your tasks, on your phone

Beside your calendars, two more collections appear that hold no documents at all:

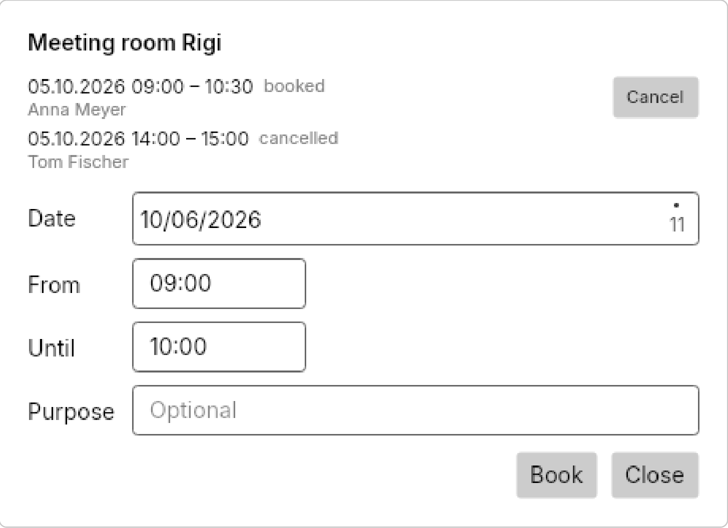
- **My tasks** — the reviews assigned to you, as to-do items. This is what a reminders or task app subscribes to.
- **My task deadlines** — the same reviews as dated entries, for calendar programs that do not show to-dos.

Both are **read-only**. They are a view of the **My tasks** tab, published so you can see what is waiting for you without opening the archive; approving or rejecting a review still happens in the workbench, where it is recorded with who did it and why.

> **A review only gets a deadline if its document type defines one.** The review SLA is set per document type, and where none is set the review simply has no due date — it still appears in **My tasks**, but it cannot appear in **My task deadlines**, which is a list of dates. An empty deadlines list usually means no document type in your tenant has an SLA configured.

7.5 Reserving a meeting room

A **meeting room** is a folder whose type is *Meeting room* — an administrator creates one wherever rooms should live, and whoever may see the folder may see its reservations. Select a room and its context menu and the ribbon offer **Bookings...** under **Actions**: the reservations that exist, and a row to take a new one.



Meeting room Rigi

05.10.2026 09:00 – 10:30 booked
Anna Meyer

05.10.2026 14:00 – 15:00 cancelled
Tom Fischer

Date: 10/06/2026
From: 09:00
Until: 10:00
Purpose: Optional

Figure 14: The Bookings dialog on a room: existing reservations above — a cancelled one stays visible as history — and the new reservation below, its times typed rather than picked.

Pick the date, type the times — `0950`, `9:50` and `09:50` all work — add a purpose if you want one, and **Book**. A slot that overlaps an existing reservation is **refused, and the refusal says so**: the room is never silently double-booked, and there is no queue to wonder about. **Cancel** keeps the row, marked cancelled, so “who had the room booked?” stays answerable; cancelling frees the slot for the next person.

A reservation can repeat. The dialog books single slots; a weekly jour fixe is made as a repeating calendar entry in the room’s Schedule — from the Calendar tab or from your phone — and **every occurrence** is conflict-checked, so a series that would collide with anything on any of its dates is refused as a whole. The one condition: a repeating **reservation** must have an end (a last date or a number of times). “Every Monday, forever” cannot be checked against the next endless series, so the archive does not accept it.

Behind the dialog there is no separate booking system. The first reservation creates a **Schedule** calendar under the room, and every reservation **is** a calendar entry in it — a document like any other, with versions, an audit trail and the room’s own permissions. Open the Schedule in the tree and you are looking at the same reservations the dialog shows; tick it in the **Calendar** tab and they overlay your own appointments, colour-coded like any other collection.

And because the Schedule is a calendar, your phone can subscribe to it — the same CalDAV subscription as any other calendar (Section 7.3). That cuts both ways, deliberately:

- **Reading:** tick the room’s Schedule on your phone and the room’s reservations appear beside your own appointments — the fastest answer to “is the room free at three?” is the calendar already in your pocket.
- **Writing:** creating an event in that calendar from your phone **is** booking the room, and it passes the **same** conflict check as the dialog. Dragging a reservation to another time is a rebooking, checked again; there is no way in — workbench, phone, or any calendar program — that bypasses the rule.

7.5.1 Three calendars, three different questions

A bookable thing — a room, a vehicle, a person’s time — answers three questions about the same timeline, and each gets its own calendar because each obeys a **different rule**. Putting them in one calendar would mean one rule for three jobs, and two of the three would be wrong.

Calendar	It says	And the rule that makes it different
Schedule	This time is *claimed*	Two reservations may **never** overlap. A slot that collides is refused, and the refusal says so.
Blockers	This time is *withdrawn*	Blocks **may** overlap each other — two defects reported on one vehicle are two entries, and refusing the second would mean the second finding cannot be recorded.
Availability	This time is *offered*	Windows may overlap each other, **and** reservations sit inside them — that overlap is the whole point, not a conflict.

Out of service. A block filed in Blockers (“projector broken, Tuesday to Thursday”) **suspends** every reservation it overlaps rather than deleting them; when the block is cleared, those reservations simply stand again. Nothing is stored as “suspended”, so nothing can be left stale, and nobody can edit a reservation back into service — the only way out is clearing the block, which is a right of its own. A suspended reservation shows on your phone as **tentative**, so you can see it is in doubt without it vanishing from your week.

The two moments differ, deliberately. A **new** booking that lands in a block is **refused** at the door; a booking that already existed when the block appeared is **suspended**. The archive will not let you walk into a known problem, and it will not quietly cancel a commitment you already made.

Offered time. An Availability window is somebody publishing when a thing may be booked — an instructor’s teaching hours, a pool car’s lending times. Two rules about it are worth knowing because the obvious guess is wrong in both cases:

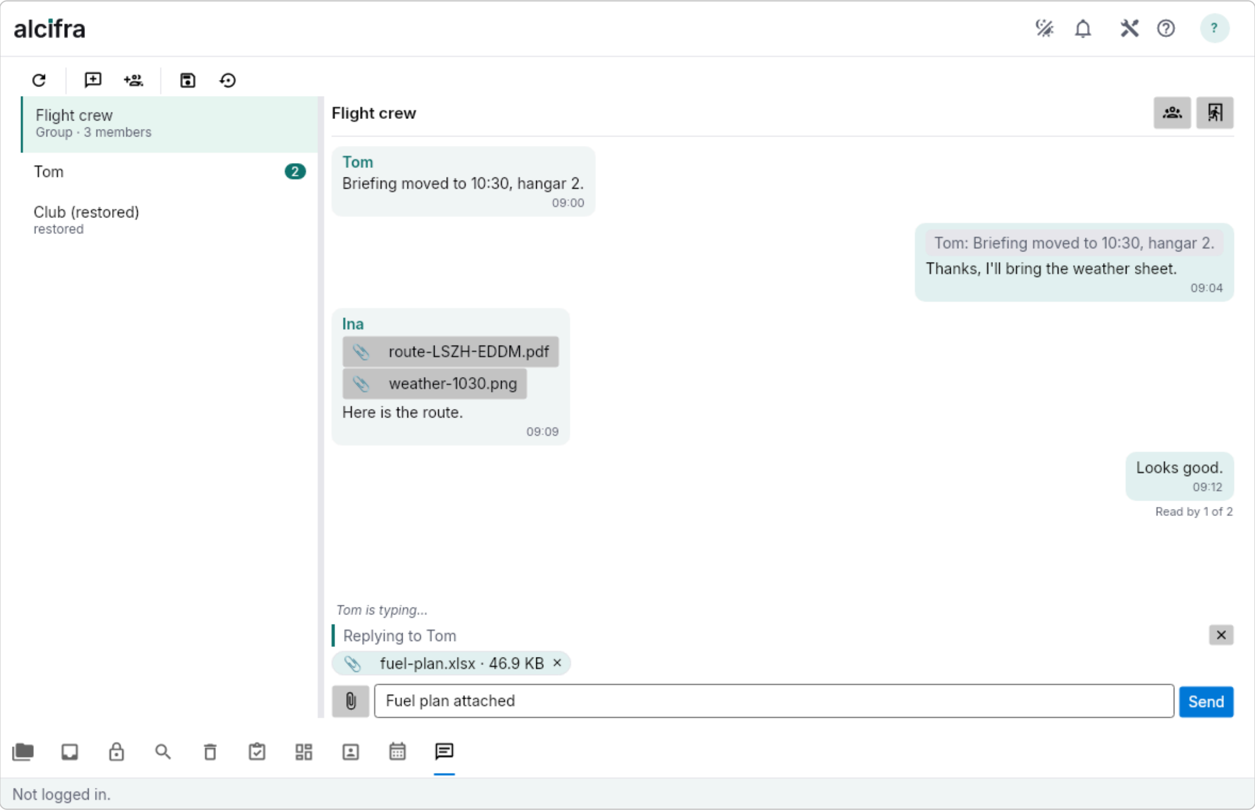
- **A window must cover the whole booking, on its own.** Two adjacent windows do not add up: 09:00–11:00 and 11:00–13:00 will not between them carry a 10:00–12:00 booking. Accepting

half an offer as consent for a whole booking is a failure nobody would see — the booking succeeds, somebody is committed to a time they never offered, and the first anyone learns of it is when they do not turn up.
- **No window means not offered.** A thing nobody has published time for offers nothing. Absence of an offer is not consent.

Whether a reservation **requires** an offer at all depends on what is being booked, and that is decided by the industry module that owns the thing rather than by the archive: booking an instructor may need their published hours as their consent, while booking a meeting room needs nothing of the sort. The archive stores the offer and answers the question; what the answer **obliges** is the module's rule.

7.6 Messages on the desktop

Where the archive offers the **Messenger**, the workbench shows a **Messages** tab: your conversations on the left — a group with its number of members, an unread count as a badge — and the selected one on the right, drawn and handled exactly as on the phone. A message opens with your own key — a key on this computer if you have one, otherwise your card, which asks for its PIN once.



Tom
Briefing moved to 10:30, hangar 2.
09:00

Tom: Briefing moved to 10:30, hangar 2.
Thanks, I'll bring the weather sheet.
09:04

Ina
route-LSZH-EDDM.pdf
weather-1030.png
Here is the route.
09:09

Looks good.
09:12
Read by 1 of 2

Tom is typing...
Replying to Tom
fuel-plan.xlsx · 46.9 KB ×
Fuel plan attached

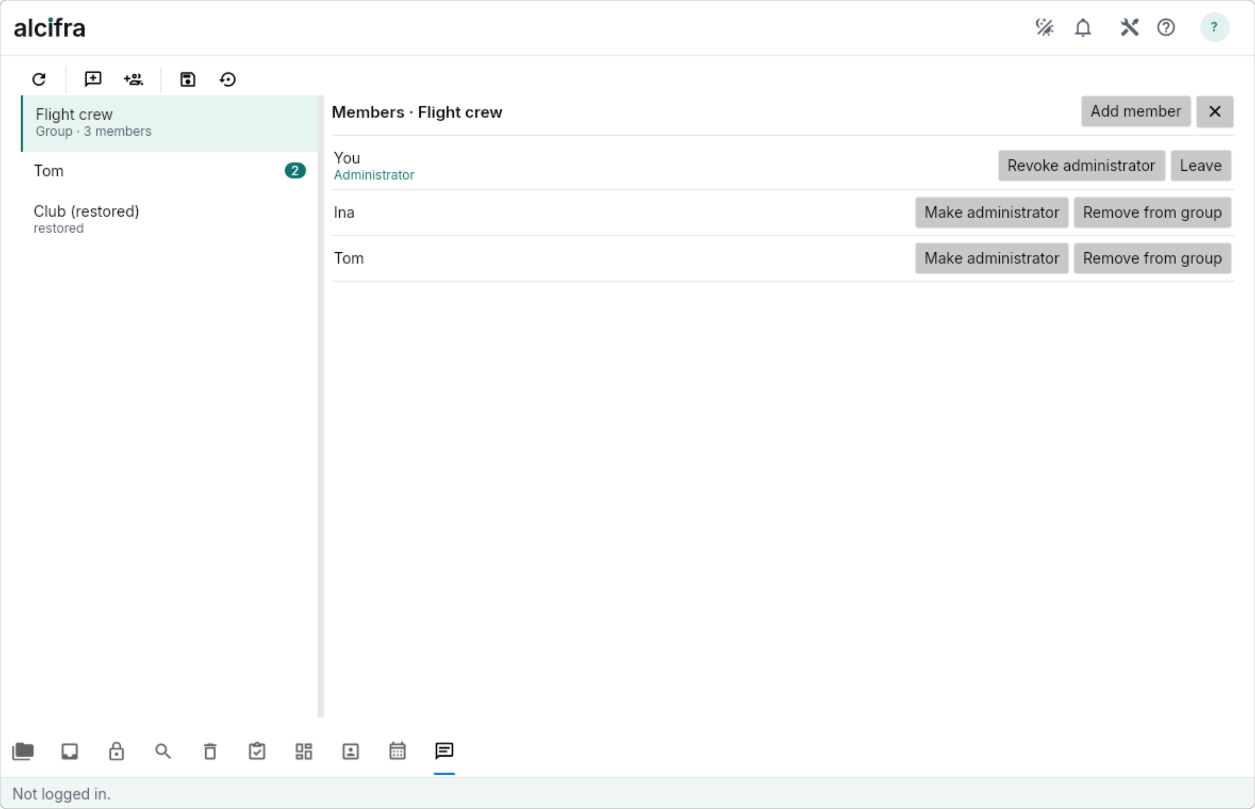
Figure 15: The Messages tab: a thread with a reply, two files in one message and a read receipt; below it, a reply being written with a file waiting as a chip.

Writing. Type into the field at the bottom: **Enter** sends, **Shift+Enter** starts a new line. While a message is on its way it shows as **Sending…** and the field waits, so one click never sends twice. The paper clip adds files — several at once — as chips above the field, each with its own ×; **dragging files** onto the conversation does the same. **Send** sends them and your text as **one** message, the text as their caption. Right-click a message for what you can do with it: **Reply** (a bar above the field says to whom; × or **Esc** takes it back), **Edit** your own message (it moves into the field; **Save** or **Ctrl+S** or **Cmd+S** keeps the change, **Esc** leaves it as it was), **Copy text**, **Read by…** in a group, and **Delete for everyone** or **Delete for me**. Who else is typing shows beneath the thread; the keyboard button in the ribbon switches that off in both directions. If you have scrolled up to read older messages, something new does not pull you down: **New messages ↓** takes you there.

Files stay in the app. A photo, a PDF or a plain text opens right here, over the conversation, with find and zoom — read from memory, nothing written to the disk. What the desktop cannot show (an office document, a video or a sound) can be handed to another application with **Open with…** or saved with **Save as…** — only after a warning that the copy leaves the archive's protection, and not at all where your organisation keeps files in the app.

Sealed stays sealed. If you cancel the PIN prompt, or no key here opens a conversation, its messages show **sealed** rather than not at all; **Open with my key** asks again. A conversation whose newest message stayed sealed is not marked read. When you take your card out of the reader, the conversation keys are forgotten with it and the thread shows sealed until you open it again.

Chats and groups. The ribbon starts a **new chat** or a **new group** (a name and several people); only colleagues with an enrolled key are offered, because nothing can be sealed to anyone else. In a group, the people button shows its **members**, administrators first: an administrator adds and removes members — each time this computer makes a new key, so a newcomer reads from now on and someone removed reads nothing more — and makes others administrators; the last administrator cannot step down. **Leave** asks first.



Flight crew
Group · 3 members

Tom 2

Club (restored)
restored

Members · Flight crew

You
Administrator

Ina

Tom

Not logged in.

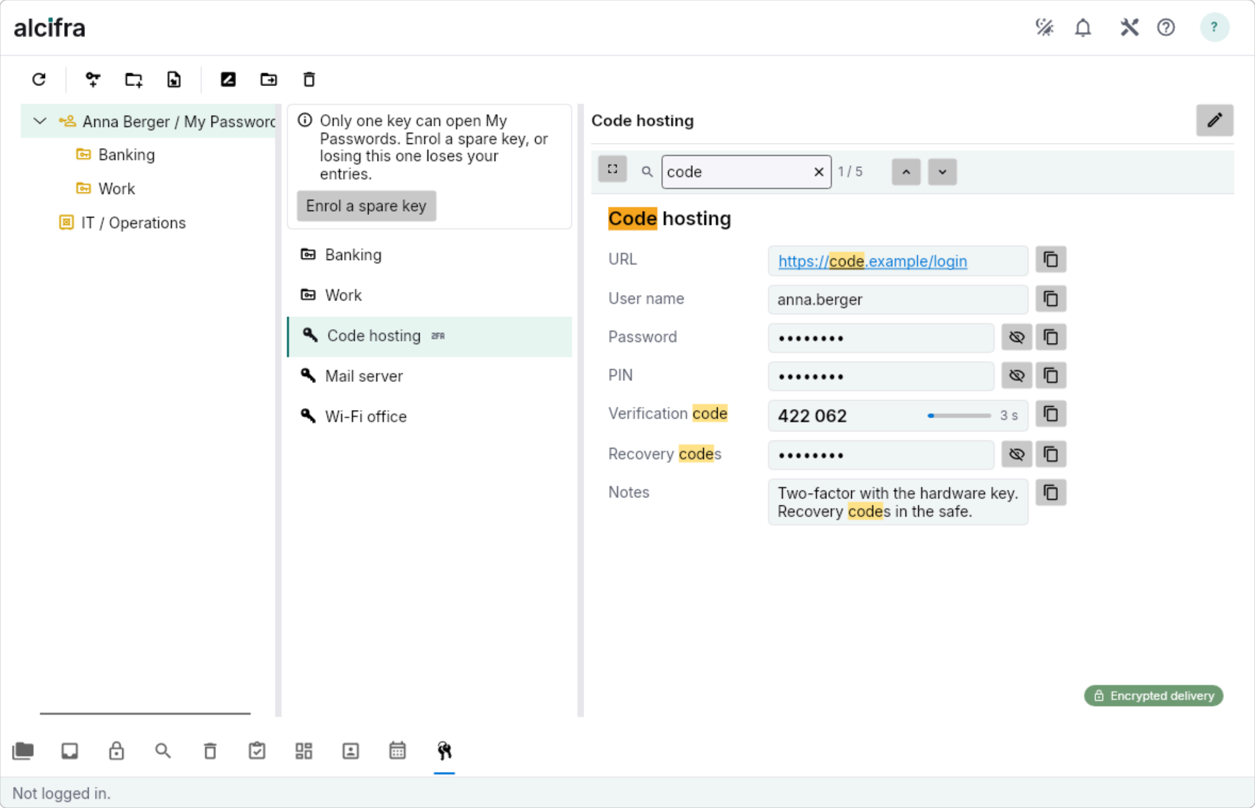
Figure 16: A group's members, administrators first, each with what you may do.

Back up writes all your conversations to a file you choose, sealed exactly as they were delivered — nothing is decrypted to make it, so only your own key opens it again. **Restore** brings such a backup back as history to read; once everyone from a restored conversation is on this installation, **Continue here** turns it into a live conversation with the history above it. A "New messages" notification in the bell opens its conversation here, and an open conversation reads new messages as they arrive.

What the desktop does not do, because a desktop cannot: take a photo with a camera (choose one from a file instead), be woken by a push while it is closed, or back up on a schedule in the background.

7.7 Passwords on the desktop

Where the archive offers the **password store**, the workbench shows a **Passwords** tab — the key ring at the end of the tab bar. On the left are your stores: **My Passwords** first (it is created the first time you open it), then the shared stores you can decrypt. The middle lists the selected folder’s folders and entries; the right shows the selected entry in the preview.



Only one key can open My Passwords. Enrol a spare key, or losing this one loses your entries.

code

Code hosting

URL: https://code.example/login

User name: anna.berger

Password: ••••••••

PIN: ••••••••

Verification code: 422 062 3 s

Recovery codes: ••••••••

Notes: Two-factor with the hardware key. Recovery codes in the safe.

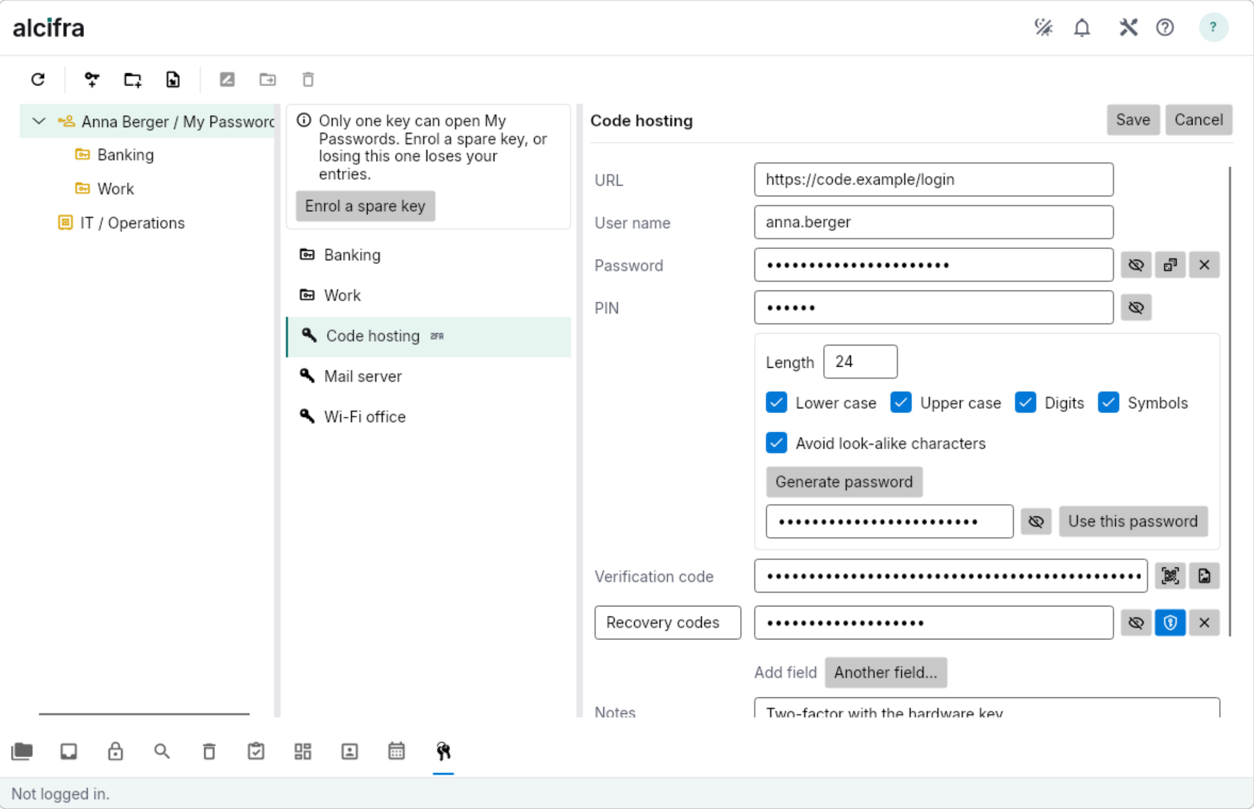
Figure 17: The Passwords tab: My Passwords with its folders, an entry opened in the preview, and the find box lighting what is visible. The password, the PIN and the recovery codes stay hidden.

Opening an entry. Selecting an entry opens it with your hardware key: the card asks for its PIN once and remembers it while the card stays in the reader. Only a hardware key opens an entry, never a key stored on the computer, because entries are sealed only to attested hardware keys. If you cancel the PIN prompt, the entry stays sealed and **Open with key** asks again. When you take the card out, the entry closes with it.

Reading it. The fields come in the same order as on the phone: URL, user name, password, PIN, verification code, then everything else. The password, the PIN and every field marked secret show dots. Hold the eye button down (or hold the space bar while it has the focus) to see the value; it hides again the moment you let go, or when the window goes to the background. While a secret is visible, the window is kept out of screenshots and screen sharing, where your system allows it. The copy button copies a field; a copied secret is removed from the clipboard after 30 seconds, unless you have copied something else since. The find box above the entry finds its name, its labels and its plain values, but never a hidden value, so it cannot be used to guess one.

Verification codes. An entry can also hold the six- or eight-digit code a site asks for as a second step when you sign in. Its row always shows the code valid right now, in two groups (**123 456**), with the seconds it stays valid beside it; when they run out, the next code appears. The copy button copies the code, and it is removed from the clipboard after 30 seconds like any secret. The key behind the codes is never shown: you can only replace it or remove it. In the list, an entry with a verification code carries a small **2FA** mark.

Changing it. The pencil beside the entry's name opens the form in the preview's place, with **Save** and **Cancel** where the pencil was (**Ctrl+S** or **Cmd+S** saves, **Esc** cancels). The form always offers URL, user name and password; **Add field** offers the PIN first, then the verification code, then a field you name yourself, which you can hide like a password. For a verification code, paste the setup key the site shows (spaces and case do not matter) or its `otpauth://` link; a value that is neither is flagged beside the box. Most sites show a QR code instead: take a screenshot of it with your system's screenshot tool, so that it is on the clipboard, and click the QR button beside the box; the button beside it reads the QR code from a saved image file. In the form, a **click** on a secret box's eye shows the value for 15 seconds, so you can see what you type: every key you press starts the 15 seconds again, and a second click, **Save**, **Cancel** or leaving the window hides it at once. The dice beside the password open the generator: type a length, choose the kinds of characters, and tick **Avoid look-alike characters** for a password you will type from a screen; **Use this password** puts it into the form. Everything is sealed on your computer before it leaves it. If somebody else changed the entry since you opened it, saving says so and shows you the entry as it is now.



Only one key can open My Passwords. Enrol a spare key, or losing this one loses your entries.

URL: https://code.example/login

User name: anna.berger

Password: •••••••••••••••••••••

PIN: ••••••

Length: 24

☑ Lower case ☑ Upper case ☑ Digits ☑ Symbols

☑ Avoid look-alike characters

••••••••••••••••••••••••

Verification code: ••

Recovery codes: ••••••••••••••••••

Notes: Two-factor with the hardware key

Not logged in.

Figure 18: Editing an entry: the secret boxes hidden until their eye is clicked, the generator open below the password, and Save and Cancel where the pencil was.

Organising. The ribbon creates entries and folders, renames, moves and deletes to the recycle bin. An entry moves into another store only if you can decrypt both, because it is sealed again for the new store's audience on the way. When a store's audience or a member's key has changed, a notice above the list says that its entries need sealing again; a member who may edit the store re-seals all of them with **Re-seal** and one PIN, and is told how many were re-sealed, how many were skipped because somebody changed them meanwhile, and which ones this key could not open. If only one key can open My Passwords, a notice recommends enrolling a spare key.

Who can decrypt a store. Being allowed to see a shared store is not the same as being able to decrypt it. A store is encrypted for its **audience**: the people and groups its **managers** chose from among those who may read it. Whoever creates a store is its first manager. **Encrypted for…** on the ribbon shows the audience to everybody who can see the store; a manager adds a reader or a group, removes somebody, or makes another person a manager, and **Save and re-seal** seals the store for the new audience with one PIN in the same step. Somebody removed can no longer fetch even the entries that were sealed to them, from the moment you save. A store must keep at least one manager; if every manager has left, a tenant administrator may appoint a new one from among its readers. My Passwords is always yours alone. In the **Repositories** tab, a store or an entry you can see but not decrypt is marked **Not encrypted for you**, and one you were just added to is marked **Not yet encrypted for you** until a member's next re-seal. A sealed document works the same way: whoever files it chooses its audience, and **Encrypted for…** changes it.

Inspect seal… in a sealed document's menu shows what its content is sealed to, without opening it: the format and encryption, each recipient's key with its holder (where you may see them), whether a key was revoked or is not attested, and whether the envelope still matches the audience. Somebody who left the audience but can still open this version, and a member the envelope does not name yet, are listed; in the desktop app a manager re-seals from there, in the browser, which holds no key, the desktop or phone app does. Each recipient opens to show how the key reaches it, and the ASN.1 structure is at the bottom, as `alcifra inspect` prints them. In the desktop app, **Which of my keys opens it?** tries the keys on this computer and then your hardware key (asking for its PIN as for any document), and **Try a key file…** tries a key you pick; either says which key opened it and discards the content at once. **Inspect a file…** in the tools menu beside the help icon shows the same for a sealed file on your computer, without uploading it.

Importing. The import button reads the CSV file a password manager exports, on your computer: every row becomes an entry, sealed here. Only a row whose name and user name match an entry already in the folder is asked about — skip it, add it as a new version, or keep both. A column holding verification codes (named `totp`, `otpauth` or the like) becomes each entry's verification code. Afterwards the desktop offers to delete the export file, because it holds every password in plain text.

An entry you come across in the **Repositories** tree shows its fields read-only in the preview, opened with your key each time; changing it is the Passwords tab's job. In the **Repositories** list, of the desktop and of the web client alike, an entry with a verification code is marked as such; the web client never opens an entry, so the mark is all it shows. The browser extension fills a sign-in's code field with the current code of the entry you choose, and the command-line tool prints it too (below).

From the command line. `apass` reads the same stores in a terminal, opening each entry with your hardware key and its PIN, which it does not keep. It is installed with `dotnet tool install --global Alcifra.Pass`, and `apass login --url https://your.server` signs it in. An entry is named by its path, as the tree shows it:

```
apass show Mail/webmail              # every field, in the form's order
apass password Mail/webmail          # the password alone
apass otp Mail/webmail               # the current code; "valid 17 s" on the next line
apass otp Mail/webmail --code-only   # the code alone
apass password Mail/webmail -c       # onto the clipboard, not the screen
```

With `-c` (`--clip`), `apass password` and `apass otp` put the secret on the clipboard instead of printing it, and take it off again after 45 seconds (a code, when it expires) — but only if it is still there, so anything you copied in the meantime stays.

`apass show` labels each field as the form does and shows a verification code as the code valid now, never its setup. `--store` names a shared store (the default is My Passwords); `apass ls` and `apass find` list what there is.

The Tab key completes commands, store names and paths once your shell loads the completion script. For zsh, add this line to `~/.zshrc`; for bash, write `bash` instead of `zsh` and add it to `~/.bashrc`:

```
eval "$(apass completion zsh)"
```

`apass show Ma` then completes to `Mail/`, and the next Tab goes on inside that folder. Without the line, Tab completes local files instead. Completion stays silent when you are not signed in.

The sign-in lasts while you work: every command extends it, it ends after 15 minutes without one, and after 12 hours you approve again whatever you were doing. `apass` and `alcifra` share it, and `apass logout` ends it.

The browser extension. The extension fills sign-ins from your stores. It is not in a browser's store yet, so you load it yourself, in Chrome or Edge:

1. Open `chrome://extensions` (or `edge://extensions`), switch on **Developer mode**, choose **Load unpacked**, and pick the `browser-extension/chrome` folder of the source repository. The extension's id is shown on its card.
2. Connect it to a client. In the desktop client, open **Browser autofill…** in the account menu and enter the id. Or, from a terminal:

   ```
   apass browser install chrome --extension-id <the id>
   ```
3. Restart the browser.

The extension asks the desktop client first, then `apass`. With `apass`, it can fill only while you are signed in: after 15 minutes without a command or a fill, run `apass login` again.

Bookmarks in your browser. The browser extension also keeps your bookmarks in step. The first time it syncs, it makes a file called Bookmarks in your My Documents, and that file appears as a folder of its own among the browser's bookmarks. Add, rename, move or delete bookmarks in that folder, and the change is saved half a minute later; a change made in another browser arrives when the browser starts, and every quarter of an hour. The file is the ordinary bookmark format every browser imports, so you can also download it and use it as it is.

A team shares bookmarks by putting a Bookmarks file in a folder it shares, and the folder's rights decide who gets it. If you may only read that folder, a change you make in the browser is undone at the next sync. A shared file holds web addresses only; a script address (a bookmarklet) works only in your own file.

When the same bookmark was changed in two places at once, the browser's change wins and the other is kept in a folder called Conflicts. Deleting the browser's folder does not delete your bookmarks: it comes back at the next sync.

7.8 CI/CD secrets

A **CI/CD store** keeps the secrets and variables your build pipelines need, sealed to the hardware keys of the people allowed to read them, like a password store. Make one with New ▸ **CI/CD Store** in any folder; who it is sealed to is its audience, as for a shared password store. Its contents are **CI/CD secrets**, and each one is a rotation unit: the values that change together, such as a signing certificate and its password, kept together.

A secret names its **targets**, the repositories it is set on (`github:owner/repository`, or `github:owner/repository@environment` for one environment), and holds any number of keys. Each key is a **secret** typed as text, a **file** kept as the bytes it was (a `.p12`, say) and sent base64-encoded, or a **variable**, which is not secret and is stored in the clear. In a CI/CD store, New ▸ **CI/CD Secret** opens the editor: its targets one per line, and a row per key. A file comes from **Choose file…** on its row, or is dropped anywhere on the window, where it becomes a new file key named after it.

Selecting a secret shows its targets, key names and variables at once, with no key involved. **Open with key** opens the values with your hardware key, once: a text value then shows only while its eye is held and copies like a password; a file has **Save as…**, which writes it back exactly as it was. Every value shown, copied or saved is recorded by key name, never with the value. **Edit…** changes the secret in one step, refused if somebody changed it meanwhile.

From the command line, `apass cicd apply <store>` sets what is missing or stale on every target: one PIN for the run, each value handed to the platform's own tool on its standard input and never printed. `apass cicd apply <store> --dry-run` reports what it would set and needs no key; `apass cicd show` prints or saves one value; `apass cicd set` makes or replaces a secret.

7.9 On a phone or a tablet

The same address, in the same browser, gives you a layout built for the screen you are holding. There is no separate mobile site and nothing to install — and nothing is **taken away**: what the narrower layouts cannot show side by side, they show one at a time.

A tablet is recognised by its touch screen, not by its width. That distinction matters more than it sounds: a large tablet is about 1024 points across upright and 1366 across sideways, so measuring width alone would call the same device a tablet one way up and a desktop the other. Turning your tablet changes how many panes you get, and never which kind of device the archive thinks you have.

7.9.1 A phone: one pane, and a way back

The folder tree becomes a drawer, the folder’s contents fill the screen, and a document opens over the top with its own **Preview**, **Details** and **Comments** tabs — so the panes the wide layout drops are a tap away rather than gone.

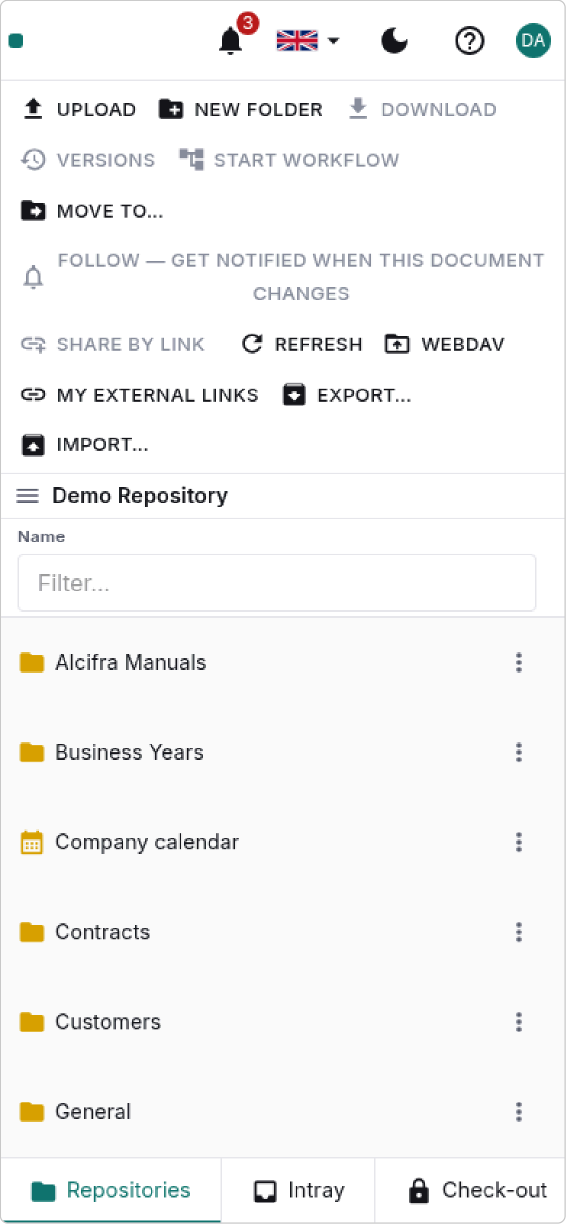

3
DA
UPLOAD
NEW FOLDER
DOWNLOAD
VERSIONS
START WORKFLOW
MOVE TO...
FOLLOW — GET NOTIFIED WHEN THIS DOCUMENT CHANGES
SHARE BY LINK
REFRESH
WEBDAV
MY EXTERNAL LINKS
EXPORT...
IMPORT...
Demo Repository
Name
Filter...
Alcifra Manuals
Business Years
Company calendar
Contracts
Customers
General
Repositories
Intray
Check-out

Figure 19: A folder’s contents on a phone, filling the screen. A single tap opens — there is no double-click here.

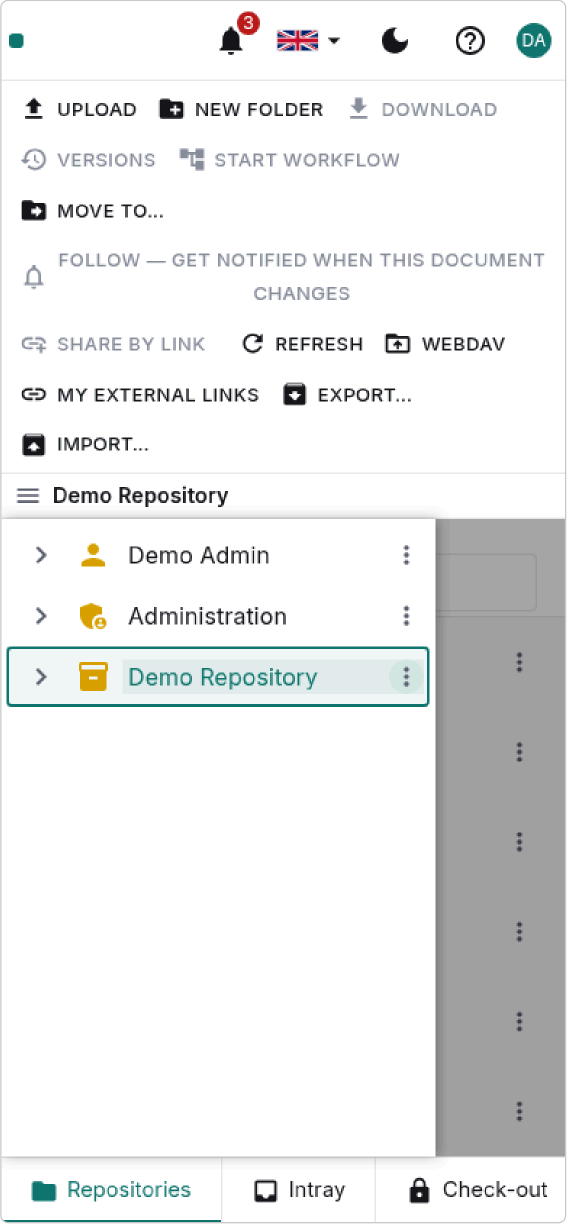

3
DA
UPLOAD
NEW FOLDER
DOWNLOAD
VERSIONS
START WORKFLOW
MOVE TO...
FOLLOW — GET NOTIFIED WHEN THIS DOCUMENT CHANGES
SHARE BY LINK
REFRESH
WEBDAV
MY EXTERNAL LINKS
EXPORT...
IMPORT...
Demo Repository
Demo Admin
Administration
Demo Repository
Repositories
Intray
Check-out

Figure 20: The tree, slid in from the left. Choosing a folder closes it again.

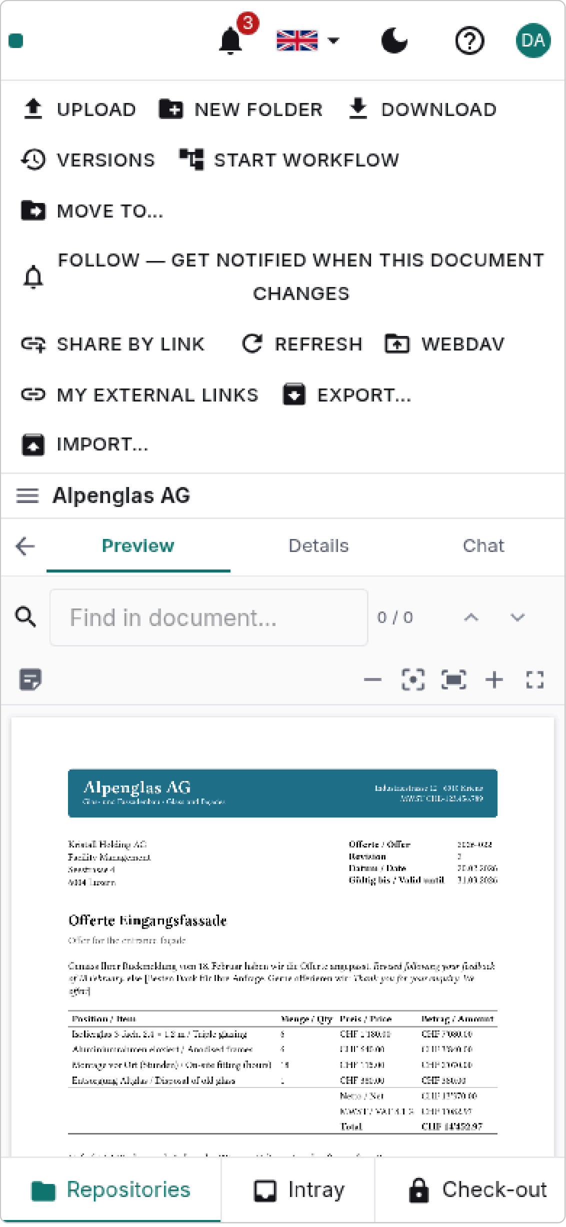
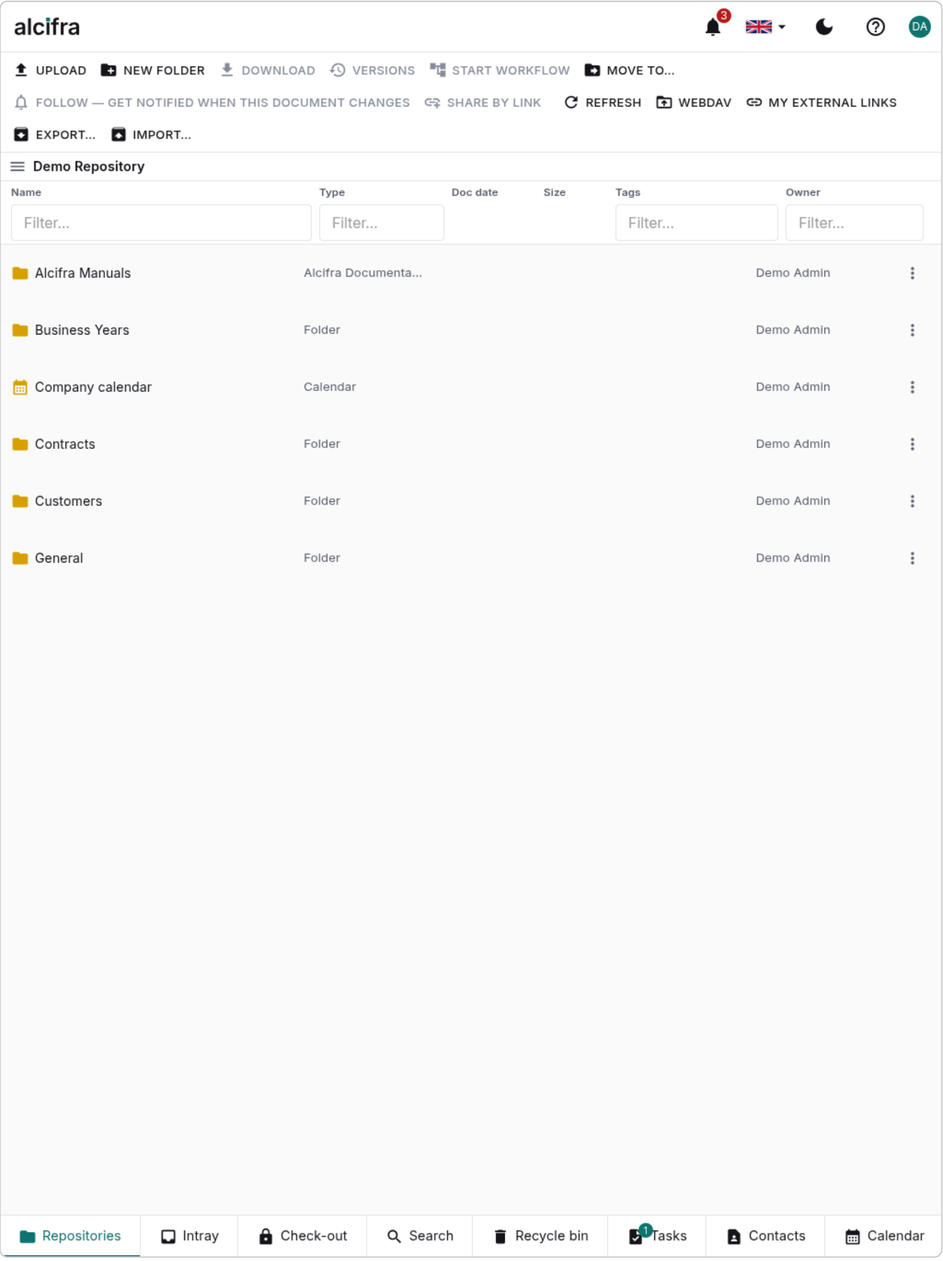

3
DA
UPLOAD
NEW FOLDER
DOWNLOAD
VERSIONS
START WORKFLOW
MOVE TO...
FOLLOW — GET NOTIFIED WHEN THIS DOCUMENT CHANGES
SHARE BY LINK
REFRESH
WEBDAV
MY EXTERNAL LINKS
EXPORT...
IMPORT...
Alpenglas AG
Preview
Details
Chat
Find in document...
0 / 0
Alpenglas AG
Offerte Eingangsfassade
Repositories
Intray
Check-out

Figure 21: A document, full screen, with its three tabs. Back returns to the list.

> **A single tap navigates on a phone.** On a desktop a click selects and a double click opens, because there is a second pane to show the selection in. On a phone there is not, so a tap goes straight in.

7.9.2 A tablet: one pane upright, two sideways

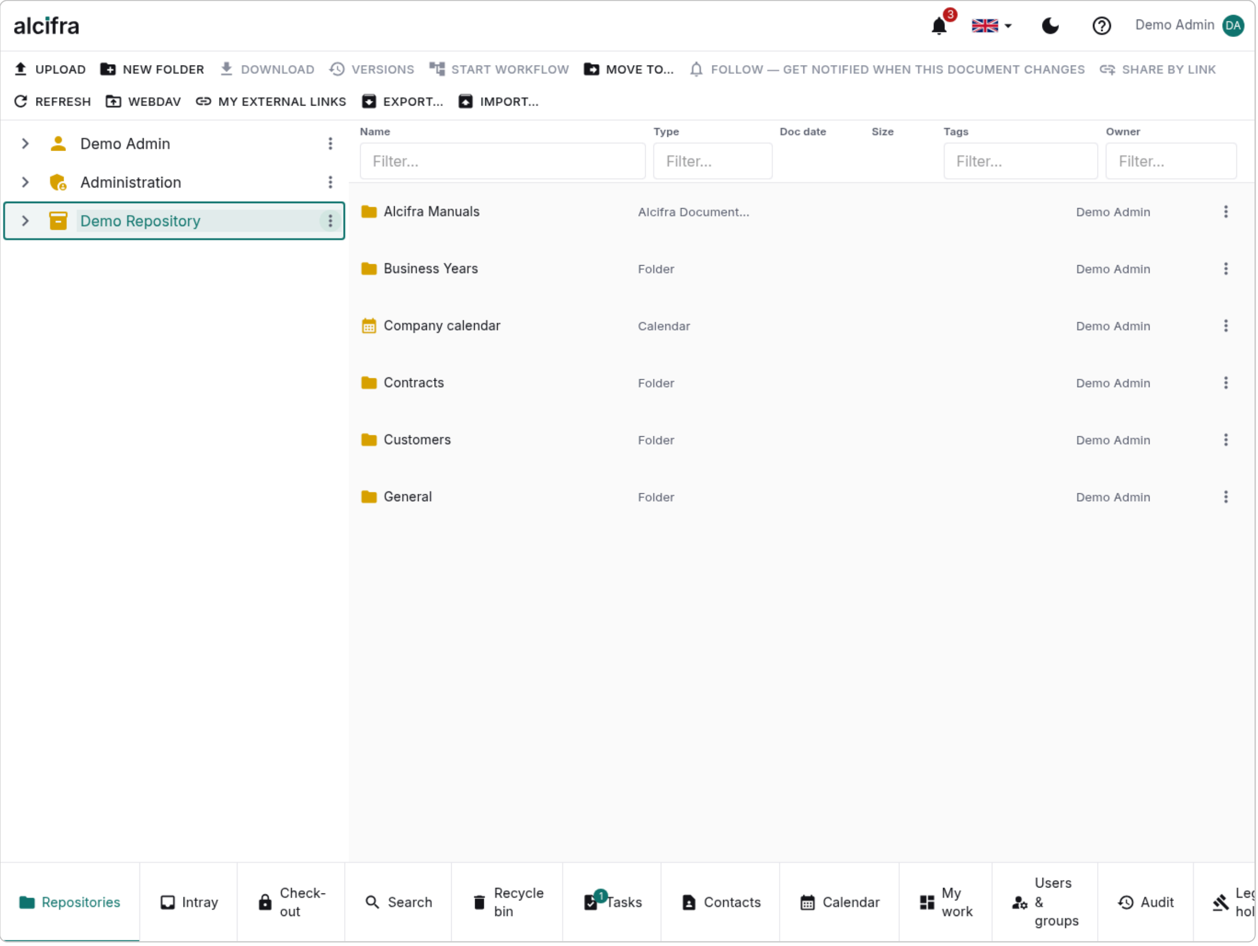
Held upright, a tablet behaves exactly as a phone does — one pane, the tree in a drawer.


alcifra
3
DA
UPLOAD
NEW FOLDER
DOWNLOAD
VERSIONS
START WORKFLOW
MOVE TO...
FOLLOW — GET NOTIFIED WHEN THIS DOCUMENT CHANGES
SHARE BY LINK
REFRESH
WEBDAV
MY EXTERNAL LINKS
EXPORT...
IMPORT...
Demo Repository
Filter...
Repositories
Intray
Check-out
Search
Recycle bin
1
Tasks
Contacts
Calendar

Name	Type	Doc date	Size	Tags	Owner
Alcifra Manuals	Alcifra Documenta...				Demo Admin
Business Years	Folder				Demo Admin
Company calendar	Calendar				Demo Admin
Contracts	Folder				Demo Admin
Customers	Folder				Demo Admin
General	Folder				Demo Admin

Figure 22: Upright: one pane at a time, the same shape as a phone.

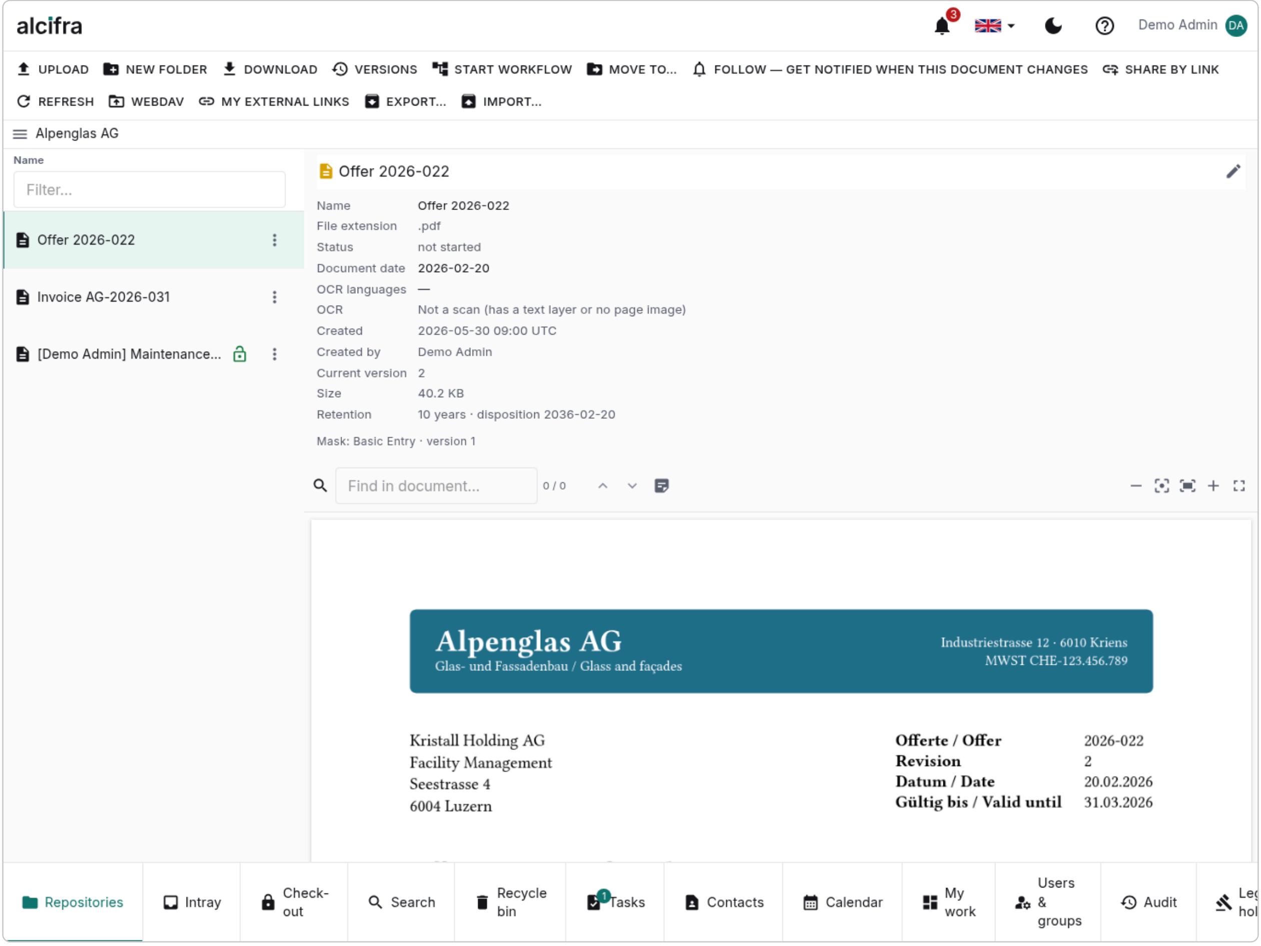
Turn it sideways and you get two of the three panes. While you are browsing, the tree sits beside the folder’s contents.


alcifra
3
Demo Admin
DA
UPLOAD
NEW FOLDER
DOWNLOAD
VERSIONS
START WORKFLOW
MOVE TO...
FOLLOW — GET NOTIFIED WHEN THIS DOCUMENT CHANGES
SHARE BY LINK
REFRESH
WEBDAV
MY EXTERNAL LINKS
EXPORT...
IMPORT...
Demo Admin
Administration
Demo Repository
Repositories
Intray
Check-out
Search
Recycle bin
1
Tasks
Contacts
Calendar
My work
Users & groups
Audit

Name	Type	Doc date	Size	Tags	Owner
Filter...	Filter...			Filter...	Filter...
Alcifra Manuals	Alcifra Document...				Demo Admin
Business Years	Folder				Demo Admin
Company calendar	Calendar				Demo Admin
Contracts	Folder				Demo Admin
Customers	Folder				Demo Admin
General	Folder				Demo Admin

Figure 23: Sideways, browsing: the tree beside the contents list.

Open a document and the tree steps aside for it, so what you came **from** stays next to what you opened — and the tree is one tap away from the ≡ button whenever you want it back.


alcifra
3
Demo Admin
DA
UPLOAD
NEW FOLDER
DOWNLOAD
VERSIONS
START WORKFLOW
MOVE TO...
FOLLOW — GET NOTIFIED WHEN THIS DOCUMENT CHANGES
SHARE BY LINK
REFRESH
WEBDAV
MY EXTERNAL LINKS
EXPORT...
IMPORT...
Alpenglas AG
Name
Filter...
Offer 2026-022
Invoice AG-2026-031
[Demo Admin] Maintenance...
Find in document...
0 / 0
Repositories
Intray
Check-out
Search
Recycle bin
1
Tasks
Contacts
Calendar
My work
Users & groups
Audit
Leg
hol

Offer 2026-022

Name	Offer 2026-022
File extension	.pdf
Status	not started
Document date	2026-02-20
OCR languages	—
OCR	Not a scan (has a text layer or no page image)
Created	2026-05-30 09:00 UTC
Created by	Demo Admin
Current version	2
Size	40.2 KB
Retention	10 years · disposition 2036-02-20

Mask: Basic Entry · version 1

Alpenglas AG
Glas- und Fassadenbau / Glass and façades

Industriestrasse 12 · 6010 Kriens
MWST CHE-123.456.789

Kristall Holding AG
Facility Management
Seestrasse 4
6004 Luzern

Offerte / Offer	2026-022
Revision	2
Datum / Date	20.02.2026
Gültig bis / Valid until	31.03.2026

Figure 24: Sideways, with a document open: the list beside the document. The tree is a tap away.

7.9.3 What each width shows

Screen	What you see
Desktop	All four panes — tree, contents, document, comments — resizable, and remembered per browser
Tablet, sideways	Two panes: tree and contents, or contents and document
Tablet, upright	One pane, tree in a drawer
Phone	One pane, tree in a drawer, document full screen

The comments pane is the first to go as the screen narrows, and the index-data pane the next — both come back as tabs inside the document view, so nothing becomes unreachable. It moves.

> **Buttons are labelled where you cannot hover.** On a desktop a ribbon button is an icon and its name appears on hover. A touch screen has no hover and therefore no tooltip, so the same buttons keep their labels — an unlabelled symbol you cannot interrogate is a guess.

This chapter is about the archive **in a browser**. For your device's own mail, contacts, calendar and files apps reaching the archive with no browser at all, see Section 8.

8 Your own device

Everything in this chapter is already yours. Alcifra installs **nothing** on your phone, tablet or laptop — the archive speaks the protocols those devices came with, so the mail app, address book, calendar and file manager you already use are the client. There is no app to find, no version to keep current, and nothing to uninstall when you leave.

That also means the archive obeys the same rules on your device as in the workbench. What you may see is what you may see; a folder you have no rights to simply is not there, rather than being there and refusing you.

8.1 What speaks what

On your device	Protocol	What you get
Mail program	**IMAP**	The archive as mail folders — see Section 6.2
Address book	**CardDAV**	Contacts sync both ways — see Section 7.3
Calendar	**CalDAV**	Calendars and your task deadlines — see Section 7.3
File manager	**WebDAV**	One mountable drive — see Section 6
Notes app	IMAP	Notes with real version history — see Section 6.2
Browser	—	The full workbench, on any screen — except opening encrypted content, see Section 14.1

8.2 A password for each device

Each device you connect — a phone, a laptop's mail program, a tablet's calendar — gets **its own password**, and none of them is your sign-in password. Open **Devices…** in the account menu:

- **Enrol a device…** asks for a name ("Anna's phone") and what it reaches — **Mail**, **Files**, **Calendars**, **Contacts**, all on unless you turn some off. The password is shown **once**, with a QR code of it, and every server address beside it, each with a **Copy** button.
- Where the device reaches mail, calendars or contacts, a second QR code opens a **setup profile** on a phone: scan it with the camera and the phone sets up the accounts itself. The link works **once, within ten minutes**, and is gone when used.
- **Change** renames a device or turns what it reaches on and off; **Revoke** cuts that device off at once, and only that one. **A lost phone costs one revocation**: your other devices and your sign-in stay as they are.
- **Encrypt mail content** seals what that device's mail program fetches to your own hardware key, so only a device that holds it can read it. Turning it on needs an enrolled, attested hardware key; where your organisation seals all mail it is shown on and cannot be turned off. Not every mail program can read sealed mail, which the dialog says beside the switch.

Passwords issued before devices existed — one for files, calendars and contacts, one for mail — appear in the list as **Earlier WebDAV/calendar password** and **Earlier IMAP password**. They keep working until you revoke them; enrol your devices, then revoke them.

8.3 Encryption is not optional

Point your devices at an `https://` address, and make sure the mail account uses **TLS** (port 993). This is not only good practice — some notes clients **silently refuse** a plaintext connection: they ask the server what it supports, dislike the answer and hang up, showing you nothing at all. A server that works perfectly from every other app can look broken from that one, and the reason never appears on screen.

> **If a device connects but shows nothing**, suspect the transport before the account. A wrong password produces an error you can read; a refused plaintext connection produces silence.

8.4 What each one does not give you

Naming the boundary is what stops a working feature looking broken:

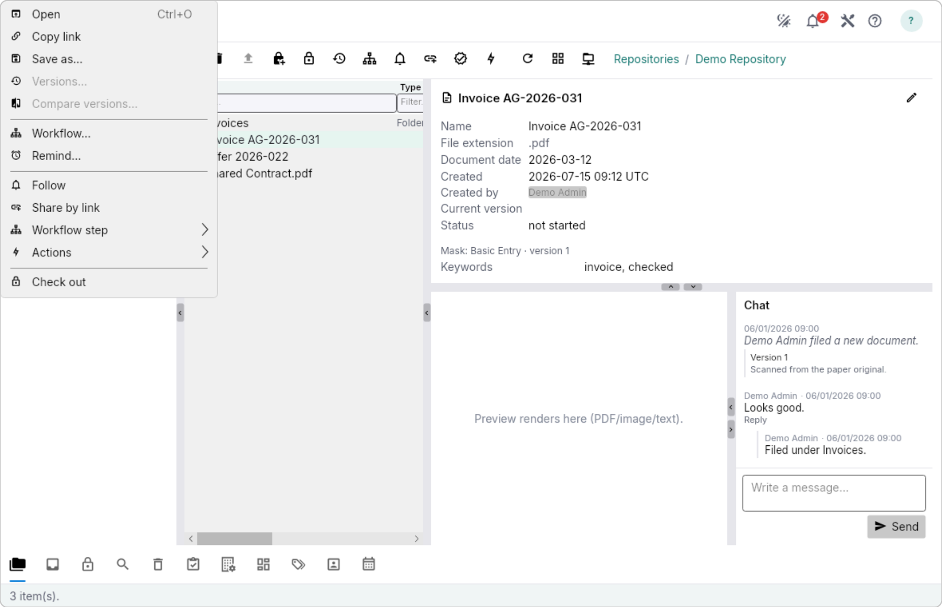
- **A mail program shows the archive, not the application.** No recycle bin, no workflow, no permissions dialog, no search across metadata — those live in the workbench.
- **A mounted drive shows the tree, not the history.** Versions, comments and index data are not files, so they do not appear; saving over a file makes a new version, which is the one piece of history the drive can express.
- **Calendars and address books carry their own items only.** A calendar holds appointments; it will not show you the documents filed beside it.
- **Task collections are read-only.** You can see what is waiting; approving it is recorded in the workbench, with who did it and why.

8.5 On a small screen

The workbench itself adapts rather than sending you to a separate mobile site, and it decides by what you are holding rather than by how wide the window is — a tablet is recognised by its **touch** screen, so turning it does not turn it into a laptop.

- **Phone, and a tablet held upright** — one pane at a time. The folder tree slides in from the left, the folder contents fill the screen, and opening a document covers it with **Preview**, **Details** and **Comments** tabs. A single tap navigates, rather than the desktop's click-then-double-click.
- **Tablet held sideways** — two panes. Browsing shows the tree beside the folder contents; opening a document swaps the tree for the document, so what you came from stays next to what you opened. The tree is always one tap away, from the ☰ button.
- **Laptop or desktop** — the full workbench, with resizable panes that remember their widths.

What you do to a document is in its context menu and the ribbon. The detail pane holds only what belongs to the details: the pencil that edits them (Save and Cancel take its place while you edit), the tags, the arrows that fold the pane away, and what state a review is in. Everything you do **to** the document — **Follow**, **Share by link**, **Remind**, **Compare versions**, **Manage access**, the next step of a review (**Approve**, **Reject**, …), and whatever a module adds, such as **Bookings…** for a meeting room — is in the document's context menu (a right-click on the desktop, the ⁝ at the end of the row on the web) and in the ribbon's document group, beside Open, Rename and Check out. The review's steps are grouped under **Workflow step**, what a module adds under **Actions**.



Open Ctrl+O
Copy link
Save as...
Versions...
Compare versions...
Workflow...
Remind...
Follow
Share by link
Workflow step
Actions
Check out
Repositories / Demo Repository
Type
Filter
Folder
voices
voice AG-2026-031
fer 2026-022
ared Contract.pdf
Write a message...
Send
3 item(s).

Invoice AG-2026-031

Name Invoice AG-2026-031
File extension .pdf
Document date 2026-03-12
Created 2026-07-15 09:12 UTC
Created by Demo Admin
Current version
Status not started

Mask: Basic Entry · version 1
Keywords invoice, checked

Preview renders here (PDF/image/text).

Chat

06/01/2026 09:00
Demo Admin filed a new document.
Version 1
Scanned from the paper original.

Demo Admin · 06/01/2026 09:00
Looks good.
Reply

Demo Admin · 06/01/2026 09:00
Filed under Invoices.

Figure 25: A document’s context menu in the desktop client: the document’s own actions — Follow, Share by link — then **Workflow step**, the next steps of a review in progress, and **Actions**, what a module offers for this document.

9 Metadata & classification

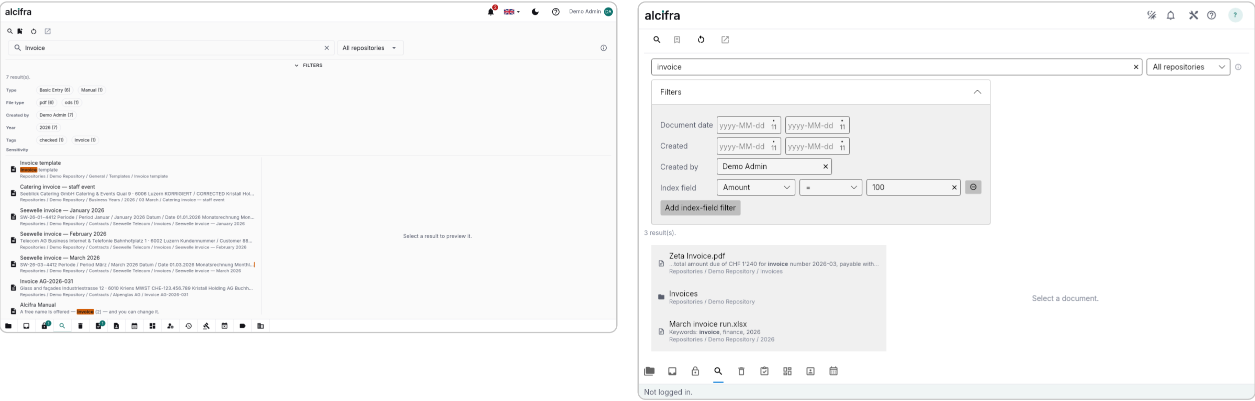
Each document has a **mask** (document type) that defines its **index fields** — typed metadata such as an invoice number or a date. Open the **detail** pane's edit toggle to fill them in. A document also carries a **document date** and one or more **OCR languages** used to make scans searchable. Where the automatic recognition decided a scan was not worth converting, the detail pane says why. To overrule it, check the document out: **Make searchable** is in the Check-out tab's ribbon.

A field that points at another document of a given kind, such as a video's thumbnail, is **picked**: type part of a name, and the documents of that kind are listed with their pictures. Where the field takes exactly one kind, **From a file…** files a new one beside the document and picks it.

> Index fields are validated by the mask — a field marked required must be filled, and format/range rules are enforced when you save. Well-known masks (Folder, Basic Entry, e-Mail) exist in every tenant; administrators can add more.

10 Search

The **Search** tab runs a full-text search across document **content**, **names**, and **index-field** values, ranked by relevance, with hits highlighted on the preview — so a distinctive word buried in a document's text (a product name on an invoice, say) finds exactly that document. **Refinement filters** and **facets** narrow the results — by document type, date, sensitivity, and more — and you can **save** a search to re-run or share it later.



Invoice

7 result(s).

Type: Basic Entry (6) · Manual (1)
File type: pdf (6) · ods (1)
Created by: Demo Admin (7)
Year: 2026 (7)
Tags: checked (1) · invoice (1)
Sensitivity

Invoice template
Invoice template
Repositories / Demo Repository / General / Templates / Invoice template

Catering invoice — staff event
Seeblick Catering GmbH Catering & Events Quai 9 · 6006 Luzern KORRIGIERT / CORRECTED Kristall Hol...
Repositories / Demo Repository / Business Years / 2026 / 03 March / Catering invoice — staff event

Seewelle invoice — January 2026
SW-26-01-4412 Periode / Period Januar / January 2026 Datum / Date 01.01.2026 Monatsrechnung Mon...
Repositories / Demo Repository / Contracts / Seewelle Telecom / Invoices / Seewelle invoice — January 2026

Seewelle invoice — February 2026
Telecom AG Business Internet & Telefonie Bahnhofplatz 1 · 6002 Luzern Kundennummer / Customer 88...
Repositories / Demo Repository / Contracts / Seewelle Telecom / Invoices / Seewelle invoice — February 2026

Seewelle invoice — March 2026
SW-26-03-4412 Periode / Period März / March 2026 Datum / Date 01.03.2026 Monatsrechnung Monthl...
Repositories / Demo Repository / Contracts / Seewelle Telecom / Invoices / Seewelle invoice — March 2026

Invoice AG-2026-031
Glass and façades Industriestrasse 12 · 6010 Kriens MWST CHE-123.456.789 Kristall Holding AG Buchh...
Repositories / Demo Repository / Contracts / Alpenglas AG / Invoice AG-2026-031

Alcifra Manual
A free name is offered — Invoice (2) — and you can change it.

Select a result to preview it.

invoice

Document date: yyyy-MM-dd · yyyy-MM-dd
Created: yyyy-MM-dd · yyyy-MM-dd
Created by: Demo Admin
Index field: Amount · = · 100

3 result(s).

Zeta Invoice.pdf
...total amount due of CHF 1'240 for **invoice** number 2026-03, payable with...
Repositories / Demo Repository / Invoices

Invoices
Repositories / Demo Repository

March invoice run.xlsx
Keywords: **invoice**, finance, 2026
Repositories / Demo Repository / 2026

Select a document.

Figure 26: Full-text search with refinement filters and facets, in the web (left) and desktop (right) clients.

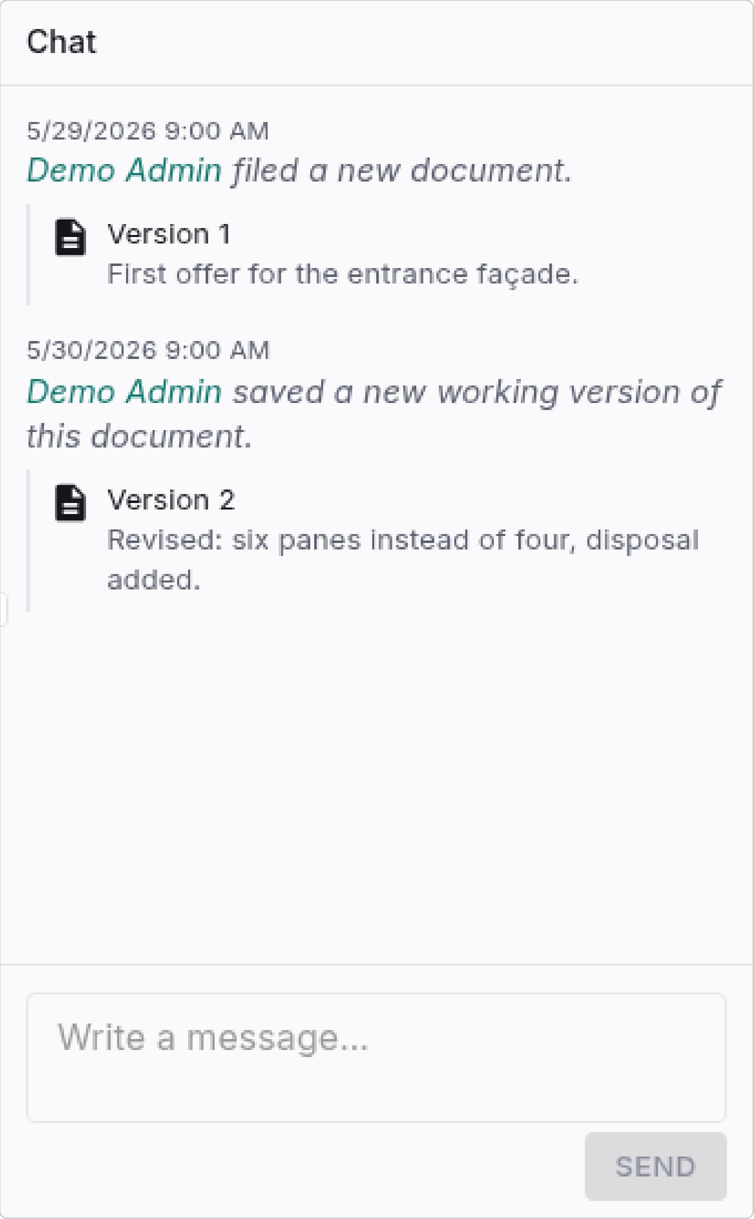
Parts of words. Search matches whole words first, which is what keeps the most relevant document at the top. If that finds nothing, the search is retried looking for your text **inside** words — so `montage` finds a document that only ever says `Montagehalterung`, and `sechskant` finds the longer compound containing it. This matters most in German, where one word does the work of several. The second attempt is slower and its ranking is flatter, so it only runs when the first found nothing at all.

What the field does not do. Operators are not interpreted: quotes, `AND`, `field:value` and wildcards are searched as literal text, deliberately — a query can never fail over a stray character. Structured narrowing — by document type, owner, year, tags, or a specific index field — lives in the refinement filters beside the results, not in the text box.

11 Collaboration

Documents are collaborative: post **comments** in a feed, attach **annotations** (sticky notes, highlights, and shapes) onto the preview, and **follow** a document or folder to be **notified** of activity. Set yourself a **reminder**, and track everything assigned to you on the **My work** dashboard. To edit a document exclusively, **check it out** — others see it is locked — then **check in** your changes as a new version.

The chat thread. Every document has its own thread, shown beside the preview. Use it for the conversation about the document — a question about a figure, a note that a revision is on its way — so that discussion stays with the document instead of in someone's mailbox. Replies nest under the message they answer, and the thread also records what happened to the document: a new version, a filing, and other activity appear in the same timeline, so reading it tells you both what people said and what changed.



Chat

5/29/2026 9:00 AM

Demo Admin filed a new document.

Version 1

First offer for the entrance façade.

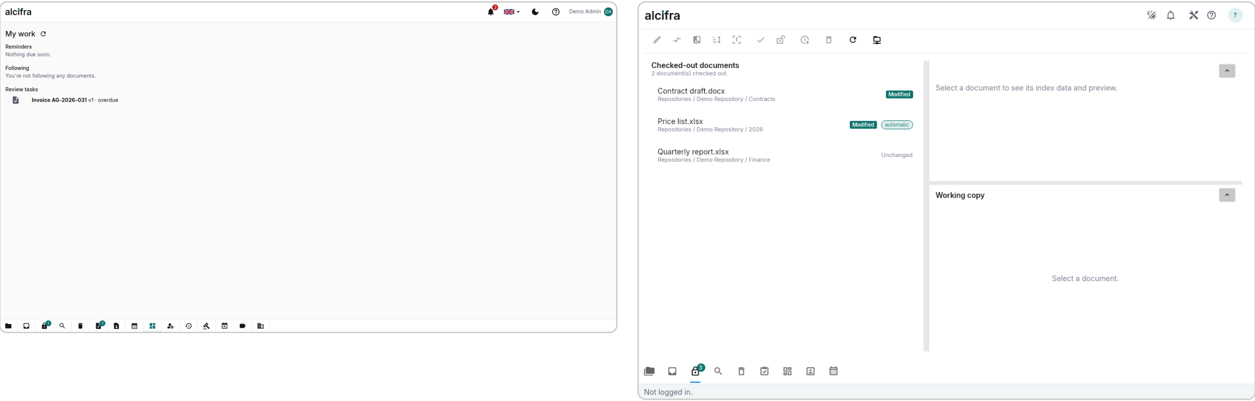
5/30/2026 9:00 AM

Demo Admin saved a new working version of this document.

Version 2

Revised: six panes instead of four, disposal added.

Figure 27: The chat pane on a document: a threaded conversation interleaved with the document's own activity — here a question and its reply, followed by the two versions as they were saved.



alcifra
Demo Admin DA

My work

Reminders
Nothing due soon.

Following
You're not following any documents.

Review tasks
Invoice AG-2026-031 v1 · overdue

alcifra

Checked-out documents
2 document(s) checked out.

Contract draft.docx Repositories / Demo Repository / Contracts	Modified
Price list.xlsx Repositories / Demo Repository / 2026	Modified automatic
Quarterly report.xlsx Repositories / Demo Repository / Finance	Unchanged

Select a document to see its index data and preview.

Working copy

Select a document.

Not logged in.

Figure 28: Left: the **My work** dashboard. Right: the **Check-out** tab listing documents locked for exclusive editing — here one edited (*Modified*) and one untouched (*Unchanged*).

Seeing your edit before you commit it. The **Check-out** tab shows what you are **about to check in**, not what is already archived. Select a document you hold and the pane beside the list shows its index data and a preview of your **working copy** — the file as you have edited it. That is the version the **Check in** action will create, so you can look at it before committing, without leaving the tab to go and find the archived copy (which would show you the one thing that is certainly not your edit).

A document you have edited is marked **Modified**, and only those offer the actions that make sense for an edit:

Action	What it does
Compare	An inline, side-by-side view of what changed between the archived version and your working copy.
Beyond Compare	Opens the same two files in Beyond Compare, if you use it — a desktop-only convenience, since a browser is not allowed to launch another application. Not affiliated with Alcifra the button points you to the vendor if you do not have it.
Check in	Files your working copy as a new version and releases the lock.
Discard	Throws the working copy away and releases the lock, leaving the archived version untouched.

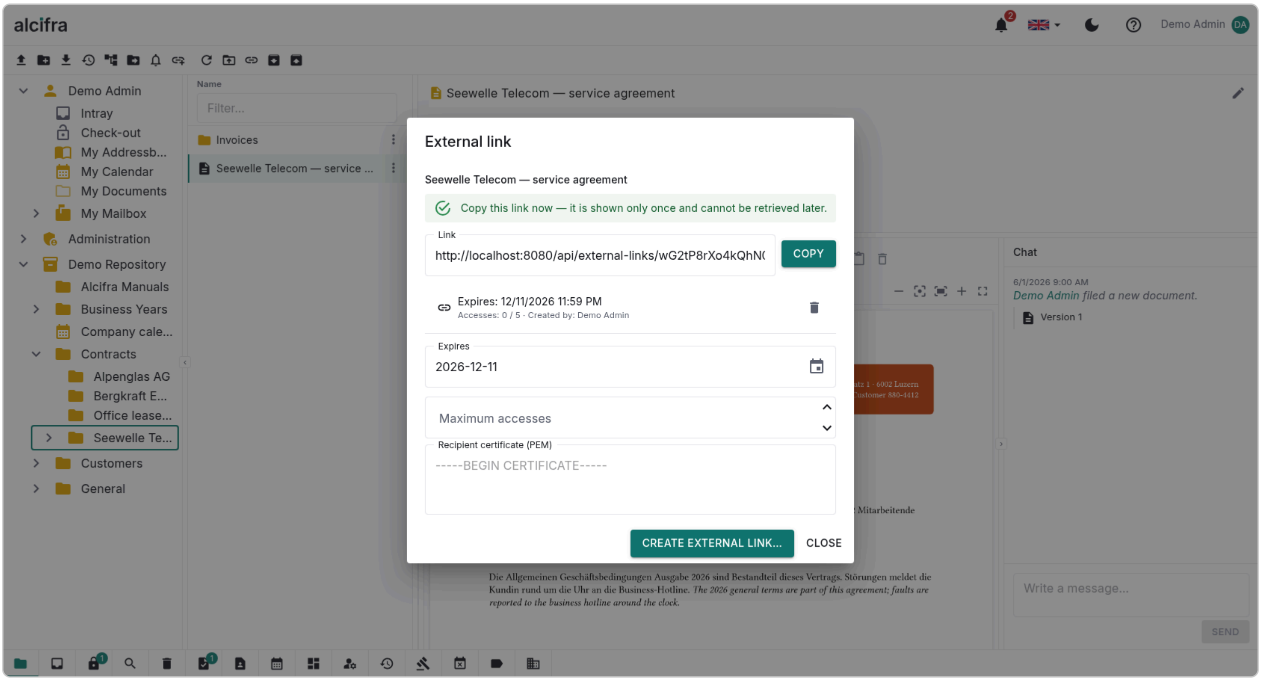
> If you have not saved anything to your working copy yet, the preview stays empty rather than showing the archived version. There is genuinely nothing to preview, and showing the archived file there would be answering a different question.

12 Sharing outside Alcifra

Everything above assumes the other person has an account. An **external link** is for when they do not: a plain URL that opens one document, for anyone who has it, with no sign-in. Use it for the customer who needs the signed contract or the auditor who needs one invoice — not as a general-purpose way to move documents around.

Creating one. Select the document and choose **Share by link** — in its context menu or in the ribbon. Choose when it should expire and how many times it may be opened, then create it.

> **The URL is shown once, at creation, and never again.** Copy it before you close the dialog. It appears in no list afterwards — deliberately, because a list is read far more widely, and far more casually, than the moment you deliberately created something.



alcifra
Demo Admin
Demo Admin
Intray
Check-out
My Addressb...
My Calendar
My Documents
My Mailbox
Administration
Demo Repository
Alcifra Manuals
Business Years
Company cale...
Contracts
Alpenglas AG
Bergkraft E...
Office lease...
Seewelle Te...
Customers
General
Name
Filter...
Invoices
Seewelle Telecom — service ...
Seewelle Telecom — service agreement
Chat
Version 1
Write a message...
SEND
COPY
CREATE EXTERNAL LINK...
CLOSE

External link

Seewelle Telecom — service agreement

Copy this link now — it is shown only once and cannot be retrieved later.

Link: http://localhost:8080/api/external-links/wG2tP8rXo4kQhN(

Expires: 12/11/2026 11:59 PM
Accesses: 0 / 5 · Created by: Demo Admin

Expires: 2026-12-11

Maximum accesses

Recipient certificate (PEM): -----BEGIN CERTIFICATE-----

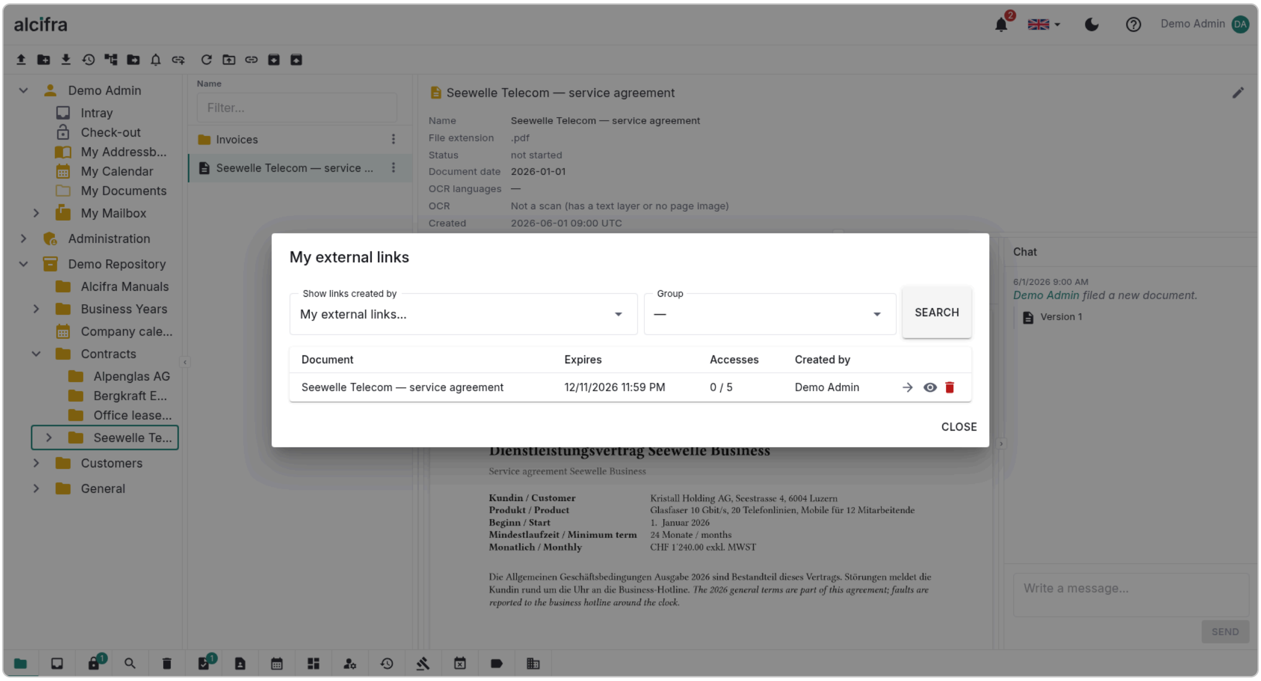
6/1/2026 9:00 AM
Demo Admin filed a new document.

Figure 29: Creating an external link. The URL is revealed once, here, and cannot be retrieved later.

Managing what you have shared. My external links on the ribbon lists your live links: which document, when it expires, how many times it has been opened. From a row you can jump to the document, review a link's details, or **revoke** it. An administrator sees everyone's, filtered by user or group.

A link near its expiry can be **extended** — but only within 30 days of lapsing, by at most 90 days, and measured from today rather than added to whatever time remains. So extending is a deliberate renewal, not a way to accumulate an indefinite link.

> **Revoking does not delete the row.** It stamps it revoked and leaves the record standing — who shared what, with effect from when. After a link has leaked, that record is exactly what the investigation needs, and it would be gone if revoking tidied it away.


My external links

Show links created by: My external links...

Group: —

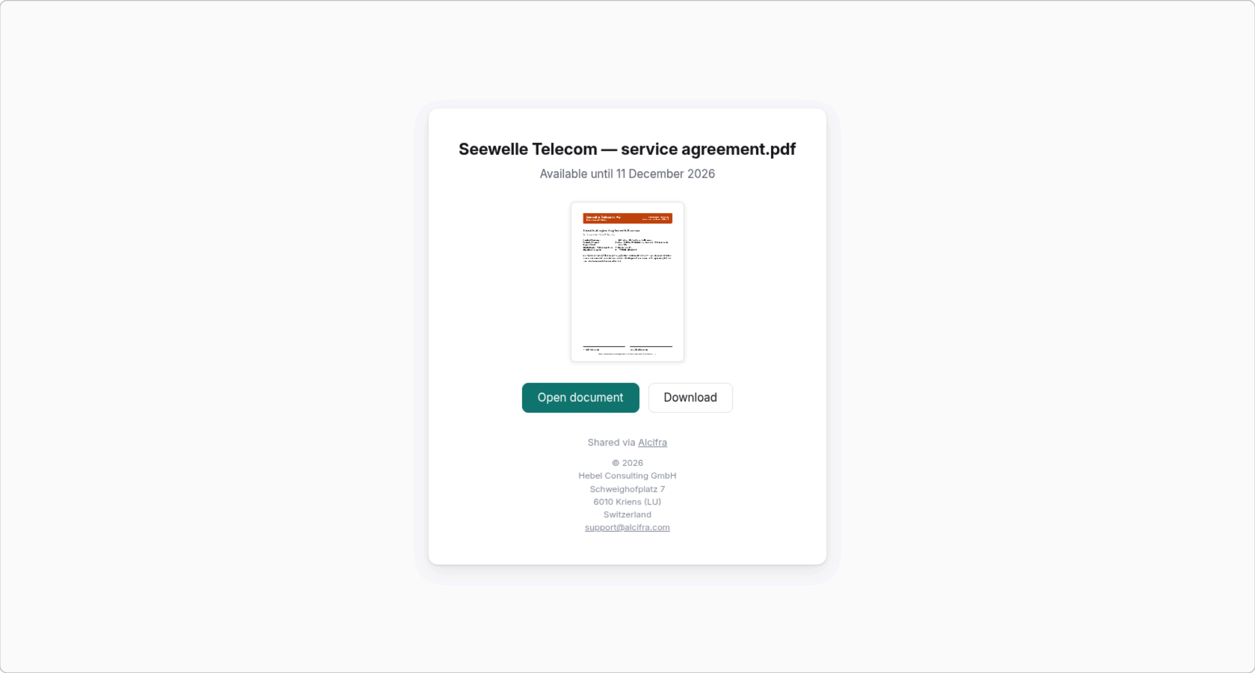
Document	Expires	Accesses	Created by
Seewelle Telecom — service agreement	12/11/2026 11:59 PM	0 / 5	Demo Admin

Figure 30: **My external links** — every live link you have shared, with go-to, details and revoke on each row.

What the recipient sees. A single page with the document's name, a picture of its first page, and buttons to open or download it — with the number of pages marked on the picture when there is more than one, so they know what they are about to open. Nothing else: no tree, no navigation, no route to any other document.

The picture is drawn when you create the link, not when the recipient opens it, so there is nothing to wait for. A document with no picture to draw — an archive, a binary — simply shows its name and the buttons.

An unknown, expired, exhausted or revoked link all produce the **same** response. That is deliberate — telling a stranger which of those they hit would confirm a real link exists and hint at how to reach a usable one.



Seewelle Telecom — service agreement.pdf

Available until 11 December 2026

Shared via Alcifra

© 2026
Hebel Consulting GmbH
Schweighofplatz 7
6010 Kriens (LU)
Switzerland
support@alcifra.com

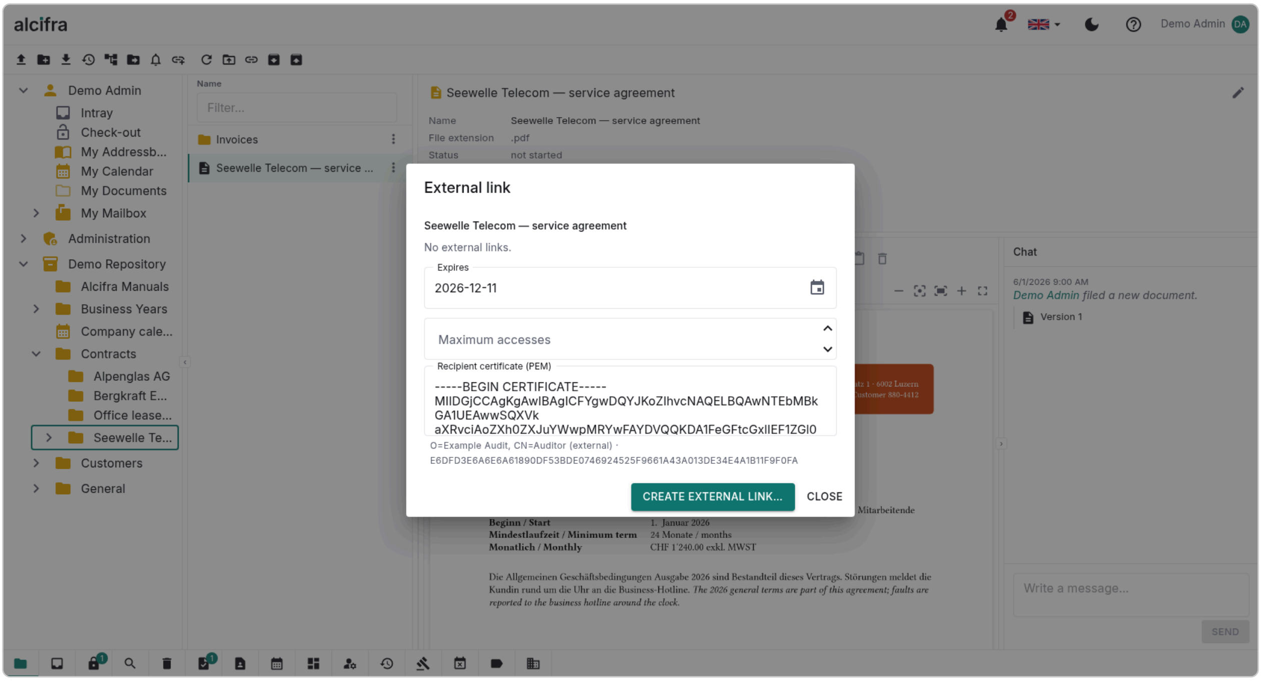
Figure 31: The recipient’s view: the document’s name and a picture of its first page. This one is a single page, so it carries no page-count marker. No account, one document, nothing else reachable from it.

12.1 Sharing to one named recipient

An external link is open by construction: whoever holds the URL can open the document. When that is too much, the link can be **addressed to a certificate** instead — and then only the holder of the matching private key can read what it delivers.

How you do it. In the create dialog, paste the recipient’s certificate (their `.pem` or `.crt` — the certificate only, never a private key) into the certificate field before creating the link. The dialog reads it and shows you back the **subject** and the **fingerprint** it found.

> **Check the subject and fingerprint before you send the link.** They are your only confirmation that the certificate is the one you meant, and addressing a document to the wrong key cannot be undone: the person you intended will not be able to open it, and whoever holds the key you actually used will. The remedy is to revoke the link and create a new one — an existing link cannot be re-addressed.


alcifra
Demo Admin
Intray
Check-out
My Addressb...
My Calendar
My Documents
My Mailbox
Administration
Demo Repository
Alcifra Manuals
Business Years
Company cale...
Contracts
Alpenglas AG
Bergkraft E...
Office lease...
Seewelle Te...
Customers
General
Name
Filter...
Invoices
Seewelle Telecom — service ...
Chat
Write a message...
SEND

Seewelle Telecom — service agreement

Name: Seewelle Telecom — service agreement
File extension: .pdf
Status: not started

6/1/2026 9:00 AM
Demo Admin filed a new document.
Version 1

External link

Seewelle Telecom — service agreement

No external links.

Expires: 2026-12-11

Maximum accesses

Recipient certificate (PEM):

```
-----BEGIN CERTIFICATE-----
MIIDGjCCAgKgAwIBAgICFYgwDQYJKoZIhvcNAQELBQAwNTEbMBk
GA1UEAwwSQXVk
aXRvciAoZXh0ZXJuYWwpMRYwFAYDVQQKDA1FeGFtcGxlIEF1ZGl0
```

O=Example Audit, CN=Auditor (external) · E6DFD3E6A6E6A61890DF53BDE0746924525F9661A43A013DE34E4A1B11F9F0FA

CREATE EXTERNAL LINK...
CLOSE

Figure 32: Addressing a link to a recipient: the pasted certificate is described back as its subject and fingerprint, which is what to check before sending the URL.

What the recipient gets. The landing page looks the same, but the download hands them a `.p7m` — the document inside an encrypted envelope addressed to their certificate. Mail software that handles S/MIME opens it with their own key; a browser or file manager cannot, and will simply save a file they must open with the right program. Tell them to expect that, because a `.p7m` nobody warned them about reads as a broken download.

Where the archive itself is encrypted, this is the only way to share outward. On such an installation the archive serves no readable content at all, so a link with no recipient is one it could never fulfil — and the dialog **refuses to create one**, rather than letting you send a URL that fails when it is opened. The certificate is not an option there but the requirement. Two smaller differences follow from the same cause: the recipient's page carries **no picture of the first page** (the picture is content too), and **Open document** has nothing to show inline, so downloading the envelope is the whole of what the link does.

12.2 The controls an administrator holds

Sharing outward is the one action that leaves the system, so it is gated twice and can be stopped in one move.

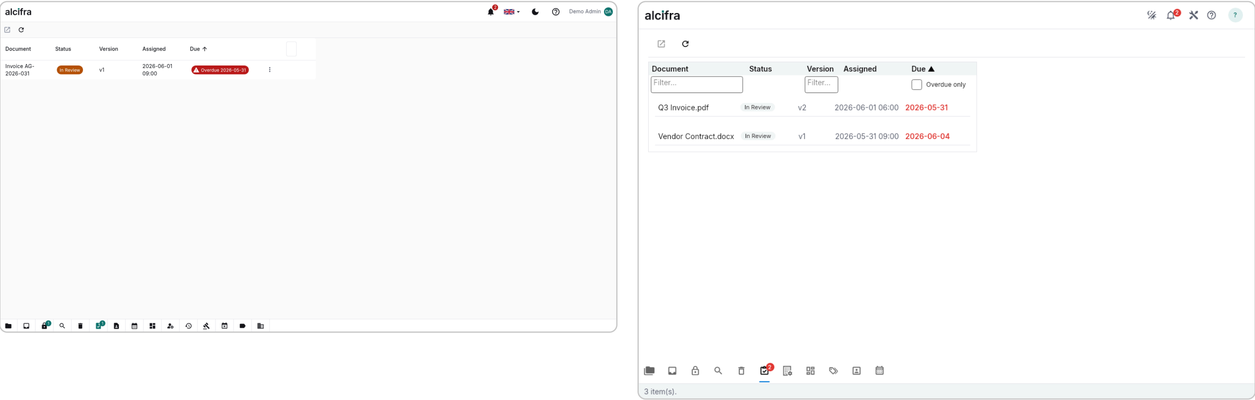
- **Two gates on creating a link.** The tenant-wide **Allow external links** switch must be on, and the individual needs the **Create external link** right, granted per user or group from **Users & groups**. Either alone is enough to prevent sharing; both must be present to allow it. A service account can never create one — only a person shares, so there is always someone accountable for a link's existence.
- **The kill switch.** Turning **Allow external links** off is checked when a link is **opened**, not only when one is created — so it stops every link already out in the world, not merely future ones. This is the control to reach for when something has leaked, and it is worth knowing about before you need it.
- **The caps you set. Maximum link lifetime** (default 180 days) and **default access count** (default 5) are the rails everyone else shares within. Tighten them to your own policy rather than relying on people choosing well.

13 Workflow & records

Approval workflow. Submit a document for review and it moves through a fixed state machine — Draft → In Review → Approved / Rejected → Released. The detail pane says which state a document is in; the next step you may take — approve, reject, reassign, release — is under **Workflow step** in its context menu and the ribbon. Reviewers act on their **Tasks** tab — sortable by any column (a click on a header; the default puts the nearest deadline on top, overdue in red first) and narrowable through the visible filter row: document and version text, plus an **Overdue only** switch. Reviews can be reassigned, and overdue reviews escalate.

Records management. A **legal hold** freezes documents so they cannot be changed or deleted. **Retention** policies dispose of documents once their retention period ends (with review before disposition, if required). Deleted documents rest in the **Recycle bin**, from which an administrator can restore them or purge them permanently.

On the desktop the number of tasks waiting for you also rides on the application icon itself — a badge on the macOS Dock, an overlay on the Windows taskbar — so a window you have minimised still tells you there is something to do.

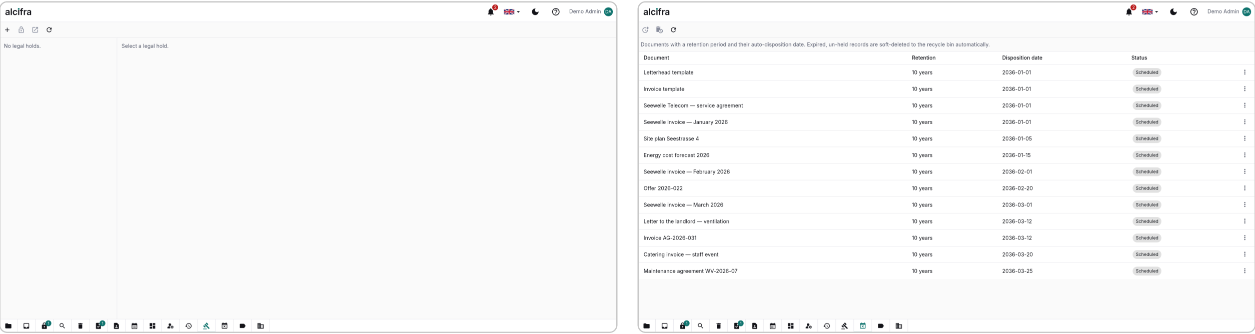


Document	Status	Version	Assigned	Due ↑	
Invoice AG-2026-031	In Review	v1	2026-06-01 09:00	Overdue 2026-05-31	⋮

Document	Status	Version	Assigned	Due ▲
Filter...		Filter...		☐ Overdue only
Q3 Invoice.pdf	In Review	v2	2026-06-01 06:00	2026-05-31
Vendor Contract.docx	In Review	v1	2026-05-31 09:00	2026-06-04

3 item(s).

Figure 33: The Tasks tab — a reviewer's approval queue — in the web (left) and desktop (right) clients.



No legal holds.

Select a legal hold.

Documents with a retention period and their auto-disposition date. Expired, un-held records are soft-deleted to the recycle bin automatically.

Document	Retention	Disposition date	Status	
Letterhead template	10 years	2036-01-01	Scheduled	⋮
Invoice template	10 years	2036-01-01	Scheduled	⋮
Seewelle Telecom — service agreement	10 years	2036-01-01	Scheduled	⋮
Seewelle invoice — January 2026	10 years	2036-01-01	Scheduled	⋮
Site plan Seestrasse 4	10 years	2036-01-05	Scheduled	⋮
Energy cost forecast 2026	10 years	2036-01-15	Scheduled	⋮
Seewelle invoice — February 2026	10 years	2036-02-01	Scheduled	⋮
Offer 2026-022	10 years	2036-02-20	Scheduled	⋮
Seewelle invoice — March 2026	10 years	2036-03-01	Scheduled	⋮
Letter to the landlord — ventilation	10 years	2036-03-12	Scheduled	⋮
Invoice AG-2026-031	10 years	2036-03-12	Scheduled	⋮
Catering invoice — staff event	10 years	2036-03-20	Scheduled	⋮
Maintenance agreement WV-2026-07	10 years	2036-03-25	Scheduled	⋮

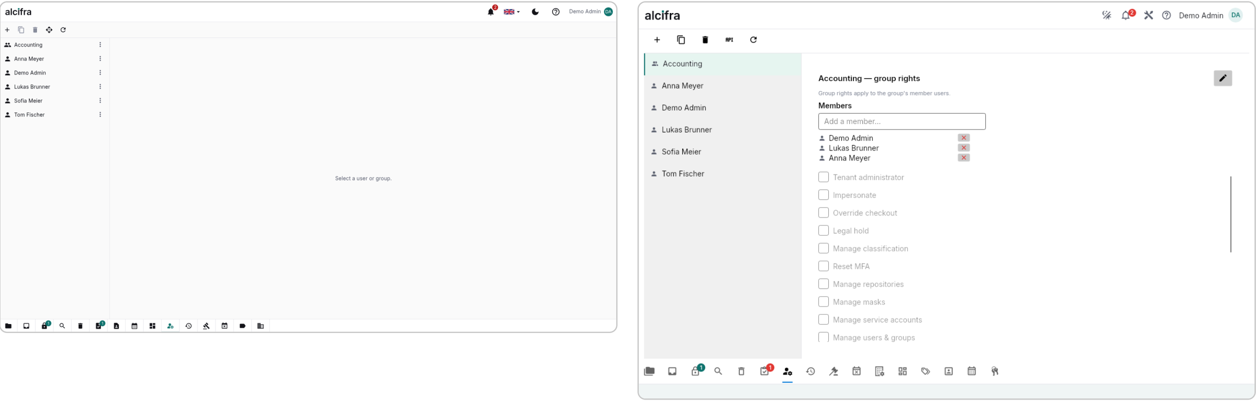
Figure 34: Records management: **legal holds** (left) freeze documents; **retention** (right) governs disposition.

14 Administration & account

Administrators manage **users & groups** and the **rights** granted to them, configure **tenant** settings, and review the tamper-evident **audit trail**. They also curate catalogues (sensitivity labels, tags), set the **storage quota**, and run **import / export**. Every user manages their own **account security** — password, multi-factor authentication (authenticator app or passkeys), and profile photo.

Your own account lives behind **Edit profile…** in the avatar menu (top right in both clients). It shows which account you are signed in as, the photo you currently have with a crop to replace it, and a button through to changing your password. Two-factor authentication, passkeys and your devices stay as their own entries in that menu, since each is a separate credential rather than part of your profile.

One thing to expect from the authenticator code: it is **single-use**. A six-digit code is accepted once and then refused for the rest of the half-minute it remains on screen, which is what "one-time password" means and what stops a code read over your shoulder — or relayed by a page pretending to be this one — being used again behind you. If you have just signed in somewhere with the code showing, wait for the next one rather than retyping it.



Accounting
Anna Meyer
Demo Admin
Lukas Brunner
Sofia Meier
Tom Fischer

Select a user or group.

Accounting — group rights

Group rights apply to the group's member users.

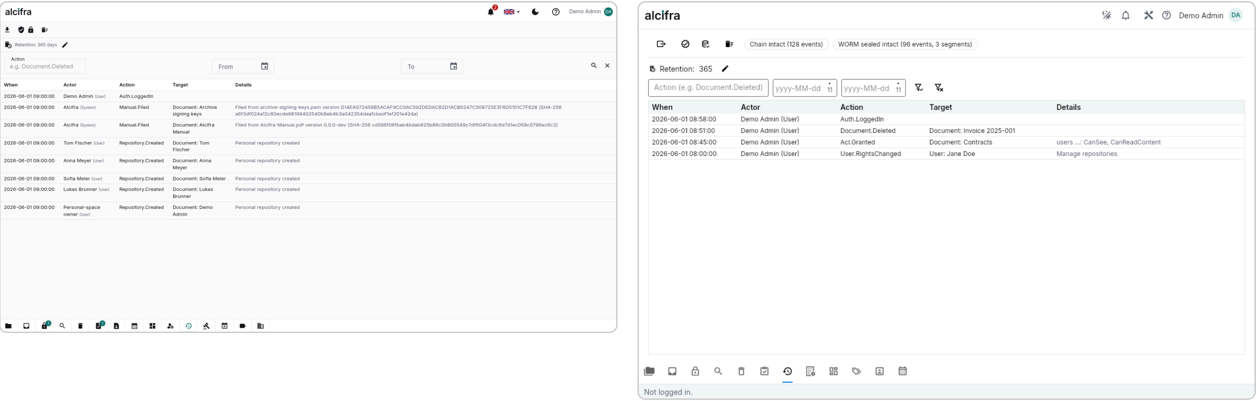
Members

Add a member…

Demo Admin
Lukas Brunner
Anna Meyer

☐ Tenant administrator
☐ Impersonate
☐ Override checkout
☐ Legal hold
☐ Manage classification
☐ Reset MFA
☐ Manage repositories
☐ Manage masks
☐ Manage service accounts
☐ Manage users & groups

Figure 35: Users & groups administration in the web (left) and desktop (right) clients.



Retention: 365 days

Action: e.g. Document.Deleted — From — To

When	Actor	Action	Target	Details
2026-06-01 09:00:00	Demo Admin (User)	Auth.LoggedIn		
2026-06-01 09:00:00	Alcifra (System)	Manual.Filed	Document: Archive signing keys	Filed from archive-signing-keys.pem version D14EA072458B5ACAF9CC0AC592DEDACB2D1ACB0247C508725E3F8D5101C7F628 (SHA-256 a6f3df024a12c60ecde98199402540b8eb4b3a042354daafcbadf1ef201e42da)
2026-06-01 09:00:00	Alcifra (System)	Manual.Filed	Document: Alcifra Manual	Filed from Alcifra-Manual.pdf version 0.0.0-dev (SHA-256 cd596f06fbab4bdab925b86c0b800549c7df604f3cdc6d7d1ec059c0796ec6c2)
2026-06-01 09:00:00	Tom Fischer (User)	Repository.Created	Document: Tom Fischer	Personal repository created
2026-06-01 09:00:00	Anna Meyer (User)	Repository.Created	Document: Anna Meyer	Personal repository created
2026-06-01 09:00:00	Sofia Meier (User)	Repository.Created	Document: Sofia Meier	Personal repository created
2026-06-01 09:00:00	Lukas Brunner (User)	Repository.Created	Document: Lukas Brunner	Personal repository created
2026-06-01 09:00:00	Personal-space owner (User)	Repository.Created	Document: Demo Admin	Personal repository created

Chain intact (128 events) — WORM sealed intact (96 events, 3 segments)

Retention: 365

Action (e.g. Document.Deleted) — yyyy-MM-dd — yyyy-MM-dd

When	Actor	Action	Target	Details
2026-06-01 08:58:00	Demo Admin (User)	Auth.LoggedIn		
2026-06-01 08:51:00	Demo Admin (User)	Document.Deleted	Document: Invoice 2025-001	
2026-06-01 08:45:00	Demo Admin (User)	Acl.Granted	Document: Contracts	users …: CanSee, CanReadContent
2026-06-01 08:00:00	Demo Admin (User)	User.RightsChanged	User: Jane Doe	Manage repositories

Not logged in.

Figure 36: The audit trail — an append-only, hash-chained log — in the web (left) and desktop (right) clients.

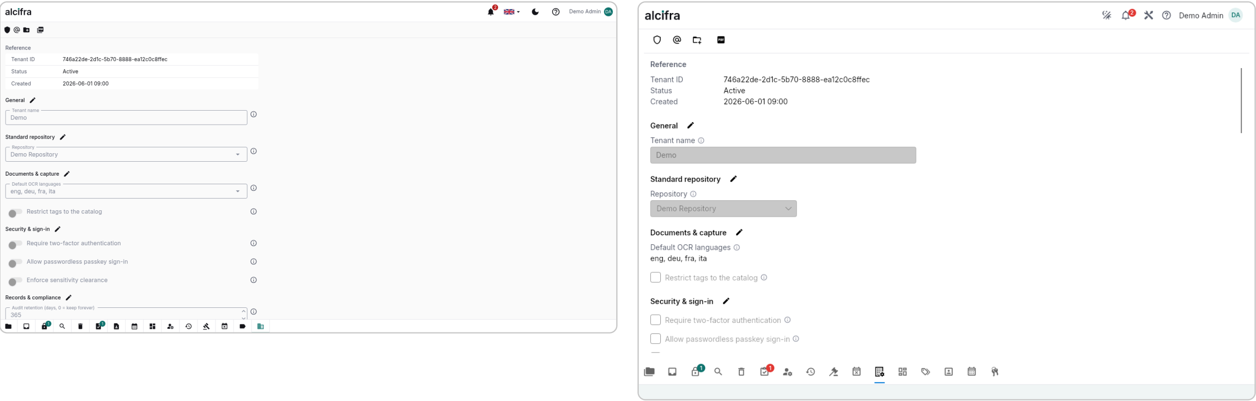
What reaches that trail is worth being precise about, because an audit trail is read by people deciding whether something happened — and a trail believed to hold everything is one nobody checks. It records **sign-in and account events** (signing in, signing out, a refused credential, an account walled up after repeated attempts), **document acts** (filing, versioning, moving, deleting, restoring, rights changes, legal holds, disposition), **administration** (users, groups, tenant settings, service accounts, certificate enrolment) and the **deliberately dangerous** ones — impersonation, purge, encryption-key rotation and retirement.

It is **not** a record of literally every write. A small, named and shrinking set of operations still mutate without recording — chiefly some document sub-items such as appointment and contact edits — and the project tracks that set explicitly rather than claiming coverage it does not have. If an entry's absence would be evidence for you, confirm that the act you care about is one of the recorded kinds rather than assuming silence means it did not happen.

A refused sign-in and a lockout are recent additions. Before them the trail carried only successful sign-ins, so a reader could see that somebody **got in** and not that somebody had been trying for an hour — which is the question an investigation actually asks.

Among the tenant settings is the account your notifications **leave through**. Most installations run no mail relay of their own and authenticate against an ordinary provider — and such an account is naturally per tenant, because each tenant's mail should come from its own domain. Fill in a server and the port, encryption, user name, password and sender address appear with it; leave the server empty and the installation's own account is used, which is what every existing deployment keeps doing.

The password is write-only. No screen ever shows it back to you: the pane says whether one is stored, and an empty box on saving means **keep the one you have** rather than **clear it**, so changing the port does not silently wipe the credential. **Send test message** is offered whether or not this tenant has its own account — with none it proves the installation's, which is what your mail would really use. The point of it is to fail in front of somebody who can fix it, rather than at three in the morning inside a reminder nobody receives.



Reference
Tenant ID 746a22de-2d1c-5b70-8888-ea12c0c8ffec
Status Active
Created 2026-06-01 09:00

General
Tenant name: Demo

Standard repository
Repository: Demo Repository

Documents & capture
Default OCR languages: eng, deu, fra, ita
Restrict tags to the catalog

Security & sign-in
Require two-factor authentication
Allow passwordless passkey sign-in
Enforce sensitivity clearance

Records & compliance
Audit retention (days, 0 = keep forever): 365

Reference
Tenant ID 746a22de-2d1c-5b70-8888-ea12c0c8ffec
Status Active
Created 2026-06-01 09:00

General
Tenant name: Demo

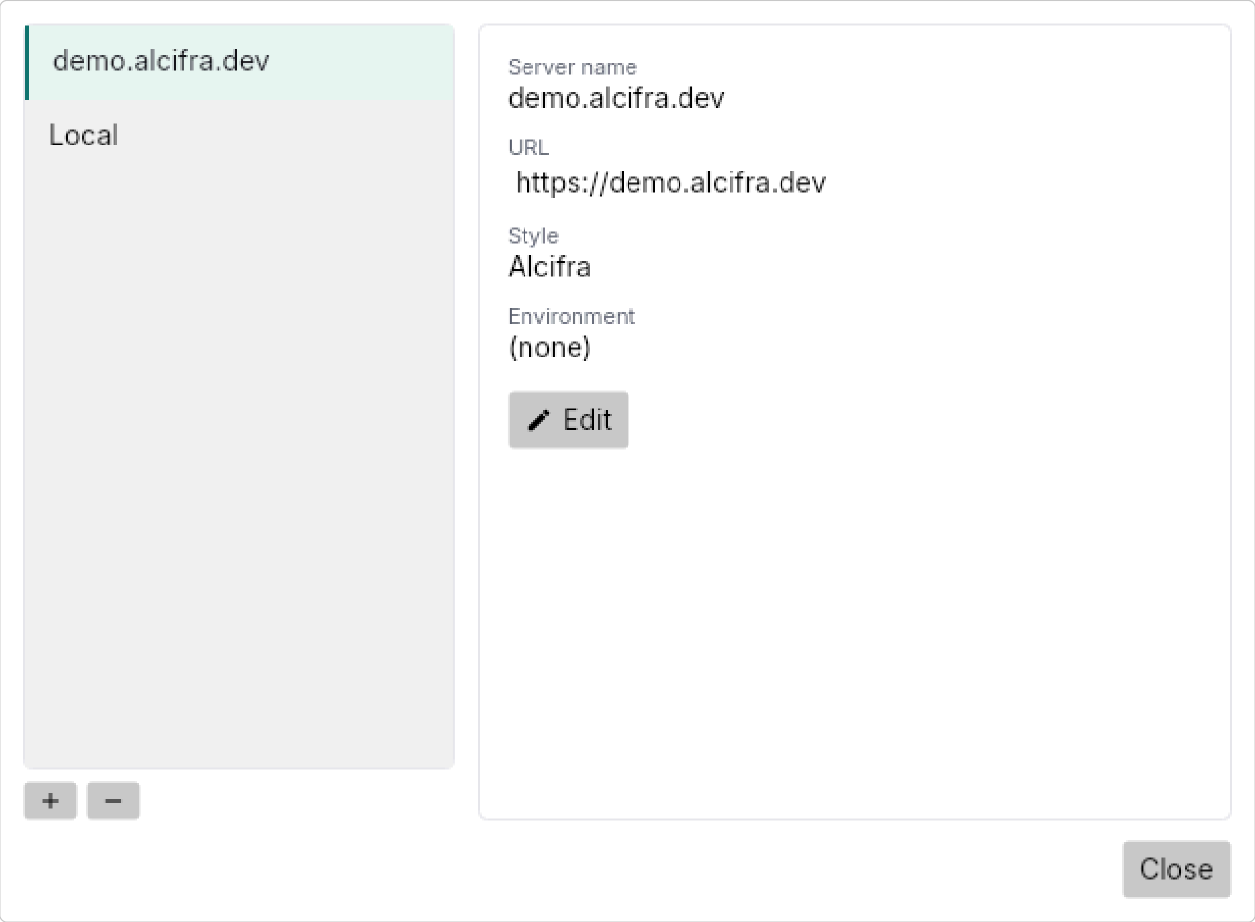
Standard repository
Repository: Demo Repository

Documents & capture
Default OCR languages
eng, deu, fra, ita
☐ Restrict tags to the catalog

Security & sign-in
☐ Require two-factor authentication
☐ Allow passwordless passkey sign-in

Figure 37: Tenant settings in the web (left) and desktop (right) clients.

The desktop client can connect to **several servers** its server manager stores a profile (name + address) for each. A server is not a tenant: one Alcifra installation hosts many tenants, and which tenant you belong to follows from the account you sign in with.



Server name
demo.alcifra.dev

URL
https://demo.alcifra.dev

Style
Alcifra

Environment
(none)

Figure 38: The desktop server manager: connection profiles for several Alcifra servers.

14.1 When the archive hands you encrypted content

On an installation configured for it, the archive does not serve readable document content at all. What arrives is an **envelope addressed to your own certificates** — the reader certificates you enrolled (see Section 15.8), the same ones your mail is sealed to — and it is opened on your own machine, with your own key, by the client you are working in.

> **This describes the strict level.** An administrator can also set a level where documents are offered **both** ways — as an envelope to whoever can open one, and in the ordinary readable way to everyone else. There the buttons below stay where they are and keep working, the web client keeps showing documents, and what changes is only that the desktop client reads through your card. If your buttons work, you are on that level rather than this one, and nothing here has gone wrong. Section 14.2 sets out the levels.

Two consequences are worth meeting in a manual rather than in the application, because neither looks like what it is.

Without a registered certificate, download and preview are not offered — on the strict level. Not refused with an error — **absent**. The document, its index data and its history all still render; the buttons that would hand you content simply are not there. That is deliberate: a button that fails when clicked teaches nothing, while a missing one, beside the certificate entry in your account menu, points at the remedy. Register a certificate and the buttons appear — and note that on an installation that runs certificate management there are **two** entries there, of which only one addresses documents to you. Section 15.8 sets out which.

The desktop client opens these envelopes; the web client cannot. A browser has no access to the certificate store your private key lives in, so on such an installation reading documents is work for the desktop client. The web client's preview says *This document is sealed. Open it in the desktop or phone app.* and stays fully usable for everything that is not content — browsing, index data, search results, workflow, administration.

Find-in-document follows the same line, and for the same reason. Highlighting your search words on the page needs the words and their positions, and on such an installation the server does not have them — it holds only the envelope. The desktop client finds within a document because it has already opened that envelope and holds the page in the clear; the web client cannot, so the find box is not offered there. Where a document carries no text of its own — a photograph, or a scan that was never recognised — find has nothing to search and says so, on either client.

> **A key held on a smartcard never leaves it.** Where your private key lives on a card or a token rather than in a file, the card performs the one operation that opens each envelope, and the key itself is never read out — not by the archive, not by the client, not by anything else running on the machine.

Reading with your card. The first document you open asks for your **card PIN**, once. It is not asked again while you stay signed in and the card stays in the reader — one document's preview, its page images and its searchable text are three separate reads, and a prompt for each would be unusable. The PIN itself is not stored: what the client keeps is the unlocked connection to the card, not the number you typed.

Two cards in readers at once is fine. Where more than one card is plugged in — your own and a colleague's, or a card you are preparing beside the one you work with — the client first asks the **document** who it was addressed to, and compares that against what each card carries. That question is answered from public information, so **a card that cannot open the document is never asked for its PIN**: a prompt whose answer was always going to be no is how people learn to click through prompts.

Where **several** cards could open it — a card and a laptop identity are one person, and a document is addressed to all of them — each is tried in turn, and the PIN prompt **names the card it is about**. So declining one, or mistyping it, does not lose a document another card would have opened. You are not offered a choice between them, deliberately: every card that can open it opens the same document, so the choice would have no outcome you could see.

Taking your card out closes what it opened. Within a few seconds of removing the card, any document on screen that it decrypted is closed and the pages it drew are discarded. You stay signed in and can carry on browsing folders, index data and search results — none of that is encrypted — and putting the card back lets you open documents again. This protects the document you left on an unattended screen; it does not sign you out, so lock your computer as you normally would when you leave it.

Only your own keys are used. The client opens an envelope only with a key that is enrolled for **the user who is signed in**. A colleague's card in the reader — or their certificate on the computer — is not used and never asked for its PIN, even where the document is addressed to it: an administrator who may see a colleague's sealed items in the archive still cannot read them with the colleague's key. This holds for documents, the Messages tab and the Passwords tab alike.

When a document stays sealed, the preview says why, in place of the document, with **Open with my key** to try again:

- *This document is sealed. Insert your key "…".* — it is sealed to one of your enrolled keys, named, which is not in the reader. Insert it; putting a card back makes the preview try again by itself. Where several of your keys would do, or you cancelled the PIN prompt, it asks for *a hardware key that can open it*.
- *Sealed for other keys. — None of your enrolled keys is among them.* The document was sealed for keys that are not (or no longer) enrolled for you.
- *This key is not enrolled for you. — Sign in as the key's owner, or use a key enrolled for you.* The key in the reader could open the document, but it belongs to somebody else.
- *Your enrolled keys could not be checked.* — the client could not find out which keys are enrolled for you, so it uses none.

A document that arrived damaged, or could not be fetched, says so plainly in the same place.

Preparing a card in the first place — putting a key on it, getting a certificate onto it, and registering that certificate here — is set out in Section 15.

14.2 Encryption of the archive itself

Alcifra's encryption is extended by **two separate paid products**, and which one you need depends on what you are trying to achieve. They are named here because the division is not where most people guess.

The Encryption Service encrypts **content where it is stored**, under keys held in hardware the operating organisation controls.

The Encryption Module manages **certificates at the scale of an organisation**: several per person — one per device, so a smartcard in a computer and an identity on a phone can both be addressed at once — enrolment **on somebody else's behalf** and in bulk, **revocation**, and a **tenant policy** stating what makes a certificate acceptable at all (minimum key sizes, permitted curves, the holder's own e-mail address on it). It is also **where a reader's certificates come from** wherever content is delivered as an envelope.

What may surprise you is what is **not** in either list. **Addressing a document to your own certificate needs neither product.** This application can do that with the certificate you registered yourself — it is the same mechanism that makes your mail readable on your devices alone (see Section 6.2). What the paid products add is protection of the stored bytes, and the means to run certificates for hundreds of people rather than one.

Where the Service is installed, an administrator sets a **level per tenant**, so an archive that needs this can sit beside one that does not. Three of those levels concern you as a reader, and they differ in what happens when you have no certificate yet. The first needs only the Service; the other two deliver envelopes, so they also need the Module, which is where your certificates are enrolled:

Stored encrypted Content is encrypted where it is kept. Nothing changes in how you read it.

Rehearsal Every document is **also** offered as an envelope to whoever can open one, while every ordinary way of reading stays open. It exists so an organisation can find out whether its machines and cards are ready **before** the strict level is switched on — nobody is blocked while the gaps are found. It is a readiness exercise and **not** additional protection: everything readable before is still readable, by anyone who could read it before.

Strict Content is served **only** as an envelope. This is the level Section 14.1 describes, and the one where a missing certificate means no content at all.

Chats are never sealed. At every level, Strict included, a document's chat is neither encrypted at rest nor encrypted for its reader: the levels protect a document's **content**, its files, and not the conversation beside it. Like everything the clients exchange with the server, a chat still travels encrypted over the network (TLS); what it lacks is the per-reader encryption. Whatever must stay protected belongs in the document, not in its chat.

Everything else these products do, including what their protection does and does not claim, is described in **their own manuals**: they are separate products with their own releases, and documenting them twice is how the two come to disagree.

14.2.1 What encryption here does not claim

One question is worth answering wherever encryption is described, because the honest answer is not the one the word "encrypted" suggests: **this is quantum-migratable, not quantum-resistant**.

The **content** holds up. Documents and messages are encrypted with AES-256, and a quantum computer does not break it — the best known attack halves the effective key strength, which leaves 256-bit AES at a level still considered safe. The bulk of what is stored is therefore fine.

What does not hold up is every **asymmetric** step beside it: wrapping a content key under a hardware-held key, and addressing an envelope to somebody's certificate. Those rest on RSA and elliptic-curve mathematics, which a sufficiently capable quantum computer breaks outright.

The consequence has a name — **harvest now, decrypt later**. An adversary who copies today's wrapped keys and envelopes can decrypt them on the day such a machine exists. For something meant to last a week that is academic. For an **archive**, whose whole purpose is to keep documents readable for decades, the time the content must stay protected plausibly overlaps the time that machine might appear, so it is worth stating plainly rather than leaving in the small print.

Migratable is the claim that is true. The content keys never have to change: moving to post-quantum protection means re-wrapping those keys under a new scheme, not re-encrypting a single document — and in the Encryption Service the rolling key generations that already exist are exactly the machinery that would do it. What is missing today is not the design but the parts: hardware security modules, card vendors and mail clients have not shipped post-quantum support yet.

The two paid products' own manuals carry this in more detail, each for its own half. It is repeated here because a reader of **this** manual may never open those, and a boundary nobody states is one a reader assumes does not exist.

15 Appendix — preparing a smartcard or token

This is the one-time technical setup behind Section 14.1: putting a key on a card, getting a certificate onto it, and registering that certificate with the archive. It is work for whoever administers the cards — an ordinary user meets only the last step.

The procedure below is for a **PIV** card or token, which is what the archive's clients speak. A specific device is used as the worked example because the key-generation commands belong to the card's own tool and differ per vendor; everything from the certificate request onwards is the same for any PIV device.

> **About the device in the example.** This manual uses a YubiKey because they are widely available and affordable, which makes the example easy to follow along with. It is not a recommendation, and the archive requires nothing of that vendor: any PIV-capable card or token works, and only the commands in steps 1, 2 and 3 — the ones that speak to the card itself — would change. Substitute your own device's tool there.

15.1 What has to be true, and why

> Four choices are not free. Getting one wrong produces a card that looks right and cannot read a document.
>
> - **Slot 9D (Key Management).** That is the decryption slot. 9A is authentication and 9C is signing; a certificate in either cannot open an envelope, and using the wrong one is the classic first mistake.
> - **An elliptic curve (P-256) is preferred; RSA 2048 also works.** Both are addressed: an EC key by key agreement, an RSA key by key transport. EC is the better choice on a modern token and is what the provisioning command below picks unless told otherwise. This manual said **RSA, not an elliptic curve** until the EC path was completed and proven on two cards — if your card already carries an RSA key there is no reason to replace it.
> - **PIN policy on, touch policy OFF.** A touch requirement means a physical tap **per decryption** — and one document's preview, page images and searchable text are three separate reads. Acceptable for signing, unusable for an archive.
> - **The key is generated ON the card.** That is the whole point: it cannot be copied off, so there is no file of it to lose.

15.2 Tools

Two are needed, and a third is not.

- **The card's own tool**, for generating the key and writing the certificate back. Card personalisation and key generation are card-applet operations, not PKCS#11 ones. In this example that is `ykman`.
- **The PKCS#11 module**, so the desktop client can reach the card at all. Install **OpenSC**; the client looks for `opensc-pkcs11` in the usual places and offers a file picker if it is somewhere else.
- **`caconsole`**, the certificate tool, if the certificate is issued by your own certificate authority. It installs from the public package feed and needs no credentials:

```
dotnet tool install --global HebelConsulting.CAManagement.Cli

# ONCE PER MACHINE: a global tool lands in ~/.dotnet/tools, which is not on PATH by default.
# Without this the next line fails with "command not found: caconsole".
export PATH="$PATH:$HOME/.dotnet/tools"   # add to ~/.zshrc to keep it

caconsole list-slots --module <the PKCS#11 module>
```

> **"command not found: caconsole" does not mean the install failed.** It almost always means `~/.dotnet/tools` is not on your PATH — the installer prints that warning once and it is easy to miss. Check with `dotnet tool list --global`: if the table lists the package and the command `caconsole`, the tool is there and only the PATH is missing. `~/.dotnet/tools/caconsole` will run it in the meantime.

Where the module is. `list-slots` needs the path to OpenSC's PKCS#11 library, and it differs per system. These are the places the desktop client itself looks, so one of them is almost certainly right:

macOS, Homebrew on Apple silicon
`/opt/homebrew/lib/opensc-pkcs11.so`

macOS, Homebrew on Intel
`/usr/local/lib/opensc-pkcs11.so`

macOS, the OpenSC installer package
`/Library/OpenSC/lib/opensc-pkcs11.so`

Windows
`C:\Program Files\OpenSC Project\OpenSC\pkcs11\opensc-pkcs11.dll`

Linux, Debian and Ubuntu
`/usr/lib/x86_64-linux-gnu/opensc-pkcs11.so`

Linux, Fedora and RHEL
`/usr/lib64/opensc-pkcs11.so`

A working run names the token rather than staying silent — with a card inserted it prints the Cryptoki version and a table with one row per slot, and the token column shows `PIV_II` for a personalised card. If the table is empty the module loaded but the reader sees no card; if the command fails on the module path, the path is wrong rather than the card.

> **Two tools, two jobs — and both are now involved.** `caconsole` belongs to the certificate authority: it prepares the card and issues the certificate. `alcifra` belongs to the archive: it **enrols** a finished certificate against a person, lists what is enrolled and revokes one (`alcifra certificates enrol|list|revoke`).

> This manual said `alcifra` had no certificate commands, which was true until they were added. The division still holds and is worth keeping straight: the authority issues, the archive enrols.

15.3 One command, where your own authority issues the certificate

If the certificate comes from a certificate authority you run with `caconsole`, the whole of the next section is one command:

```
caconsole yubikey provision --holder anna@example.test \
    --token-label ca-token --pin "$CA_PIN" --ca-label ca-root --ca-cert ca.crt \
    --label "YubiKey 5C" --manifest enrolments.json
```

It generates the key in slot 9D, has the **card itself** sign the certificate request — which is the only available proof that the private key is really there and reachable — issues a certificate for decryption, imports it, and reads the slot back to confirm what the card now holds.

Three things it will not do quietly:

- **It will not overwrite a card without being told.** If the slot already holds something it prints what is there and stops. `--force` replaces the **certificate** and keeps the key — which is the repair for a card whose certificate declares the wrong usage — and `--regenerate-key` replaces the **key**, which is irreversible: anything encrypted to the old one becomes unreadable. Any certificate already in the slot is saved beside the new one first.
- **It will not trust a card that cannot answer.** A token whose PIV firmware is older than 5.3 reports `Private key type: EMPTY` whether or not a key is present — it simply does not publish that detail. The command treats "empty" on such a card as **unknown** rather than **absent** and refuses to generate over it.
- **It will not hide what it did.** Every command it runs against the card, and the card's answer, is printed — with the PIN and management key removed.

With `--manifest` it also records what was issued and to whom, in a file `alcifra certificates import` reads — so a box of prepared cards is enrolled in one go rather than one at a time:

```
alcifra certificates import enrolments.json --dry-run   # what would change
alcifra certificates import enrolments.json
```

It is safe to re-run: a certificate already enrolled is reported as **unchanged** rather than as an error, so an interrupted import is finished by running it again. (`alcifra certificates enrol <file> --holder <email>` is the single-certificate sibling, for one card prepared on its own.)

15.4 The walk-through

This is what the command above does, and the route to take when the certificate comes from somewhere else — a corporate authority, or a card that arrives already personalised.

1 — Personalise the card, once per card. These two objects must exist before anything else will behave.

```
ykman piv objects generate chuid
ykman piv objects generate ccc
```

2 — Generate the key on the card, into the Key Management slot.

```
ykman piv keys generate 9d public.pem \
    --algorithm rsa2048 --pin-policy once --touch-policy never
```

3 — Put a certificate in the slot. The slot needs one before anything can use the key at all: a PKCS#11 module does not surface a PIV slot that carries only a key. There are two routes, and **for this archive the simple one is enough** — so it is given first rather than as an afterthought.

3a — Self-signed, with no authority at all. The card's own tool both makes the certificate and writes it to the slot, in one command.

```
ykman piv certificates generate 9d public.pem \
    --subject "CN=Card Holder,O=Example,C=CH" --valid-days 730
```

> **The archive accepts a self-signed certificate, and that is deliberate rather than an oversight.** Nothing on the reading path validates a chain or an issuer: content is enveloped to a **public key**, and the only thing that can open the envelope is the private key on the card. A certificate authority answers a different question — **who vouches that this key belongs to that person** — which matters for an organisation issuing identities and not at all for whether a document opens.
>
> What the archive does check is that the file parses as X.509 and carries an **RSA** or an **elliptic-curve** key — the two kinds **What has to be true, and why** above lists, the one addressed by key transport, the other by key agreement.
>
> **Mind the lifetime.** `--valid-days` defaults to **365**, and while the archive itself does not refuse an expired certificate on the reading path, the encryption service **does** refuse to register one — so a certificate that has already lapsed cannot be re-registered after a reset or a move to another installation. Choose a lifetime you are willing to live with.

3b — Or issued by your own authority, if your organisation wants identities it controls — a common trust root, a name it owns, a register of who was issued what. Ask the card for a request, issue it, and write the result back.

```
ykman piv certificates request 9d public.pem holder.csr \
    --subject "CN=Card Holder,O=Example,C=CH"

caconsole issue --token-label ca --pin <ca-pin> \
    --ca-label root --ca-cert ca.crt --csr holder.csr --days 365 --out holder.crt

ykman piv certificates import 9d holder.crt
```

> **Use the card's tool for the request, not the certificate tool.** `caconsole` can produce a request from a key that lives on a token — but a PKCS#11 module only surfaces a PIV slot once that slot carries a **certificate**, and this one does not yet. Asking the certificate tool first fails with

> `No private key with label '…' found on the token`, which reads like a wrong label and is not one.

> **An empty `ERROR:` with no message means the slot has no key.** Both commands in 3a and 3b sign with the private key in slot 9D, so running either before step 2 fails — and the card's tool reports it as a bare `ERROR:` with nothing after the colon, which says nothing at all about the cause. Check with `ykman piv info`: if the output has no **Slot 9D** section, there is no key there yet.
>
> This is also what a **reset** leaves behind. `ykman piv reset` — the way out of a blocked PIN when the PUK is blocked too — erases every key and certificate on the card, so it puts you back at step 1 rather than where you left off. Prefer `ykman piv access unblock-pin`, which takes the PUK and keeps the key.

4 — Remove the card and put it back in. The operating system creates its view of a token when the card is **inserted**, so a card personalised in place is not yet visible to anything. Skipping this makes a correctly prepared card look broken.

15.5 Testing the key from the desktop client

Test the hardware key is the tools icon (a screwdriver and a wrench) on the logon window — so it works before you sign in — and the first entry of the tools menu beside the help icon in the main window's title bar. Insert the key and press **Run the test**.

It reads the certificate in the key's key-management slot (no PIN), seals a fresh random message to it on your computer, asks for the PIN, opens the message with the key and compares. It needs no server, sends nothing anywhere, and works for any RSA or elliptic-curve key the archive accepts, yours or one your organisation issued. Each step is listed as it runs; the first one that fails says why, with the exact code the smartcard software answered (`CKR_PIN_INCORRECT` for a wrong PIN — mind the retry counter).

The report for support. Below the steps is a report you can copy and send to whoever supports you. The window says what it contains before you copy it: the app and system versions, the PKCS#11 library found, your readers, the key's manufacturer, model, firmware and **serial number**, its certificates, and every step with its outcome. Signed in, a switch (on unless you turn it off) adds the archive side: the server's address and version, whether the tested certificate is enrolled for you, the PIN policy and your organisation's encryption mode. The report never contains your PIN, documents, sign-in tokens or other secrets.

15.6 Checking the card before you rely on it

Beside the window above, the desktop client carries two hooks that answer **"is this card actually going to work?"** without a server, an account, or a document. They are worth running before registering anything, and they are the first thing to reach for when a card misbehaves later.

Both are the **installed desktop client**, run from a terminal with a flag — there is nothing extra to install and no source tree to build.

```
# The installed executable. This is the macOS path; the others are below.
CLIENT=/Applications/Alcifra.app/Contents/MacOS/Alcifra.DesktopClient
```

macOS
`/Applications/Alcifra.app/Contents/MacOS/Alcifra.DesktopClient`
Windows, in the unpacked folder
`Alcifra.DesktopClient.exe`
Linux, in the unpacked folder
`./Alcifra.DesktopClient`

What the client can see — no PIN, because certificates are public:

```
$CLIENT --card-test
```

It prints the PKCS#11 module it found, then every certificate on every token in every reader, with the subject, the expiry and a **concern** — `KeyUsage` when the certificate does not mention key encipherment, `Expired`, or `None`. A card that shows nothing here will show nothing in the dialog either, and the cause is the module or the reader rather than the archive.

Whether it can actually decrypt — this one needs the PIN:

```
CARD_PIN=<pin> $CLIENT --card-open-test
```

It builds an envelope **the way the server builds one**, addressed to the certificate on the card, and opens it through the same code a real document goes through. `MATCHES: True` on the last line means the whole chain works: the certificate, the key behind it, the PIN, the module, and the client's own reading path.

> **This is the test that settles arguments, and it is the only one that can.** A certificate looks identical whether or not a usable private key stands behind it — and on YubiKey firmware older than 5.3 `ykman piv info` reports `Private key type: EMPTY` for a slot holding a perfectly good key, because key metadata did not exist before that version. Reading that as "there is no key" and regenerating would destroy the real one. A decryption is the only thing that proves the key is there.
>
> It also tells a **wrong PIN** apart from a **broken card**: the failure names `CKR_PIN_INCORRECT` rather than reporting that nothing could open the envelope. Mind the retry counter while trying — three wrong entries block the PIN.

15.7 Registering it with the archive

In the desktop client, open **Enrol a certificate…** from the account menu and choose **Add a card or token…**. It lists every certificate on every device in a reader — not just the first — because a card can carry several and a computer can have more than one reader. Pick the one from the Key Management slot and enrol it. No PIN is asked: a certificate is public, and only reading a **document** needs the key.

From then on the archive addresses that person's content to that certificate, and Section 14.1 describes what they see.

A warning about key usage is expected and is not a problem. Many certificate authorities mark a key-management certificate for signing only. The card decrypts regardless — neither the archive nor the card enforces that field — so the dialog says so and lets you continue rather than refusing a certificate that works.

15.8 Enrolling it yourself, where that is open to you

Enrol a certificate… in the account menu is where every reader certificate is enrolled, in both clients and on every installation. Without the **Encryption Module** (Section 14.2) a person holds at most **two** certificates — a card for the desk and a security key for the road, say; with it, its certificate policy decides how many, and what counts as an acceptable certificate. A certificate that is revoked no longer counts towards the two.

Whether you may use it is your organisation's decision, and the dialog tells you rather than the menu hiding it. Enrolling your own certificate is switched **off** until an administrator opens it — so where it is closed, the dialog says that it is closed and that somebody who administers certificates can enrol one for you, or open it. That is deliberately not the same message as "this installation does not do this at all": one of the two is something a colleague can change this afternoon, and a missing entry could not tell them apart.

Enrolment happens in the desktop client, not in the browser. A reader certificate is accepted only when its key is **attested** — the token itself proves the key was generated on it and cannot leave it (Section 15.10) — and only a program that talks to the token can read that proof. So the web client's dialog shows the certificates enrolled for you, each with its attestation state, and points you to the desktop client (or the phone, or `alcifra`); the desktop client's **Add a card or token…** reads the certificate **and** its attestation from the token in one step.

What it asks for is the certificate — from the token, or the public half as a file (`.pem`, `.crt`, `.cer` or the binary `.der` that card tools export). A file carries no attestation, so it is accepted only where your organisation allows hardware without attestation. **A file containing a private key is refused before anything is sent**, with the reason: the private key belongs on the card or in the keystore that generated it, and the moment worth saying so is while it is still only on your own machine.

You also give the key a name you will recognise — "YubiKey 5C", "work laptop". That is what stops several certificates per person being confusable later, and it is worth a moment's thought because a document is addressed to **all** of them: the card on your desk and the identity on your laptop open the same document, with nothing having to guess which one you are at.

A refusal names which of four things happened, because the remedy differs every time and only one of them is yours to act on (and a fifth, **you already hold two**, where the limit applies: revoke one first): enrolment is not open to you here (an administrator's), the certificate is not one the product can use at all (a re-issue), your organisation's policy does not accept this one because its key is too short, its curve is not on the permitted list or the e-mail address on it is not yours (ask for one that is — Section 15.11), or it is enrolled already (nothing to do). The sentence comes from the certificate management itself, in your own language, rather than a single “could not enrol” that would send all four to the same place.

> **One store.** Earlier versions had a second entry, **Encrypted mail (S/MIME)…**, holding one certificate per person beside this one. It is retired: the certificates enrolled here are the only ones the archive addresses documents and mail to, so revoking one revokes it everywhere.

15.9 Revoking a key

My keys… in the account menu lists the keys enrolled for you, each with its state, and offers **Revoke…** on every one still valid. An administrator who manages users finds the same list under **Keys…** on a person in **Users & groups**; nobody else can revoke your key. Revoking asks **why** — lost, stolen or compromised, replaced by a new key, the holder left the organisation, or in your own words — and that reason stays on the key. When somebody else revoked your key, a notification tells you which, by whom and why.

Replacing a key? Enrol the new one first, then revoke the old one. From the moment a key is revoked nothing is sealed to it any more, and what it could read is marked exposed; if it was your only valid key, the dialog warns you, because until another is enrolled you cannot open sealed items. A revocation cannot be undone. From the command line it is `alcifra certificates revoke --thumbprint … --reason lost`.

15.10 Only attested hardware keys are enrolled

A reader certificate is accepted only when its key is attested: the token signs a statement that it generated this key itself, with an attestation key its manufacturer put on it, and the archive checks that statement before it files anything. A key that was ever outside a token — generated in software, or imported onto the card — cannot be attested, so it cannot be enrolled by mistake as if it were hardware.

Which manufacturers count is your installation’s decision. The platform administrator switches on the manufacturers’ attestation roots the installation trusts (`alcifra attestation-roots`); a fresh installation trusts none. A tenant administrator may switch some of them **off** for the tenant, never add one (`alcifra certificates attestation-roots`). Where a token cannot attest at all — many cards cannot — an administrator may allow **hardware without attestation** for the tenant (`alcifra certificates settings set --unattested-hardware allowed`); such certificates are then marked **not attested**.

Every certificate shows its state in the enrolment dialog of both clients and in `alcifra certificates list`: **attested by** the root’s name and the token’s serial number; **not attested**; or attested to a root that is not trusted here. Only the first addresses you where your organisation requires attestation, and a switch takes effect at once: nothing new is sealed to a key whose root was switched off. Documents already sealed to it still open with it — nothing is ever re-sealed.

From `alcifra`, the attestation travels as two files the token’s own tool exports — the slot’s attestation certificate and the token’s attestation certificate that signed it (for slot 9D: `attest 9d` and `export f9`):

```
alcifra certificates enrol holder.crt --holder anna@example.test \
    --attestation attestation-9d.pem --attestation-intermediate f9.pem --slot 9d
```

A manifest for `alcifra certificates import` carries the same two certificates per row, as an `attestation` object.

15.11 A certificate carries its holder's address

A reader certificate is enrolled only for the person whose e-mail address it carries. Documents addressed to you are sealed to the certificates enrolled **for you**, so a certificate issued to a colleague's address, enrolled under your name, would make your documents readable with your colleague's key — whether you enrolled it yourself or an administrator named you as its holder (`alcifra certificates enrol … --holder`). So the address on the certificate (its subject-alternative e-mail, compared without regard to case) must be the account's own, and a mismatch is refused with both addresses named:

```
This tenant requires a certificate to carry the e-mail address of the account it is enrolled
for.
This one says 'florian@example.test' and the account is 'crypt@example.test'.
[CERTIFICATE_POLICY_REFUSED]
```

The remedy is a certificate issued to the right address, or to one of the account's identity aliases. **This binding, strict, is the default for every organisation that has not said otherwise.** An authority whose certificates carry no e-mail address at all cannot meet it, and for that case an administrator can bind keys by **possession** instead. Then a certificate needs no particular address. A member enrols their own by proving they hold its key: the server seals a challenge to the certificate, and the client opens it with the key, asking for the PIN as for any sealed document. An administrator can also enrol a certificate for its holder, vouching for whose key it is. Under either binding, a key belongs to one person in the tenant, even under a second certificate.

```
alcifra certificates policy show
alcifra certificates policy set --binding possession   # an authority without e-mail
addresses
alcifra certificates policy set --binding strict       # back to the default
```

To accept only certificates your own authorities issued, name them as trust anchors: one certificate, or several in one PEM file, any of which suffices.

```
alcifra certificates policy set --trust-anchors our-authorities.pem
alcifra certificates policy set --trust-anchors none   # any authority again
```

The rule is applied when a certificate is enrolled, never backwards. Switching it on does not refuse, flag or re-check anything already enrolled; a certificate enrolled while it was off goes on addressing its holder until it is revoked. An organisation that saved its own certificate policy before this became the default keeps what it saved, which said **off**: `alcifra certificates policy show` says which applies, and `set … on` adopts the default.

15.12 Your key stopped opening new documents: enrol it again

When your organisation started requiring attested keys, a certificate you had enrolled **before** — without the key's attestation — stopped being addressed: documents you could open before still open, but nothing **new** is sealed to that key. The enrolment dialog shows it as **not attested — enrol it again from the token**. You do not need a new key:

1. Insert the token and open the **desktop client**.
2. Open the account menu, **Enrol a certificate…**.
3. Choose **Add a card or token…**, pick the certificate you enrolled before, and confirm. The dialog says that the key's attestation was read from the token.
4. **Enrol**. The list below the form now shows the certificate as **attested by** your token's manufacturer, with the token's serial number, and new documents are sealed to it again.

This needs no administrator where your organisation lets you enrol your own certificates; where it does not, an administrator does the same with `alcifra certificates enrol … --attestation …` for you.

If the dialog says the token offers no attestation for the key, the key was most likely **imported** onto the token rather than generated on it — and a token attests only keys it generated itself. That key cannot be attested, ever. An administrator then has a new key generated **on** the token and a certificate issued for it (the walk-through above, from step 2), and you enrol that one. Ask before anything is deleted: the old key still opens every document that was sealed to it.

15.13 When the certificate expires, and replacing a key

The key does not expire; the certificate does. Validity dates are a property of the certificate, and the private key on the card outlives them — it keeps decrypting whatever was ever addressed to it.

Expiry is checked when a certificate is REGISTERED, and never again. Both doors refuse one that has already lapsed: the archive's own registration answers `it expired <date>`, and so does enrolment in the Encryption Module. After that, nothing re-checks it. Content keeps being addressed to it, the card keeps opening it, and no warning appears — which is deliberate rather than an omission, because the alternative is worse: a reader whose renewal is a week late would stop receiving anything, and the key that opens their existing documents would still be in their pocket.

> **So an expired certificate is a LATENT problem, not an immediate one.** Everything works until the day it has to be registered again — a new installation, a card re-issued after a reset, the same person added to a second archive — and only then is it refused. The fix at that point is a new certificate, and if a new **key** comes with it, every document already addressed to the old one stops opening. Renew before that day, not after it.

15.13.1 Replacing a key without losing the past

Content is addressed to a **certificate**, so the archive cannot re-address history for you — doing that would need the plaintext it deliberately never keeps. If you overwrite slot 9D with a fresh key, everything enveloped to the old one becomes unreadable, and there is no undo.

A PIV slot holds exactly one key and one certificate, so 9D cannot hold both the old and the new. What a YubiKey adds is **twenty retired key-management slots** — `82` through `95` — meant for precisely this: the old key moves aside instead of being destroyed.

> **The archive needs no configuration for this, and there is nothing to register a second time.** The desktop client enumerates **every** certificate on the token rather than looking in a particular slot, and an envelope names the certificate it was sealed to — so an old key sitting in a retired slot is found and used automatically when an old document is opened. New documents go to the new certificate, old ones keep opening, and the reader sees no difference.

Two practical cautions. `ykman piv keys move` exists, but moving a key between slots depends on the card's firmware — try it on a spare card before relying on it, rather than discovering the limit on the card that holds the only copy of a key. And a card has twenty retired slots, not an unlimited number: they are room for a working lifetime of rotations, not a place to keep everything forever.

15.14 A blocked PIN, a blocked PUK, and resetting the token

A hardware key's PIV application protects its keys with two codes: the **PIN**, which you type to open a sealed document, and the **PUK**, which exists only to unblock the PIN. Both allow three wrong attempts by default.

1. **The PIN is blocked** after the last wrong attempt. Nothing is lost: the PUK sets a new one (`ykman piv access unblock-pin` in this example's management tool, PUK first, then the new PIN).
2. **The PUK is blocked too.** Now only a **reset of the PIV application** is left (`ykman piv reset`). The token allows it only while PIN and PUK are both blocked, and it **erases every PIV key and certificate on it** and restores the factory PIN, PUK and management key. The token is not damaged, and its manufacturer's attestation stays, so a key generated on it afterwards can be attested again (see Section 15.10).
3. **The token's other applications are separate.** Passkeys, one-time passwords and others have their own PINs and their own resets; a PIV reset does not touch them, and theirs do not touch PIV.

> **What a reset costs in this archive: the key in slot 9D is gone for good.** A key generated afterwards is a **different** key, so everything sealed only to the old one — documents, password entries, conversation keys — can no longer be opened by anybody. The archive cannot help, because it never holds the plaintext. The one protection is a **second enrolled key**: everything is sealed to **all** of a person's enrolled keys, so a spare key keeps opening what the reset one did. Enrol it **before** you need it.

After a reset, generate a new key on the token, have its certificate issued and enrol it with its attestation (Section 15.8), then remove the old certificate. Revoking it marks every password entry that key could read as *Exposed since…* — more caution than a reset needs, since the key was destroyed rather than stolen, but harmless: the store asks for a re-seal, and changing those passwords clears the mark. Keep the PUK where you would keep a spare house key; it is the only way back from a blocked PIN that does not cost the key.

15.15 What a card covers, and what it may not

The certificates you enrol are used for **two** different things: the archive envelopes document content to them, and it envelopes the mail a device fetches where that device's **Encrypt mail content** is on (see Section 6.2). Those are read by different software, and a key on a card changes what can read them.

- **The desktop client opens card-held content.** That is what Section 15 prepares, and it is the case this manual describes end to end.
- **Your mail program is its own question.** It can only open mail enveloped to a card if it can use a hardware token for S/MIME itself. Some desktop mail programs can, through the facilities their operating system provides for smartcards; whether yours does is a question for its documentation, not for this manual.
- **On an iPhone or iPad it cannot.** Mail there reads S/MIME using an identity installed into the device, and there is no route for a mail app to use a card — over NFC or otherwise. A card-held certificate therefore leaves mail on those devices unreadable, even while documents open normally on the desktop.

> **So choose by what you need to read, device by device.** Whether mail is sealed is each device's setting (Section 8.2): leave **Encrypt mail content** off on a phone whose mail app cannot use the card, and on for a desktop mail program that can. Documents are sealed to the card either way. Where your organisation seals all mail, every device's mail is sealed, and only programs that can use the card read it.

15.16 When two certificates per person are not enough

Without the Encryption Module, a person enrols up to **two** certificates, each addressed alongside the other, so the same document opens with either key — a card on the desk and a security key carried along, say. Both are hardware keys under the same attestation rules (Section 15.10).

More certificates per person, and **enrolment** for more people than one administrator wants to prepare by hand, belong to the separate **Encryption Module** described in Section 14.2. If you need either, contact support and ask about it; it is a paid extra. Nothing in this appendix is wasted if you go that way — the card is prepared the same, and Section 15.8 stays the step that enrols it.

15.17 When something looks wrong

- **The dialog says no smartcard software was found.** OpenSC is not installed, or is somewhere unusual — use **Choose module…** and point at it.
- **It says no card or token is in a reader,** with the card plainly in the reader. Remove it and re-insert it (see step 4), and check nothing else on the machine is talking to the card at the same time.
- **The card's own tool reports the private key as `EMPTY`.** On firmware older than 5.3 the card cannot be asked what a slot holds, so the tool prints that whether or not a key is there. It is not evidence. Settle it by using the key rather than by reading a summary.

16 Appendix — glossary & links

> **Repository** — a root folder (a document with no parent). **Mask** — a document type / set of index fields. **ACL** — per-document access-control list. **Version** — an immutable snapshot of a document's file. **Legal hold** — a freeze that blocks change/deletion. **Retention** — a policy that disposes of documents after a period. **vCard** / **iCalendar** — the standard file formats a contact and a calendar entry are stored in (`.vcf` / `.ics`). **CardDAV** / **CalDAV** — the standards a phone or mail program uses to sync addressbooks and calendars. **UID** — the identifier those programs use to recognise an item across devices; changing it makes a duplicate rather than an edit. **SLA** — the review deadline a document type sets, which is what gives a task a due date.

Further reading:

- The live API description — the OpenAPI document at `/openapi/v1.json`.
- Desktop-client downloads — the download area at `/download`.

This manual's screenshots are regenerated from the running application on every release, so they never fall out of date.

17 Appendix: the desktop client's log

When the desktop client misbehaves — a preview that stays empty, a server that will not connect, a file that does not open — its log usually says why. The client writes one rolling log file per day, always at full detail, so there is nothing to switch on after the fact: whatever just went wrong is already recorded.

Where to find it. The easiest way is in the app itself: **Help ▸ Show log folder** opens the folder directly. The location by operating system:

- Windows — `%APPDATA%\Alcifra\logs`
- macOS — `~/Library/Application Support/Alcifra/logs`
- Linux — `~/.config/Alcifra/logs`

Files are named `simplarchive-YYYYMMDD.log`; the newest one is today's. The log is small by design (it rolls at 4 MB and keeps seven files) and never contains a password or a token, so it is safe to attach to a support request as it is.

Watching it live. Start the client from a terminal with the `--verbose` flag and the same full detail is printed to the terminal as it happens — useful when reproducing a problem step by step:

- Windows — `Alcifra.DesktopClient.exe --verbose`
- macOS — `Alcifra.app/Contents/MacOS/Alcifra.DesktopClient --verbose`
- Linux — `./Alcifra.DesktopClient --verbose`

Without the flag a terminal shows only the important lines; the file always carries everything, flag or no flag.

18 Appendix — who does what

A short inventory of what each kind of participant can do, for readers who need the shape of the system before its detail: an evaluator sizing it up, an administrator planning a rollout, or anyone deciding which chapter of this manual they actually need. Everything here is described properly in the chapters above; this is the map, not the territory.

Everybody's rights are per document. None of the lists below overrides that. What a person may do to a particular document comes from that document's permissions (and the groups they belong to), inherited down the tree unless a folder breaks the inheritance. An action missing from the interface is usually not a missing feature — it is the server declining to offer it here, to you, now.

18.1 The everyday user

Find and read	Browse the tree, open a folder, preview a document without downloading it, search by content or index data, jump to a search hit inside the page, and follow references to wherever else a document is filed.
Add and change	Upload by drag or from the ribbon, add a new **version** to an existing document, stage work in the **Intray** until it is ready to file, check a document **out** and back **in**, and edit index data.
Organise	Create folders, move documents, place a **reference** so one document appears in a second folder, tag, and classify with a **document type** (mask).
Collaborate	Comment in a document's chat and mention colleagues, annotate a page, subscribe to a document to hear about changes, set a reminder, and act on the tasks a workflow sends you.
Share	Create an **external link** for someone without an account — time-limited and access-limited — and revoke it.
Work elsewhere	Mount the archive in the operating system's file manager, sync contacts and calendars to a phone, read the archive's mail folders in a mail client, and open a document in its native desktop application from the desktop client.
Their own account	Language and appearance, profile photo, password, two-factor authentication and passkeys, and the separate app passwords their devices use.

18.2 The tenant administrator

Everything above, plus the settings that govern their own organisation. A tenant administrator can reach every document in their tenant: the bypass is deliberate, and what they **do** with it is audited like anything else.

People	Create and deactivate users, build groups, grant the system rights that let somebody administer users, view the audit log, import, or manage records — and issue **service accounts** for integrations.
Document types	Define masks and their index fields, which folders they may live in, what they may contain, their retention period and their default sensitivity label.
Records	Place and lift **legal holds**, review documents that have reached the end of their retention before they are disposed of, and purge when that is the decision.
Oversight	Read, filter, export and verify the **audit log**, and stream it to a SIEM over a signed webhook.
Policy	Require multi-factor authentication for everybody, cap storage, allow or forbid external links and bound their lifetime, restrict tags to a catalogue, and set the retention and check-out defaults.

18.3 The platform administrator

Runs the **installation** rather than any one organisation in it. They create and suspend tenants and provision the first administrator of each. They deliberately belong to no tenant, so they do not see documents; the separation is the point, and it is why a platform administrator cannot quietly read a customer's archive.

18.4 The service account

A machine identity for an integration — a scanner, a line-of-business system, a script. It authenticates with a client id and secret rather than a password, holds exactly the rights it was granted (and never more than the person who granted them held), and appears in the audit log under its own name. It is the supported way to put documents in automatically, and it is why nobody needs to share a human's credentials with a robot.

18.5 The recipient of an external link

Somebody with no account at all, holding a URL. They can open the one document it names — and no other — until the link expires or its permitted number of uses runs out. Nothing else in the archive is reachable from it, and the link can be revoked the moment it turns out it should not have been sent.

18.6 Programs, not people

A file manager	Mounts the archive over WebDAV and shows exactly the tree the workbench shows. Opening, editing and saving a file works the way it does on a network drive.
A phone or a calendar program	Syncs contacts and calendars over CardDAV and CalDAV. What it changes, the archive keeps — and versions.
A mail client	Reads the archive's mail folders over IMAP, and can file a message into the archive by copying it there.
A mail server	Delivers mail straight into a user's Intray, so an address can be filed simply by being written to.

> All four use a **password per device**, issued from **Devices…** in the account menu and revocable one at a time — never the password used to sign in. A lost phone costs one revocation, not a password change.

19 Appendix — how Alcifra is put together

The vocabulary the system actually thinks in. Worth ten minutes if you are evaluating it, integrating with it, or wondering why something behaves the way it does — several of the answers in this manual are consequences of the shape below rather than of any particular screen.

19.1 The document is the spine

Almost everything is a **document**. A document has a name, a place in the tree, permissions of its own, and — usually — content.

A folder	A document with no content. That is the whole difference: folders are not a separate species, which is why a folder has index data, permissions, a chat thread and a history like anything else.
A repository	A document with no parent — a root. Again not a separate species, which is why the same permission rules reach the top of the tree without a special case.
Your own space	A repository belonging to one person, provisioned for them, holding their **Intray** and their checked-out documents. It is why every user has somewhere to put something before deciding where it belongs.

19.2 Content lives in versions

A document's file is a **version**, and versions are immutable. Uploading again adds a new one; the old one stays readable. This is what makes "restore an old version" honest — nothing is overwritten, so nothing has to be recovered.

> **Metadata belongs to the document, content belongs to the version.** Rename a document, re-classify it or edit its index data and the history is untouched; upload a new file and the metadata carries across. That split is why re-filing a document never costs its history, and why a version can be restored without copying anything.

19.3 Document types and index data

A **mask** is a document type: a named set of **index fields** with their types, validation and defaults. Assigning a mask to a document decides what may be recorded about it, and — through the mask — where it may live and what it may contain.

Masks are **versioned and immutable**. Changing one publishes a new version; documents already classified keep the version they were classified under. A field's meaning therefore never changes retrospectively, which is what makes a ten-year-old document still readable on its own terms.

19.4 Who may do what

Permissions are an **access-control list per document** — a set of rights granted to a user, a group or a service account. Rights inherit down the tree until a folder is set to break inheritance, at which point that subtree starts again from what is granted there.

Two rules are worth knowing because they explain refusals:

- **Nobody can grant a right they do not hold.** An administrator of a folder cannot manufacture access to it for someone else beyond their own.
- **A deactivated user and a suspended tenant have no rights at all**, decided before any administrator bypass — so deactivating an account is immediate and total rather than a matter of stripping grants one at a time.

19.5 One installation, many organisations

A **tenant** is an organisation. Every row that belongs to one carries its tenant, and the isolation is enforced in a single place — a filter the database context applies to every query — rather than by each feature remembering. Users, groups, masks, documents, tags and audit events are all tenant-scoped; a **platform administrator** deliberately belongs to no tenant.

19.6 The things that hang off a document

Chat, mentions	The conversation about a document, and the system entries that record what happened to it — a version filed, an older one made current, an attachment refused.
Annotations	Marks on a page, anchored to the version they were drawn on.
References	The same document appearing in a second folder. One document, two places — not a copy, so there is nothing to diverge.
Tags	Free or catalogue-restricted labels, orthogonal to the tree.
Subscriptions, reminders	Who wants to hear about changes, and when somebody wants to be reminded.
External links	Time-limited, use-limited access for somebody with no account.
Legal holds, retention	A freeze that blocks change and deletion, and a policy that eventually disposes of what is no longer needed. A hold always wins.
Workflow	The state a document is in, and the trail of transitions that got it there.
Audit events	What was done, by whom, to what — hash-chained so a gap or an edit is detectable.

19.7 What the archive stores it in

Content goes to object storage, never through the application: the browser and the desktop client transfer bytes directly against a short-lived signed URL. Metadata, permissions and the audit trail go to PostgreSQL. Full-text search is a separate index built from the extracted text, and the archive works without it — search falls back to metadata alone rather than failing.

> Nothing above is peculiar to the web client or the desktop client. Both are views onto the same model, which is why a document filed from a phone’s calendar, a mail server, a file manager and the workbench are the same kind of thing afterwards — and why the permissions on it mean the same thing in all four.

20 Appendix — security posture

A self-assessment against the **OWASP Top 10 (2021)** and a summary against **ASVS Level 2**, for readers who have to answer for what they deploy.

> **What this is, and is not.** It is a structured review of the software by the people who wrote it, kept current with the code. It is **not** a penetration test and not a third-party certification, and it says nothing about any particular installation's network, backups or operating procedures. The distinction the project holds elsewhere applies here too: the product is production software; the public demonstration instance is not, and is documented as such.

20.1 The finding that mattered

The review's most useful result was not any single weakness. It was a **pattern**: the four gaps it ranked highest had all been **decided, written down, and never built**. Response headers, sign-in throttling, an outbound-request guard and an upload content check each had an architecture record specifying them — in one case naming the exact four controls — and none of the four existed in the code. Nothing was visibly broken, which is precisely why they survived: the absence of a preventive control is invisible until the day it is needed.

All four are now built, each with its own record explaining what was decided and what it concedes. The lesson is kept: a decision recorded is not a decision delivered, and only a test can tell the two apart.

20.2 OWASP Top 10 (2021)

A01 Broken access control	Permissions are an access-control list per document, resolved by one calculator that every caller goes through — with group expansion, inheritance and explicit breaks. Nobody can grant a right they do not hold. A deactivated user or suspended organisation has no rights at all, decided **before** any administrator bypass. Multi-tenant isolation is a query filter applied in a single place rather than remembered per feature, and a test suite asserts a tenant cannot reach another's data.
A02 Cryptographic failures	Passwords are stored only as hashes, and never logged. Secrets — database credentials, object-storage keys, token-signing certificates — come from a secrets manager when one is configured, with the database credential issued afresh at every start-up and the application never using the database superuser. Outside development a secret the application must store — a one-time-code seed — is encrypted by the secrets manager, and the start-up gate refuses to run without one rather than storing it in plaintext. Transport security is relaxed only when the application is run in **development** mode: the relaxation is tied to that mode rather than to a setting, so there is no configuration switch that turns it off in a deployed installation.
A03 Injection	Data access is exclusively through a typed query layer: the application contains **no raw or string-concatenated SQL anywhere outside its schema migrations**. Output in both

	clients is framework-escaped, and the response-header policy blocks inline script rather than allowing it. XML from outside is parsed through one helper that prohibits document-type definitions, resolves no external entities and expands none — by assertion, so a changed framework default would not change it.
A04 Insecure design	Progressive throttling of credential guessing at every door that verifies one; an upload content check that refuses executables and scripts by their bytes as well as their names; storage quotas; connection caps on the mail endpoint; page-size clamps on every listing. The archive's own invariants — no cycles in the tree, no duplicate sibling names, immutable document types — are enforced at the single point where data is saved, not by each caller.
A05 Security misconfiguration	A start-up gate refuses to run outside development when any development-grade setting is present — development certificates, bootstrap secrets, default storage credentials, an unstamped build — and there is **no flag to bypass it**. Responses carry a content-security policy, frame and referrer restrictions and content-type pinning; each header is set only if absent, so a deployment that owns its own edge keeps ownership. Forwarded headers from a reverse proxy are honoured only from the proxies a deployment names, and the start-up gate refuses the option with none named.
A06 Vulnerable and outdated components	Every build scans the dependency tree for known vulnerabilities and fails on a finding; the container image is scanned separately; dependency updates are proposed automatically each week. A licence gate additionally fails the build on any dependency outside the allowed set.
A07 Identification and authentication failures	Standards-based sign-in with proof-key exchange for the interactive clients. Second factors are supported as time-based codes and as passkeys, including passwordless sign-in, and an organisation can require them for everybody. A chosen password must be at least twelve characters and must not be, or merely decorate, one of the ten thousand most common breached passwords. Failed attempts are progressively refused per account **and** per source, with blocks that expire on their own rather than needing an administrator. Access tokens live fifteen minutes and refresh tokens thirty days, and a revoked token is refused on its very next request rather than at the end of its lifetime.
A08 Software and data integrity failures	Uploaded content is re-read and hashed by the server rather than trusted from the client. Versions can be locked immutable in storage for a retention period. The audit log is hash-chained, so an altered or missing entry is detectable. Schema migrations are checked for data-destroying operations, and a destructive one must be listed with a reason before the

	build will pass. Published images are signed, carry build provenance and a software bill of materials, and the release notes say how to verify all three.
A09 Security logging and monitoring failures	Changes are audited to an append-only, hash-chained log with a retention policy, write-once archival segments, export, and signed streaming to a SIEM — including a refused sign-in, a sign-out and an account lockout. Coverage is **not** claimed to be complete: a build check fails when a new change-making endpoint records nothing, and the endpoints that still record nothing are a listed, justified backlog rather than an assumption. Requests carry a correlation identifier end to end, and every external seam can be traced in full detail when needed. Logs never contain a password, a token or a signed URL.
A10 Server-side request forgery	The two places that accept a caller-supplied URL — an organisation's audit webhook and a client's push endpoint — are checked when the URL is registered **and again at the moment of connecting**, against the address actually resolved, connecting to that address so nothing can be re-pointed in between. Redirects are refused. Private ranges are refused unless the deployment's own configuration permits them, and the cloud metadata addresses are refused regardless.

20.3 ASVS Level 2, in summary

Architecture	Layering is enforced by tests rather than convention; every architectural decision is recorded, including the ones later reversed.
Authentication	Standards-based, second factors supported and enforceable, throttled; chosen passwords meet a length and breach-list policy; failed and throttled sign-ins reach the audit trail.
Session management	Server-issued tokens with bounded lifetimes — fifteen minutes for access, thirty days for refresh. Signing out revokes that client's tokens on the server, and every request checks its token against the server's record, so a revoked token stops working at once. A refresh token used a second time is treated as stolen: the whole session it belongs to is revoked, the access token included.
Access control	Per-object, deny by default, and evaluated **server-side on every request** — the clients never decide it. They additionally hide an action the server did not advertise, so a user is rarely offered something that would be refused; but the hiding is courtesy, and the refusal is the control.
Validation & encoding	Typed data access, framework-escaped output, content checked on upload, and uploads that never pass through the application.
Logging	Structured, correlated, redaction-conscious; the audit trail is separate from the operational log and is tamper-evident.

Data protection	Content in object storage reached only by short-lived signed URLs; deletion is a two-stage process with a recycle bin, and disposal is auditable.
Communications	Transport security enforced outside development; the demonstration stack's relaxations are development-only by construction.
Configuration	Dependency, licence, secret and image scanning on every build; a start-up gate that refuses development settings in production.

20.4 What is still open

An assessment that reports nothing outstanding is one to distrust. This is tracked, and it is not a remotely exploitable defect in the shipped configuration. Each names the issue that tracks it, and a weekly check fails when one of those issues closes while its line is still here — because this list once went on describing a product that had moved on, in the one chapter a reader opens to find out what is not done.

- **External-link tokens at rest** (#1579) — the token behind a link shared outside the system is stored as issued, so reading it requires database access. Hashing it would remove the ability to show an existing link again, which is why it is a decision rather than a fix.

Recently closed, and listed so that their absence above is not mistaken for an omission: a revoked or signed-out token is refused on its next request, and a refresh token used twice revokes its whole session; chosen passwords must meet a minimum length and are checked against a list of commonly used ones; token lifetimes are set deliberately rather than left at framework defaults; a production installation refuses to start without a secrets manager, so second-factor seeds and the other stored secrets are never kept in plaintext; refused sign-ins and lockouts are audit events; published images are signed and carry provenance and a bill of materials; XML is parsed with external entities refused by assertion, and a test fails if a parser is added without it; and an installation behind a reverse proxy must name the proxies it trusts.

20.5 How this stays true

Every one of the claims above is held by something that runs on each change: a test, a gate, or both. The build fails on a dependency with a known vulnerability, on a licence outside the allowed set, on a leaked secret, on a destructive migration without a stated reason, and on a compiler warning. The rest is held by roughly two thousand automated tests, including end-to-end suites that drive the real application over HTTP and a real browser. That is the only reason a document like this one can be published rather than merely written.

21 Index

Terms are indexed where the manual explains them, not at every mention — the page listed is the one worth turning to.

ACL 2
Annotations 41
apass 27
Approval workflow 48
Attestation 64
Audience 27
Audit trail 49
Auto-rotate 10
Auto-straighten (deskew) 10
Availability 22
Block 22
Blockers 22
Booking 21
bookmarks 28
browser extension 28
CalDAV 20, 36
Calendar 17
CardDAV 20, 36
Certificate policy 65
Chat / comments 41
chat, not sealed 54
Check-out 6, 42
CI/CD store 29
Click-to-copy 4
Contact card 4
Contacts 17
Context menu 37
Desktop client 1
Device password 36, 73
Document management system 1
Edit profile 49
Encrypted content 51
Encryption (S/MIME) 16
Encryption Module 54
Encryption Service 54
Enrolling a certificate 63
Executable content 7
External link 43
External link (encrypted) 45
Facets 40
Follow / subscribe 41
Hardware key test 60
Hit highlighting 4
IMAP 15, 36
Import / export 6
Index fields 2
Intray 6, 15
Join items 9
key, enrolled for you 53
Legal hold 48
Mail card 4
Mask (document type) 2
Meeting room 21
Messenger 23
Multi-factor authentication 3
My work 41
Name conflict 7
Notes 16
Notifications 41
OCR 39
Offered time 22
Open a document 4
Outbound mail 50
Partial-word search 40
Password store 25
Personal repository 12
Preview 3
Quantum 55
Recycle bin 48
Reference 15
Reference field 39
Reference (shortcut) 12
Reminder 41
Repository 2
Retention 48
Revoking a key 64
Rights 2
Rotate pages 9
Rotate/Sort 9, 11
Schedule 22
Search 40
Sensitivity label 12
Separator sheets 9
Server manager 50
Sign-in throttling 3
Smartcard 16
Sort pages 9
Sovereign workspace 1
Split into pages 9
Storage quota 49
Tablet 29
Tags 12
Task count 48
Tasks 48
Tenant 49
Two-factor authentication 49
Upload 6
Users & groups 49
Verification code 26
Version 2
Version comment 8
Web client 1
WebDAV 13, 36
WebDAV deep link 13
Working copy 42
Zoom 4